

```

1. def rank_samples(samples: list[dict]) -> list[dict]:
2.     def score(s: dict) -> float:
3.         risk = 0
4.         risk += 35 if s.get('is_low_confidence') else 0
5.         risk += 30 if '质检异常' in (s.get('status') or '') else 0
6.         risk += 20 * int(s.get('rework_count') or 0)
7.         risk += 25 if s.get('is_duplicate') else 0
8.         risk += 40 if s.get('is_ground_truth') else 0
9.         risk += 10 if '低置信' in (s.get('risk_tags') or '') else 0
10.        return risk
11.    return sorted(samples, key=score, reverse=True)
12. from app.algorithms.iou import bbox_iou
13. def check_bbox(annotation: dict, image_w: int, image_h: int) -> list[dict]:
14.     issues = []
15.     x, y, w, h = float(annotation['x']), float(annotation['y']), float(annotation['w']...
16.     area = w * h
17.     image_area = max(1, image_w * image_h)
18.     if x < 0 or y < 0 or x + w > image_w or y + h > image_h:
19.         issues.append({"type": "标注框越界", "severity": "高", "rule": "BBoxBoundary", "pos...
20.     if area < image_area * 0.002:
21.         issues.append({"type": "标注框过小", "severity": "中", "rule": "MinBoxArea", "posit...
22.     if area > image_area * 0.65:
23.         issues.append({"type": "标注框过大", "severity": "中", "rule": "MaxBoxArea", "posit...
24.     if min(abs(x), abs(y), abs(image_w - (x + w)), abs(image_h - (y + h))) <= 2:
25.         issues.append({"type": "异常贴边框", "severity": "低", "rule": "EdgeTouch", "positi...
26.     if w <= 0 or h <= 0:
27.         issues.append({"type": "无效标注框", "severity": "高", "rule": "NonPositiveBox", "p...
28.     ratio = max(w / max(h, 1), h / max(w, 1))
29.     if ratio > 8:
30.         issues.append({"type": "长宽比异常", "severity": "低", "rule": "AspectRatio", "posi...
31.    return issues
32. def check_duplicate_boxes(annotations: list[dict]) -> list[dict]:
33.     issues = []
34.     for i, a in enumerate(annotations):
35.         for b in annotations[i + 1:]:
36.             score = bbox_iou(a, b)
37.             if score > 0.82 and a.get('label') == b.get('label'):
38.                 issues.append({"type": "同目标重复框", "severity": "高", "rule": "DuplicateI...
39.             elif score > 0.65:
40.                 issues.append({"type": "高度重叠框", "severity": "中", "rule": "OverlapIoU"...
41.    return issues
42. from math import log2
43. def label_balance_score(label_counts: dict[str, int]) -> float:
44.     total = sum(label_counts.values())
45.     if not total or len(label_counts) < 2:
46.         return 60.0
47.     probs = [c / total for c in label_counts.values() if c > 0]
48.     entropy = -sum(p * log2(p) for p in probs)
49.     max_entropy = log2(len(label_counts))
50.     return round(100 * entropy / max_entropy, 2) if max_entropy else 60.0

```

```

51. def compute_health(metrics: dict) -> tuple[float, list[str]]:
52.     complete = metrics.get('annotated_rate', 0)
53.     auto_pass = metrics.get('auto_pass_rate', 0)
54.     manual_pass = metrics.get('manual_pass_rate', 0)
55.     consensus = metrics.get('consensus_score', 0)
56.     gt = metrics.get('gt_score', 0)
57.     balance = metrics.get('balance_score', 0)
58.     duplicate_penalty = metrics.get('duplicate_rate', 0) * 100
59.     rework_penalty = metrics.get('rework_rate', 0) * 80
60.     score = complete * 0.20 + auto_pass * 0.15 + manual_pass * 0.15 + consensus * 0.1...
61.     score = max(0, min(100, score - duplicate_penalty * 0.08 - rework_penalty * 0.10))
62.     deductions = []
63.     if complete < 95:
64.         deductions.append('标注完整度不足, 建议优先完成待标注样本。')
65.     if auto_pass < 90:
66.         deductions.append('自动质检通过率偏低, 需处理越界框、重复框和空标签。')
67.     if consensus < 85:
68.         deductions.append('多人一致性不足, 建议增加仲裁和规范校准。')
69.     if balance < 75:
70.         deductions.append('标签分布不均衡, 建议补充少数类样本。')
71.     if metrics.get('rework_rate', 0) > 0.08:
72.         deductions.append('返工率偏高, 建议复盘高频错误标签。')
73.     return round(score, 2), deductions
74. import hashlib
75. from pathlib import Path
76. from PIL import Image
77. def md5_file(path: str | Path) -> str:
78.     p = Path(path)
79.     h = hashlib.md5()
80.     with p.open('rb') as f:
81.         for chunk in iter(lambda: f.read(8192), b''):
82.             h.update(chunk)
83.     return h.hexdigest()
84. def average_hash(path: str | Path, size: int = 8) -> str:
85.     img = Image.open(path).convert('L').resize((size, size))
86.     pixels = list(img.getdata())
87.     avg = sum(pixels) / len(pixels)
88.     bits = ''.join('1' if p > avg else '0' for p in pixels)
89.     return f"{int(bits, 2):0{size * size // 4}x}"
90. def hamming_hex(a: str, b: str) -> int:
91.     return bin(int(a, 16) ^ int(b, 16)).count('1')
92. def bbox_iou(a: dict, b: dict) -> float:
93.     ax1, ay1, ax2, ay2 = float(a['x']), float(a['y']), float(a['x'] + a['w']), float(...)
94.     bx1, by1, bx2, by2 = float(b['x']), float(b['y']), float(b['x'] + b['w']), float(...)
95.     ix1, iy1 = max(ax1, bx1), max(ay1, by1)
96.     ix2, iy2 = min(ax2, bx2), min(ay2, by2)
97.     iw, ih = max(0.0, ix2 - ix1), max(0.0, iy2 - iy1)
98.     inter = iw * ih
99.     area_a = max(0.0, ax2 - ax1) * max(0.0, ay2 - ay1)
100.     area_b = max(0.0, bx2 - bx1) * max(0.0, by2 - by1)

```

```

101.         union = area_a + area_b - inter
102.         return inter / union if union else 0.0
103. import re
104. from difflib import SequenceMatcher
105. def normalize_text(text: str) -> str:
106.     return re.sub(r"\s+", "", (text or "").lower())
107. def similarity(a: str, b: str) -> float:
108.     return SequenceMatcher(None, normalize_text(a), normalize_text(b)).ratio()
109. def check_text_sample(text: str, labels: list[str] | None = None, reason: str = "") -...
110.     issues = []
111.     raw = text or ""
112.     if not raw.strip():
113.         issues.append({"type": "文本为空", "severity": "高", "rule": "EmptyText", "positio...
114.     if 0 < len(raw.strip()) < 8:
115.         issues.append({"type": "文本过短", "severity": "中", "rule": "ShortText", "positio...
116.     if labels is not None and not labels:
117.         issues.append({"type": "标签为空", "severity": "高", "rule": "EmptyLabel", "positi...
118.     if labels and "退款" in labels and "物流" in labels:
119.         issues.append({"type": "标签冲突", "severity": "中", "rule": "ExclusiveIntent", "p...
120.     if reason is not None and len(reason.strip()) < 10:
121.         issues.append({"type": "评价理由过短", "severity": "低", "rule": "ReasonLength", "po...
122.     return issues
123. from pathlib import Path
124. APP_NAME = "人工智能模型训练样本智能标注与质量校验管理软件"
125. BASE_DIR = Path(__file__).resolve().parents[2]
126. DATA_DIR = BASE_DIR / "data"
127. DB_PATH = DATA_DIR / "app.db"
128. DEMO_DIR = DATA_DIR / "demo"
129. IMAGE_DIR = DEMO_DIR / "images"
130. TEXT_DIR = DEMO_DIR / "texts"
131. ANNOTATION_DIR = DEMO_DIR / "annotations"
132. IMPORT_DIR = DEMO_DIR / "imports"
133. EXPORT_DIR = DEMO_DIR / "exports"
134. DOCS_DIR = BASE_DIR / "docs"
135. for p in [DATA_DIR, DEMO_DIR, IMAGE_DIR, TEXT_DIR, ANNOTATION_DIR, IMPORT_DIR, EXPORT...
136.     p.mkdir(parents=True, exist_ok=True)
137. import sqlite3
138. from contextlib import contextmanager
139. from typing import Iterable, Any
140. from app.core.config import DB_PATH, DATA_DIR
141. SCHEMA = r"""
142. CREATE TABLE IF NOT EXISTS users (
143.     id INTEGER PRIMARY KEY AUTOINCREMENT,
144.     username TEXT UNIQUE NOT NULL,
145.     password TEXT NOT NULL,
146.     display_name TEXT NOT NULL,
147.     role TEXT NOT NULL,
148.     enabled INTEGER DEFAULT 1,
149.     created_at TEXT DEFAULT CURRENT_TIMESTAMP
150. );

```

```

151. CREATE TABLE IF NOT EXISTS dataset_projects (
152.     id INTEGER PRIMARY KEY AUTOINCREMENT,
153.     code TEXT UNIQUE,
154.     name TEXT NOT NULL,
155.     project_type TEXT,
156.     data_type TEXT,
157.     task_type TEXT,
158.     training_goal TEXT,
159.     owner TEXT,
160.     reviewer TEXT,
161.     status TEXT,
162.     deadline TEXT,
163.     version_no TEXT,
164.     sample_count INTEGER DEFAULT 0,
165.     label_count INTEGER DEFAULT 0,
166.     health_score REAL DEFAULT 0,
167.     created_at TEXT DEFAULT CURRENT_TIMESTAMP
168. );
169. CREATE TABLE IF NOT EXISTS samples (
170.     id INTEGER PRIMARY KEY AUTOINCREMENT,
171.     project_id INTEGER,
172.     sample_code TEXT,
173.     sample_type TEXT,
174.     filename TEXT,
175.     file_path TEXT,
176.     text_content TEXT,
177.     width INTEGER DEFAULT 0,
178.     height INTEGER DEFAULT 0,
179.     status TEXT DEFAULT '未开始',
180.     risk_tags TEXT DEFAULT '',
181.     is_ground_truth INTEGER DEFAULT 0,
182.     is_duplicate INTEGER DEFAULT 0,
183.     is_low_confidence INTEGER DEFAULT 0,
184.     rework_count INTEGER DEFAULT 0,
185.     assigned_to TEXT,
186.     qc_status TEXT DEFAULT '未质检',
187.     created_at TEXT DEFAULT CURRENT_TIMESTAMP,
188.     FOREIGN KEY(project_id) REFERENCES dataset_projects(id)
189. );
190. CREATE TABLE IF NOT EXISTS labels (
191.     id INTEGER PRIMARY KEY AUTOINCREMENT,
192.     project_id INTEGER,
193.     name TEXT,
194.     code TEXT,
195.     color TEXT,
196.     label_type TEXT,
197.     shortcut TEXT,
198.     required INTEGER DEFAULT 0,
199.     exclusive INTEGER DEFAULT 0,
200.     enabled INTEGER DEFAULT 1,

```

```

201.     description TEXT,
202.     positive_example TEXT,
203.     negative_example TEXT,
204.     note TEXT,
205.     FOREIGN KEY(project_id) REFERENCES dataset_projects(id)
206. );
207. CREATE TABLE IF NOT EXISTS annotation_rules (
208.     id INTEGER PRIMARY KEY AUTOINCREMENT,
209.     project_id INTEGER,
210.     rule_type TEXT,
211.     title TEXT,
212.     content TEXT,
213.     severity TEXT DEFAULT '中',
214.     enabled INTEGER DEFAULT 1,
215.     FOREIGN KEY(project_id) REFERENCES dataset_projects(id)
216. );
217. CREATE TABLE IF NOT EXISTS annotations (
218.     id INTEGER PRIMARY KEY AUTOINCREMENT,
219.     sample_id INTEGER,
220.     label TEXT,
221.     annotation_type TEXT DEFAULT 'bbox',
222.     x REAL DEFAULT 0,
223.     y REAL DEFAULT 0,
224.     w REAL DEFAULT 0,
225.     h REAL DEFAULT 0,
226.     entity_start INTEGER DEFAULT 0,
227.     entity_end INTEGER DEFAULT 0,
228.     entity_text TEXT DEFAULT '',
229.     confidence REAL DEFAULT 1,
230.     source TEXT DEFAULT '人工',
231.     status TEXT DEFAULT '已确认',
232.     created_by TEXT DEFAULT '',
233.     comment TEXT DEFAULT '',
234.     updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
235.     FOREIGN KEY(sample_id) REFERENCES samples(id)
236. );
237. CREATE TABLE IF NOT EXISTS quality_issues (
238.     id INTEGER PRIMARY KEY AUTOINCREMENT,
239.     sample_id INTEGER,
240.     annotation_id INTEGER,
241.     issue_type TEXT,
242.     severity TEXT,
243.     rule_name TEXT,
244.     position_text TEXT,
245.     suggestion TEXT,
246.     status TEXT DEFAULT '待处理',
247.     created_at TEXT DEFAULT CURRENT_TIMESTAMP,
248.     FOREIGN KEY(sample_id) REFERENCES samples(id)
249. );
250. CREATE TABLE IF NOT EXISTS consensus_results (

```

```

251.         id INTEGER PRIMARY KEY AUTOINCREMENT,
252.         project_id INTEGER,
253.         sample_id INTEGER,
254.         worker_a TEXT,
255.         worker_b TEXT,
256.         iou_score REAL,
257.         label_agreement REAL,
258.         diff_summary TEXT,
259.         need_arbitration INTEGER DEFAULT 0,
260.         created_at TEXT DEFAULT CURRENT_TIMESTAMP
261.     );
262. CREATE TABLE IF NOT EXISTS ground_truth (
263.     id INTEGER PRIMARY KEY AUTOINCREMENT,
264.     sample_id INTEGER,
265.     answer_json TEXT,
266.     score REAL DEFAULT 100,
267.     conclusion TEXT,
268.     created_by TEXT,
269.     created_at TEXT DEFAULT CURRENT_TIMESTAMP
270. );
271. CREATE TABLE IF NOT EXISTS rework_tasks (
272.     id INTEGER PRIMARY KEY AUTOINCREMENT,
273.     rework_code TEXT UNIQUE,
274.     sample_id INTEGER,
275.     project_id INTEGER,
276.     labeler TEXT,
277.     reviewer TEXT,
278.     issue_type TEXT,
279.     issue_desc TEXT,
280.     requirement TEXT,
281.     deadline TEXT,
282.     status TEXT DEFAULT '待返工',
283.     second_review TEXT DEFAULT '',
284.     arbitration_result TEXT DEFAULT '',
285.     created_at TEXT DEFAULT CURRENT_TIMESTAMP
286. );
287. CREATE TABLE IF NOT EXISTS dataset_versions (
288.     id INTEGER PRIMARY KEY AUTOINCREMENT,
289.     project_id INTEGER,
290.     version_no TEXT,
291.     sample_total INTEGER,
292.     passed_total INTEGER,
293.     qc_pass_rate REAL,
294.     train_ratio REAL,
295.     val_ratio REAL,
296.     test_ratio REAL,
297.     status TEXT,
298.     frozen_by TEXT,
299.     frozen_at TEXT,
300.     description TEXT,

```

```

301.         diff_from_prev TEXT
302.     );
303. CREATE TABLE IF NOT EXISTS export_records (
304.     id INTEGER PRIMARY KEY AUTOINCREMENT,
305.     project_id INTEGER,
306.     format TEXT,
307.     output_path TEXT,
308.     check_result TEXT,
309.     created_by TEXT,
310.     created_at TEXT DEFAULT CURRENT_TIMESTAMP
311. );
312. CREATE TABLE IF NOT EXISTS reports (
313.     id INTEGER PRIMARY KEY AUTOINCREMENT,
314.     project_id INTEGER,
315.     report_type TEXT,
316.     title TEXT,
317.     file_path TEXT,
318.     conclusion TEXT,
319.     created_by TEXT,
320.     created_at TEXT DEFAULT CURRENT_TIMESTAMP
321. );
322. CREATE TABLE IF NOT EXISTS operation_logs (
323.     id INTEGER PRIMARY KEY AUTOINCREMENT,
324.     username TEXT,
325.     action TEXT,
326.     detail TEXT,
327.     created_at TEXT DEFAULT CURRENT_TIMESTAMP
328. );
329. """
330. def _connect() -> sqlite3.Connection:
331.     DATA_DIR.mkdir(parents=True, exist_ok=True)
332.     conn = sqlite3.connect(str(DB_PATH))
333.     conn.row_factory = sqlite3.Row
334.     conn.execute("PRAGMA foreign_keys = ON")
335.     return conn
336. @contextmanager
337. def get_conn():
338.     conn = _connect()
339.     try:
340.         yield conn
341.         conn.commit()
342.     finally:
343.         conn.close()
344. def init_db() -> None:
345.     with get_conn() as conn:
346.         conn.executescript(SCHEMA)
347. def fetch_all(sql: str, params: Iterable[Any] = ()) -> list[dict]:
348.     with get_conn() as conn:
349.         cur = conn.execute(sql, tuple(params))
350.         return [dict(row) for row in cur.fetchall()]

```

```

351. def fetch_one(sql: str, params: Iterable[Any] = ()) -> dict | None:
352.     with get_conn() as conn:
353.         cur = conn.execute(sql, tuple(params))
354.         row = cur.fetchone()
355.         return dict(row) if row else None
356. def execute(sql: str, params: Iterable[Any] = ()) -> int:
357.     with get_conn() as conn:
358.         cur = conn.execute(sql, tuple(params))
359.         return int(cur.lastrowid)
360. def execute_many(sql: str, values: list[Iterable[Any]]) -> None:
361.     with get_conn() as conn:
362.         conn.executemany(sql, [tuple(v) for v in values])
363. def log_action(username: str, action: str, detail: str) -> None:
364.     execute("INSERT INTO operation_logs(username, action, detail) VALUES(?,?,?)", (us...
365.     ROLE_MENUS = {
366.         "管理员": "all",
367.         "项目经理": ["数据集驾驶舱", "数据集项目管理", "质量校验中心", "多人一致性分析", "Ground Truth 抽检", "返工闭环管理",
368.         "...
369.         "标注员": ["数据集驾驶舱", "样本标注生产工作台", "文本标注工作台", "返工闭环管理"],
370.         "质检员": ["数据集驾驶舱", "质量校验中心", "多人一致性分析", "Ground Truth 抽检", "返工闭环管理", "报告生成中心"],
371.     }
372. def can_access(role: str, menu_name: str) -> bool:
373.     allowed = ROLE_MENUS.get(role, [])
374.     return allowed == "all" or menu_name in allowed
375. import json
376. import random
377. from datetime import datetime, timedelta
378. from pathlib import Path
379. from PIL import Image, ImageDraw, ImageFilter
380. from app.core.config import BASE_DIR, IMAGE_DIR, TEXT_DIR, IMPORT_DIR, EXPORT_DIR, DO...
381. from app.core.database import fetch_one, execute, execute_many
382. def _rel(path: Path) -> str:
383.     try:
384.         return str(path.relative_to(BASE_DIR))
385.     except ValueError:
386.         return str(path)
387. def _ensure_dirs():
388.     for p in [IMAGE_DIR, TEXT_DIR, IMPORT_DIR, EXPORT_DIR, DOCS_DIR]:
389.         p.mkdir(parents=True, exist_ok=True)
390. def _demo_image(path: Path, idx: int, blur: bool = False) -> tuple[int, int]:
391.     w, h = 1120, 680
392.     img = Image.new('RGB', (w, h), (214, 229, 245))
393.     d = ImageDraw.Draw(img)
394.     d.rectangle([0, 0, w, 380], fill=(203, 221, 238))
395.     d.rectangle([0, 380, w, h], fill=(190, 184, 170))
396.     for x in range(60, w, 135):
397.         d.rectangle([x, 80, x + 14, 500], fill=(132, 144, 158))
398.     for y in range(120, 480, 80):
399.         d.rectangle([40, y, w - 70, y + 12], fill=(132, 144, 158))
400.     d.line([720, 60, 1010, 130], fill=(211, 138, 43), width=10)

```



```
400.         for x in range(40, 360, 70):
```

```

401.         d.rectangle([x, 548, x + 48, 584], fill=(241, 176, 33))
402.         d.line([x, 584, x + 48, 548], fill=(55, 65, 81), width=4)
403.     rng = random.Random(1000 + idx)
404.     for i in range(4 + idx % 3):
405.         x = rng.randint(120, 930)
406.         y = rng.randint(385, 535)
407.         scale = rng.uniform(0.85, 1.2)
408.         body_h = int(85 * scale)
409.         body_w = int(38 * scale)
410.         helmet = not (i == 0 and idx % 3 == 0)
411.         vest = i % 2 == 0
412.         d.line([x - 12, y + body_h, x - 22, y + body_h + 54], fill=(47, 63, 85), width=4)
413.         d.line([x + 12, y + body_h, x + 24, y + body_h + 54], fill=(47, 63, 85), width=4)
414.         d.rectangle([x - body_w // 2, y + 30, x + body_w // 2, y + body_h], fill=(232, 180, 24))
415.         d.ellipse([x - 17, y, x + 17, y + 34], fill=(184, 124, 84))
416.         if helmet:
417.             d.pieslice([x - 22, y - 7, x + 22, y + 22], 180, 360, fill=(246, 199, 42))
418.             d.rectangle([x - 22, y + 8, x + 22, y + 15], fill=(232, 180, 24))
419.         d.rectangle([770, 486, 1035, 580], fill=(92, 111, 128))
420.         d.rectangle([810, 445, 930, 510], fill=(110, 142, 170))
421.         d.ellipse([810, 555, 865, 610], fill=(35, 44, 55))
422.         d.ellipse([965, 555, 1020, 610], fill=(35, 44, 55))
423.         d.polygon([(530, 460), (585, 558), (475, 558)], fill=(245, 158, 11))
424.         d.rectangle([525, 515, 536, 546], fill=(31, 41, 55))
425.         for _ in range(600):
426.             x = rng.randint(0, w - 1); y = rng.randint(0, h - 1); c = rng.randint(-18, 18)
427.             r, g, b = img.getpixel((x, y))
428.             img.putpixel((x, y), (max(0, min(255, r + c)), max(0, min(255, g + c)), max(0, min(255, b + c))))
429.         if blur:
430.             img = img.filter(ImageFilter.GaussianBlur(2.3))
431.         img.save(path, quality=92)
432.         return w, h
433. def _traffic_image(path: Path, idx: int, rainy: bool = False) -> tuple[int, int]:
434.     w, h = 1280, 720
435.     img = Image.new('RGB', (w, h), (202, 216, 226))
436.     d = ImageDraw.Draw(img)
437.     d.rectangle([0, 0, w, 260], fill=(185, 207, 225))
438.     d.rectangle([0, 260, w, h], fill=(92, 100, 110))
439.     d.polygon([(450, 260), (830, 260), (1040, 720), (240, 720)], fill=(60, 66, 75))
440.     for lane in [515, 640, 765]:
441.         for y in range(300, 700, 90):
442.             offset = int((y - 260) * 0.18)
443.             d.rectangle([lane - 5 - offset, y, lane + 5 - offset, y + 45], fill=(238, 180, 24))
444.     rng = random.Random(3000 + idx)
445.     for i in range(4 + idx % 4):
446.         x = rng.randint(180, 940)
447.         y = rng.randint(330, 610)
448.         ww = rng.randint(86, 160); hh = rng.randint(42, 74)
449.         color = [(37, 99, 235), (239, 68, 68), (22, 163, 74), (245, 158, 11)][i % 4]
450.         d.rounded_rectangle([x, y, x + ww, y + hh], radius=12, fill=color)

```

```

451.         d.rectangle([x + 14, y + 8, x + ww - 18, y + 30], fill=(180, 215, 235))
452.         d.ellipse([x + 12, y + hh - 14, x + 42, y + hh + 16], fill=(24, 32, 45))
453.         d.ellipse([x + ww - 42, y + hh - 14, x + ww - 12, y + hh + 16], fill=(24, 32,...
454.     for i in range(3):
455.         x = 120 + i * 160 + rng.randint(-25, 25); y = 430 + rng.randint(-15, 50)
456.         d.ellipse([x - 12, y - 28, x + 12, y - 4], fill=(166, 113, 76))
457.         d.rectangle([x - 14, y, x + 14, y + 56], fill=(31, 94, 141))
458.         d.line([x - 8, y + 56, x - 18, y + 94], fill=(31, 41, 55), width=6)
459.         d.line([x + 8, y + 56, x + 20, y + 94], fill=(31, 41, 55), width=6)
460.     # cyclist
461.     cx, cy = 910 + rng.randint(-30, 30), 520 + rng.randint(-20, 20)
462.     d.ellipse([cx - 42, cy + 36, cx + 2, cy + 80], outline=(25, 32, 44), width=5)
463.     d.ellipse([cx + 46, cy + 36, cx + 90, cy + 80], outline=(25, 32, 44), width=5)
464.     d.line([cx - 20, cy + 58, cx + 40, cy + 20, cx + 68, cy + 58, cx - 20, cy + 58], ...
465.     d.ellipse([cx + 30, cy - 18, cx + 55, cy + 6], fill=(166, 113, 76))
466.     d.rectangle([cx + 25, cy + 8, cx + 55, cy + 42], fill=(234, 88, 12))
467.     # traffic lights / road signs
468.     d.rectangle([1030, 175, 1050, 455], fill=(55, 65, 81))
469.     d.rounded_rectangle([1005, 145, 1074, 275], radius=12, fill=(28, 36, 50))
470.     for k, col in enumerate([(220, 38, 38), (245, 158, 11), (34, 197, 94)]):
471.         d.ellipse([1025, 160 + k * 36, 1052, 187 + k * 36], fill=col)
472.     d.polygon([(1115, 330), (1172, 365), (1115, 400), (1058, 365)], fill=(245, 158, 11))
473.     if rainy:
474.         for _ in range(220):
475.             x = rng.randint(0, w); y = rng.randint(0, h)
476.             d.line([x, y, x + 8, y + 28], fill=(225, 235, 245), width=1)
477.             img = img.filter(ImageFilter.GaussianBlur(0.8))
478.         img.save(path, quality=92)
479.     return w, h
480. def _defect_image(path: Path, idx: int) -> tuple[int, int]:
481.     w, h = 1040, 720
482.     img = Image.new('RGB', (w, h), (236, 240, 244))
483.     d = ImageDraw.Draw(img)
484.     d.rectangle([0, 540, w, h], fill=(214, 220, 228))
485.     rng = random.Random(5000 + idx)
486.     # workbench table and product tray
487.     d.rounded_rectangle([95, 105, 945, 590], radius=28, fill=(225, 231, 238), outline...
488.     d.rounded_rectangle([150, 160, 890, 530], radius=22, fill=(248, 250, 252), outlin...
489.     for i in range(3):
490.         x = 220 + i * 230 + rng.randint(-10, 10)
491.         y = 260 + rng.randint(-15, 20)
492.         d.rounded_rectangle([x, y, x + 135, y + 145], radius=18, fill=(218, 230, 244)...
493.         d.rectangle([x + 26, y + 26, x + 109, y + 115], fill=(191, 219, 254))
494.     # defects
495.     if idx % 4 in (0, 1):
496.         x, y = 250 + rng.randint(0, 420), 220 + rng.randint(0, 210)
497.         for off in range(0, 50, 9):
498.             d.line([x + off, y + rng.randint(-4, 4), x + off + 38, y + 8 + rng.randin...
499.     if idx % 3 == 0:
500.         x, y = 450 + rng.randint(-60, 120), 300 + rng.randint(-40, 70)

```

```

501.         d.ellipse([x, y, x + 70, y + 42], fill=(148, 163, 184), outline=(71, 85, 105)...
502.     if idx % 5 == 2:
503.         x, y = 650 + rng.randint(-60, 70), 215 + rng.randint(-10, 90)
504.         d.ellipse([x, y, x + 95, y + 55], fill=(161, 98, 7))
505.     if idx in (5, 11):
506.         img = img.filter(ImageFilter.GaussianBlur(1.6))
507.     img.save(path, quality=92)
508.     return w, h
509. def _real_image_group(folder: str) -> list[tuple[Path, int, int]]:
510.     """Load packaged realistic demo images by category.
511.     The project now ships real-world style construction, traffic and industrial
512.     inspection images instead of synthetic low-fidelity drawings. When the
513.     database is recreated, samples point to these category folders directly.
514.     """
515.     files = sorted((IMAGE_DIR / folder).glob("*.jpg"))
516.     rows: list[tuple[Path, int, int]] = []
517.     for path in files:
518.         with Image.open(path) as img:
519.             rows.append((path, img.width, img.height))
520.     return rows
521. def _write_demo_files():
522.     _ensure_dirs()
523.     construction = _real_image_group("construction")
524.     traffic = _real_image_group("traffic")
525.     defects = _real_image_group("industrial")
526.     texts = [
527.         {"id": "T001", "text": "我申请退款已经三天了，订单还是没有处理，请尽快帮我核实。", "label": "退款", "reason...
528.         {"id": "T002", "text": "快递显示签收但我没有收到，能不能帮我查一下物流？", "label": "物流", "reason": "...
529.         {"id": "T003", "text": "收到的商品破损严重，我要投诉并要求售后处理。", "label": "投诉", "reason": "投诉...
530.         {"id": "T004", "text": "这个型号还有黑色库存吗？什么时候可以发货？", "label": "咨询", "reason": "商品咨...
531.         {"id": "T005", "text": "", "label": "其他", "reason": "空文本演示"},
532.         {"id": "T006", "text": "物流一直不更新，快递停在中转站两天了。", "label": "退款", "reason": "标注存在分...
533.         {"id": "T007", "text": "我申请退款已经三天了，订单还是没有处理，请尽快帮我核实。", "label": "售后", "reason...
534.         {"id": "T008", "text": "客服答非所问，处理态度非常差。", "label": "投诉", "reason": "差评投诉"},
535.         {"id": "T009", "text": "可以开发票吗？电子票和纸质票都支持吗？", "label": "咨询", "reason": "票据咨询"},
536.         {"id": "T010", "text": "退", "label": "退款", "reason": "文本过短"},
537.         {"id": "T011", "text": "售后维修寄回后多久能完成检测？", "label": "售后", "reason": "维修进度"},
538.         {"id": "T012", "text": "我想换大一号尺码，能直接换货吗？", "label": "售后", "reason": "换货售后"},
539.         {"id": "T013", "text": "优惠券为什么结算时不能使用？", "label": "咨询", "reason": "活动规则咨询"},
540.         {"id": "T014", "text": "配送地址填错了，现在还能改吗？", "label": "物流", "reason": "配送信息变更"},
541.         {"id": "T015", "text": "收到的颜色和页面展示不一致，怀疑发错货。", "label": "售后", "reason": "商品异常"},
542.         {"id": "T016", "text": "我要取消订单并退款。", "label": "退款", "reason": "退款取消"},
543.         {"id": "T017", "text": "你们平台泄露我的手机号，我要投诉。", "label": "投诉", "reason": "隐私投诉"},
544.         {"id": "T018", "text": "保修期是多久？电池是否单独保修？", "label": "咨询", "reason": "商品保修咨询"},
545.         {"id": "T019", "text": "包裹已经到驿站但取件码没有收到。", "label": "物流", "reason": "取件问题"},
546.         {"id": "T020", "text": "质量太差，要求赔偿。", "label": "投诉", "reason": "质量投诉"},
547.         {"id": "T021", "text": "退货地址发给我。", "label": "退款", "reason": "退款/退货"},
548.         {"id": "T022", "text": "商品参数页面看不懂，适合老人用吗？", "label": "咨询", "reason": "使用咨询"},
549.     ]
550.     csv_lines = ['id,text,label,reason'] + [f'{t["id"]},{t["text"]},{t["label"]},{t["...

```

```

551. (TEXT_DIR / "客服问答意图分类样本.csv").write_text("\n".join(csv_lines), encoding='utf-8')
552. (TEXT_DIR / "llm_preference_samples.jsonl").write_text("\n".join(json.dumps({
553.     "prompt": f"请回答用户问题: {t['text']}",
554.     "response_a": "您好, 已为您记录问题, 我会根据订单状态给出下一步处理建议。",
555.     "response_b": "不知道, 自己看订单。",
556.     "preference": "A",
557.     "score_dimensions": {"正确性": 4, "完整性": 4, "安全性": 5, "表达质量": 5, "指令遵循": 4},
558. }, ensure_ascii=False) for t in texts), encoding='utf-8')
559. table_rows = [
560.     {"id": "0001", "order_amount": "268.50", "city": "深圳", "risk": "正常", "remark": "字段完整"},
561.     {"id": "0002", "order_amount": "0", "city": "广州", "risk": "金额异常", "remark": "金额为0"},
562.     {"id": "0003", "order_amount": "99999", "city": "上海", "risk": "金额异常", "remark": "疑似测试订..."},
563.     {"id": "0004", "order_amount": "128.00", "city": "", "risk": "字段缺失", "remark": "城市为空"},
564.     {"id": "0005", "order_amount": "88.90", "city": "杭州", "risk": "正常", "remark": "字段完整"},
565.     {"id": "0006", "order_amount": "88.90", "city": "杭州", "risk": "重复疑似", "remark": "与0005..."},
566.     {"id": "0007", "order_amount": "abc", "city": "成都", "risk": "类型错误", "remark": "金额不是数字"},
567.     {"id": "0008", "order_amount": "320.20", "city": "武汉", "risk": "正常", "remark": "字段完整"},
568.     {"id": "0009", "order_amount": "-12", "city": "北京", "risk": "金额异常", "remark": "负数金额"},
569.     {"id": "0010", "order_amount": "51.30", "city": "南京", "risk": "正常", "remark": "字段完整"},
570. ]
571. table_csv = IMPORT_DIR / "电商订单表格异常校验样本.csv"
572. table_csv.write_text("id,order_amount,city,risk,remark\n" + "\n".join(f"{r['id']}"...
573. return construction, traffic, defects, texts, table_rows, table_csv
574. def _insert_labels(project_id: int, rows: list[tuple]):
575.     execute_many("""INSERT INTO labels(project_id,name,code,color,label_type,shortcut...
576.         VALUES(?,?,?, ?, ?, ?, ?, ?, ?, ?)""", [(project_id, *r) for r in rows])
577. def _insert_rules(project_id: int, rows: list[tuple]):
578.     execute_many("INSERT INTO annotation_rules(project_id,rule_type,title,content,sev...
579. IMAGE_BBOX_BY_FILENAME = {
580.     'helmet_site_01.jpg': [('person', 300, 245, 125, 405, .93, '预标注', '待确认'), ('vest', 285, 330...
581.     'municipal_worker_05.jpg': [('person', 258, 30, 76, 194, .94, '预标注', '待确认'), ('vest', 258, ...
582.     'rebar_site_04.jpg': [('person', 151, 18, 94, 218, .93, '预标注', '待确认'), ('helmet', 160, 14, 5...
583.     'scaffold_site_03.jpg': [('person', 612, 570, 170, 390, .92, '预标注', '待确认'), ('helmet', 704...
584.     'tunnel_site_02.jpg': [('person', 165, 405, 230, 360, .92, '预标注', '待确认'), ('helmet', 255, 3...
585.     'city_intersection_day_01.jpg': [('traffic_light', 70, 165, 90, 245, .92, '预标注', '待确认'), ...
586.     'highway_truck_06.jpg': [('car', 30, 154, 74, 48, .86, '预标注', '待确认'), ('car', 113, 145, 62, 4...
587.     'overhead_intersection_05.jpg': [('car', 285, 96, 92, 86, .90, '预标注', '待确认'), ('cyclist', ...
588.     'rain_expressway_04.jpg': [('car', 118, 117, 70, 42, .86, '预标注', '待确认'), ('car', 215, 136, 5...
589.     'rain_night_intersection_02.jpg': [('traffic_light', 54, 65, 70, 145, .92, '预标注', '待确认')...
590.     'school_street_03.jpg': [('bus', 535, 500, 150, 110, .88, '预标注', '待确认'), ('car', 620, 600, 2...
591.     'bottle_line_06.jpg': [('ok_region', 20, 90, 360, 95, .90, '人工', '已确认'), ('stain', 230, 92, ...
592.     'casting_crack_07.jpg': [('crack', 255, 16, 42, 195, .95, '预标注', '待确认'), ('ok_region', 48, ...
593.     'defective_flange_03.jpg': [('crack', 405, 575, 150, 210, .89, '预标注', '待确认'), ('stain', 82...
594.     'electronic_shell_04.jpg': [('scratch', 50, 575, 255, 145, .87, '预标注', '待确认'), ('stain', 7...
595.     'metal_panel_defect_02.jpg': [('dent', 270, 520, 92, 76, .86, '预标注', '待确认'), ('scratch', 7...
596.     'paint_scratch_08.jpg': [('scratch', 210, 70, 235, 58, .94, '预标注', '待确认'), ('ok_region', 4...
597.     'wall_crack_pipe_09.jpg': [('crack', 360, 0, 135, 214, .96, '预标注', '待确认'), ('ok_region', 1...
598.     'warehouse_forklift_01.jpg': [('ok_region', 115, 265, 460, 360, .84, '人工', '已确认'), ('stai...
599.     'warehouse_pallet_05.jpg': [('ok_region', 145, 65, 120, 115, .84, '人工', '已确认'), ('stain', ...
600. }

```

```

601. def _add_image_samples(project_id: int, files: list[tuple[Path, int, int]], prefix: s...
602.     ids = []
603.     for idx, (path, w, h) in enumerate(files, start=1):
604.         status = '预标注待确认' if idx % 4 else '已保存'
605.         risk = '低置信;小目标' if idx in (2, 7, 12) else ('模糊' if idx in (5, 11, 15) else '')
606.         sid = execute("""INSERT INTO samples(project_id,sample_code,sample_type,filen...
607.                     VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", (project_id, f"{pr...
608.         ids.append(sid)
609.         rows = IMAGE_BBOX_BY_FILENAME.get(path.name)
610.         if rows is None:
611.             rows = [(label, x, y, bw, bh, conf, '预标注' if j < min(3, len(label_plan)) ...
612.         for label, x, y, bw, bh, conf, src, st in rows:
613.             execute("""INSERT INTO annotations(sample_id,label,annotation_type,x,y,w,...
614.                     VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)""", (sid, label, 'bbox', x, y,...
615.         execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
616.     return ids
617. def seed_demo_data() -> None:
618.     # Fresh packages include a seeded DB. If a user deletes data/app.db, this recreat...
619.     if fetch_one("SELECT id FROM users LIMIT 1"):
620.         return
621.     construction, traffic, defects, text_samples, table_rows, table_csv = _write_demo...
622.     users = [
623.         ("admin", "admin123", "系统管理员", "管理员"),
624.         ("labeler", "123456", "标注员 A", "标注员"),
625.         ("reviewer", "123456", "质检员 A", "质检员"),
626.         ("manager", "123456", "项目经理", "项目经理"),
627.     ]
628.     execute_many("INSERT INTO users(username,password,display_name,role) VALUES(?,?,?...
629.     p1 = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,tas...
630.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", ("DS-IMG-2026-001", "安全帽佩戴检测图像数据集", "...
631.     p2 = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,tas...
632.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", ("DS-TXT-2026-002", "客服问答意图图分类文本数据集", ...
633.     p3 = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,tas...
634.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", ("DS-TRAFFIC-2026-003", "城市路口交通目标检测数据...
635.     p4 = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,tas...
636.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", ("DS-QC-2026-004", "工业产品外观缺陷检测数据集", "...
637.     p5 = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,tas...
638.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", ("DS-TABLE-2026-005", "电商订单表格异常校验数据集"...
639.     _insert_labels(p1, [
640.         ("person", "person", "#2563eb", "目标框标签", "1", 1, 0, "施工现场人员目标, 需完整贴合身体可见部分.", ...
641.         ("helmet", "helmet", "#f59e0b", "目标框标签", "2", 0, 1, "已佩戴安全帽目标.", "黄色或白色安全帽", ...
642.         ("no_helmet", "no_helmet", "#ef4444", "目标框标签", "3", 0, 1, "未佩戴安全帽人员头部.", "可见头...
643.         ("vest", "vest", "#22c55e", "目标框标签", "4", 0, 0, "反光背心区域.", "橙色/荧光背心", "普通上衣",...
644.         ("vehicle", "vehicle", "#7c3aed", "目标框标签", "5", 0, 0, "施工车辆.", "卡车/叉车/吊车", "手...
645.         ("warning_sign", "warning_sign", "#ec4899", "目标框标签", "6", 0, 0, "警示牌或三角标.", "...
646.     ])
647.     _insert_labels(p2, [(name, name, color, "分类标签", str(i+1), 1 if i == 0 else 0, 1, ...
648.         ("退款", "#ef4444"), ("物流", "#2563eb"), ("投诉", "#f97316"), ("咨询", "#22c55e"), (...
649.     _insert_labels(p3, [
650.         ("car", "car", "#2563eb", "目标框标签", "1", 1, 0, "小客车、出租车等机动车.", "完整车身", "远处无法分辨...

```

```

651.         ("bus", "bus", "#7c3aed", "目标框标签", "2", 0, 0, "公交或大型客运车辆。", "公交车/大巴", "货车", "...
652.         ("pedestrian", "pedestrian", "#22c55e", "目标框标签", "3", 1, 0, "行人目标。", "路口行人", ...
653.         ("cyclist", "cyclist", "#f97316", "目标框标签", "4", 0, 0, "骑行人和自行车整体。", "自行车/电动车骑...
654.         ("traffic_light", "traffic_light", "#ef4444", "目标框标签", "5", 0, 0, "交通信号灯。", "...
655.         ("road_sign", "road_sign", "#eab308", "目标框标签", "6", 0, 0, "道路标志牌。", "警示/指示牌",...
656.     ])
657.     _insert_labels(p4, [
658.         ("scratch", "scratch", "#ef4444", "目标框标签", "1", 1, 0, "划痕缺陷区域。", "细长线状损伤", "正...
659.         ("dent", "dent", "#f59e0b", "目标框标签", "2", 0, 0, "凹陷缺陷。", "局部变形阴影", "光照反射", "需...
660.         ("stain", "stain", "#7c3aed", "目标框标签", "3", 0, 0, "污渍/油渍。", "不规则脏污", "背景阴影", ...
661.         ("crack", "crack", "#dc2626", "目标框标签", "4", 1, 0, "裂纹。", "断裂纹路", "划痕", "严重缺陷优...
662.         ("ok_region", "ok_region", "#22c55e", "目标框标签", "5", 0, 0, "无缺陷对照区域。", "合格外观",...
663.     ])
664.     _insert_labels(p5, [(name, name, color, "表格质量标签", str(i+1), 0, 1, f"表格异常: {name}",...
665.         ("正常", "#22c55e"), ("字段缺失", "#ef4444"), ("类型错误", "#f97316"), ("金额异常", "#7c3ae...
666.     _insert_rules(p1, [
667.         ("图像", "框选边界规则", "框应贴合目标可见边界, 截断目标只标可见部分, 不允许大面积包含背景。", "高"),
668.         ("图像", "helmet/no_helmet 互斥规则", "同一头部不可同时标记 helmet 与 no_helmet; 无法判断时标疑难。", "高"),
669.         ("图像", "小目标处理规则", "短边小于 10 像素且无法判断类别的目标可不标, 但需在疑难说明中记录。", "中"),
670.         ("图像", "重复框规则", "同一目标重复框 IoU 大于 0.82 时必须合并。", "高"),
671.     ])
672.     _insert_rules(p2, [
673.         ("文本", "主诉求优先规则", "同一句包含多个问题时优先选择用户最核心的诉求。", "高"),
674.         ("文本", "评价理由规则", "偏好或质量评分必须填写不少于 10 字的评价理由。", "中"),
675.         ("文本", "同义文本一致规则", "重复或近似文本应保持相同标签, 冲突时进入质检复核。", "高"),
676.     ])
677.     _insert_rules(p3, [
678.         ("图像", "交通目标遮挡规则", "车辆和行人遮挡时按可见外轮廓标注, 严重遮挡进入疑难池。", "中"),
679.         ("图像", "小目标信号灯规则", "信号灯目标很小但可识别时仍需标注。", "高"),
680.     ])
681.     _insert_rules(p4, [
682.         ("图像", "缺陷区域规则", "缺陷框只覆盖可见缺陷, 不应包含大面积正常背景。", "高"),
683.         ("图像", "光照干扰规则", "反光与污渍容易混淆, 低置信样本需质检复核。", "中"),
684.     ])
685.     _insert_rules(p5, [
686.         ("表格", "字段完整性规则", "必填字段为空时标记字段缺失, 禁止直接进入交付。", "高"),
687.         ("表格", "数值类型规则", "金额字段必须可解析为非负数, 异常值需复核。", "高"),
688.     ])
689.     construction_ids = _add_image_samples(p1, construction, 'IMG', [
690.         ("person", 132, 360, 62, 170, 0.91), ("helmet", 145, 350, 42, 34, 0.87), ("ve...
691.     ])
692.     # one visible duplicate sample for duplicate detection: points to an existing hig...
693.     dup_path, dup_w, dup_h = construction[0]
694.     dup_id = execute("""INSERT INTO samples(project_id,sample_code,sample_type,filena...
695.         VALUES(?,?,?,?,,?,,?,,?,,?,,?,,?)""", (p1, "IMG-DUP-0001", 'image', ...
696.     construction_ids.append(dup_id)
697.     execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
698.     traffic_ids = _add_image_samples(p3, traffic, 'TRF', [
699.         ("car", 210, 420, 145, 74, 0.88), ("pedestrian", 120, 408, 48, 116, 0.81), ("...
700.     ])

```

```

701. defect_ids = _add_image_samples(p4, defects, 'DEF', [
702.     ("scratch", 320, 285, 110, 45, 0.74), ("dent", 500, 310, 85, 56, 0.69), ("sta...
703. ])
704. text_ids = []
705. for i, item in enumerate(text_samples, start=1):
706.     sid = execute("""INSERT INTO samples(project_id,sample_code,sample_type,filen...
707.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", (p2, item['id'], 'tex...
708.     text_ids.append(sid)
709.     execute("""INSERT INTO annotations(sample_id,label,annotation_type,confidence...
710.         VALUES(?,?,?,?,?,?,?,?,?)""", (sid, item['label'], 'text_class', 0.8...
711. execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
712. # additional text datasets for production-style demonstrations
713. finance_texts = [
714.     {"id": "F001", "text": "昨天转账两万元到对公账户，页面提示处理中，资金什么时候到账？", "label": "转账汇款", "re...
715.     {"id": "F002", "text": "我的信用卡被盗刷三笔境外消费，请马上冻结并协助争议处理。", "label": "盗刷风控", "reas...
716.     {"id": "F003", "text": "APP 登录一直提示设备风险，换手机后无法刷脸认证。", "label": "账户安全", "reason"...
717.     {"id": "F004", "text": "房贷提前还款需要预约吗？违约金怎么计算？", "label": "贷款咨询", "reason": "贷款...
718.     {"id": "F005", "text": "理财产品显示净值回撤，这个是不是保本的？", "label": "理财咨询", "reason": "理财...
719.     {"id": "F006", "text": "短信验证码不是我本人操作收到的，银行卡是否有风险？", "label": "账户安全", "reason"...
720.     {"id": "F007", "text": "我想注销二类账户，里面还有几块钱余额怎么处理？", "label": "账户业务", "reason": "...
721.     {"id": "F008", "text": "扣了年费但我没收到通知，要求退回。", "label": "费用争议", "reason": "费用扣费争...
722.     {"id": "F009", "text": "为什么我的转账被拦截？对方是我家人不是诈骗。", "label": "风控拦截", "reason": "...
723.     {"id": "F010", "text": "信用卡账单分期利率看不懂，能解释实际年化吗？", "label": "信用卡", "reason": "信...
724.     {"id": "F011", "text": "", "label": "其他", "reason": "空文本演示"},
725.     {"id": "F012", "text": "昨天转账两万元到对公账户，页面提示处理中，资金什么时候到账？", "label": "账户业务", "re...
726. ]
727. medical_texts = [
728.     {"id": "M001", "text": "孩子发烧到 39 度，还伴随咳嗽，应该挂儿科还是急诊？", "label": "儿科导诊", "reason"...
729.     {"id": "M002", "text": "体检报告显示血糖偏高，想预约内分泌科复查。", "label": "慢病复诊", "reason": "血...
730.     {"id": "M003", "text": "膝盖运动后疼痛，走楼梯明显，需要看骨科吗？", "label": "骨科咨询", "reason": "关...
731.     {"id": "M004", "text": "皮肤突然起红疹并瘙痒，可能是过敏吗？", "label": "皮肤过敏", "reason": "皮肤症状...
732.     {"id": "M005", "text": "昨晚胸口闷痛十分钟，现在还有点气短。", "label": "急症风险", "reason": "胸痛气短...
733.     {"id": "M006", "text": "医生开的药吃完后胃不舒服，可以自行停药吗？", "label": "用药咨询", "reason": "用...
734.     {"id": "M007", "text": "孕 12 周需要做哪些产检？NT 检查要预约吗？", "label": "产检咨询", "reason": "孕...
735.     {"id": "M008", "text": "晚上睡不着已经两周，白天注意力很差。", "label": "心理睡眠", "reason": "睡眠与心...
736.     {"id": "M009", "text": "血压 160/100，头晕，我要不要马上去医院？", "label": "急症风险", "reason": "...
737.     {"id": "M010", "text": "牙龈肿痛三天，吃东西会出血。", "label": "口腔咨询", "reason": "口腔科导诊"},
738.     {"id": "M011", "text": "膝盖运动后疼痛，走楼梯明显，需要看骨科吗？", "label": "运动康复", "reason": "重...
739.     {"id": "M012", "text": "胸", "label": "急症风险", "reason": "文本过短演示"},
740. ]
741. finance_csv = TEXT_DIR / "金融客服风险意图样本.csv"
742. finance_csv.write_text('id,text,label,reason\n' + '\n'.join(f"{t['id']},{t['text']...
743. medical_csv = TEXT_DIR / "医疗导诊意图样本.csv"
744. medical_csv.write_text('id,text,label,reason\n' + '\n'.join(f"{t['id']},{t['text']...
745. p6 = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,tas...
746.     VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", ("DS-FIN-2026-006", "金融客服风险意图文本数据集", ...
747. _insert_labels(p6, [(name, name, color, "分类标签", str(i+1), 1 if i == 0 else 0, 1, ...
748.     ("账户安全", "#ef4444"), ("转账汇款", "#2563eb"), ("盗刷风控", "#dc2626"), ("贷款咨询", "#7c3...
749. _insert_rules(p6, [
750.     ("文本", "金融风险优先规则", "盗刷、验证码异常、转账拦截等安全事件优先标为风险类，不可仅标普通咨询。", "高"),

```



```

751.         ("文本", "金额与账户实体规则", "金额、账户类型、转账对象等实体应完整标注, 用于后续风控模型训练。", "中"),
752.     ])
753.     finance_ids = []
754.     for i, item in enumerate(finance_texts, start=1):
755.         risk = '空文本' if item['text'] == '' else ('重复' if i == 12 else ('高风险' if item[...
756.         sid = execute("""INSERT INTO samples(project_id,sample_code,sample_type,filen...
757.             VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", (p6, item['id'], 'tex...
758.         finance_ids.append(sid)
759.         execute("""INSERT INTO annotations(sample_id,label,annotation_type,confidence...
760.             VALUES(?,?,?,?,?,?,?,?,?)""", (sid, item['label'], 'text_class', 0.8...
761.     execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
762.     p7 = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,tas...
763.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", ("DS-MED-2026-007", "医疗导诊问答意图文本数据集", ...
764.     _insert_labels(p7, [(name, name, color, "分类标签", str(i+1), 1 if i == 0 else 0, 1, ...
765.         ("急症风险", "#dc2626"), ("儿科导诊", "#2563eb"), ("慢病复诊", "#22c55e"), ("骨科咨询", "#7c3...
766.     _insert_rules(p7, [
767.         ("文本", "医疗安全规则", "胸痛、气短、高热、血压异常等高风险症状应优先标记急症风险, 并进入质检复核。", "高"),
768.         ("文本", "非诊断表达规则", "样本评价理由应避免给出确定诊断, 只标注意图、风险等级和导诊建议。", "高"),
769.     ])
770.     medical_ids = []
771.     for i, item in enumerate(medical_texts, start=1):
772.         risk = '高风险' if item['label'] == '急症风险' else ('重复' if i == 11 else ('文本过短' if...
773.         sid = execute("""INSERT INTO samples(project_id,sample_code,sample_type,filen...
774.             VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", (p7, item['id'], 'tex...
775.         medical_ids.append(sid)
776.         execute("""INSERT INTO annotations(sample_id,label,annotation_type,confidence...
777.             VALUES(?,?,?,?,?,?,?,?,?)""", (sid, item['label'], 'text_class', 0.8...
778.     execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
779.     execute_many("""INSERT INTO quality_issues(sample_id,annotation_id,issue_type,sev...
780.         VALUES(?,?,?,?,?,?,?,?,?)""", [
781.             (finance_ids[1], None, "疑似盗刷高风险", "高", "FinanceFraudRisk", "F002", "优先冻结和争议处理...
782.             (finance_ids[10], None, "文本为空", "高", "EmptyText", "F011", "删除或补充原始文本", "待处理"),
783.             (finance_ids[11], None, "重复文本标签冲突", "高", "DuplicateTextConflict", "F001/F012"...
784.             (medical_ids[4], None, "急症风险提示", "高", "MedicalSafety", "M005", "胸痛气短应进入高风险复核"...
785.             (medical_ids[10], None, "重复文本标签冲突", "中", "DuplicateTextConflict", "M003/M011"...
786.             (medical_ids[11], None, "文本过短", "中", "TooShort", "M012", "补充上下文或删除", "待处理"),
787.         ])
788.     table_ids = []
789.     for i, row in enumerate(table_rows, start=1):
790.         sid = execute("""INSERT INTO samples(project_id,sample_code,sample_type,filen...
791.             VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""", (p5, row['id'], 'table', t...
792.         table_ids.append(sid)
793.         execute("INSERT INTO annotations(sample_id,label,annotation_type,confidence,s...
794.     execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
795.     # quality issues across richer datasets
796.     issue_rows = [
797.         (construction_ids[1], None, "小目标疑似漏标", "中", "SmallObjectMissing", "右侧工人头部区域",...
798.         (construction_ids[3], None, "标注框越界", "高", "BBoxBoundary", "左侧 person 框越界", "调...
799.         (construction_ids[7], None, "同目标重复框", "高", "DuplicateIoU", "helmet 重复框", "删除重...
800.         (construction_ids[10], None, "图片模糊", "中", "BlurImage", "全图", "建议人工复核是否可用于训练",...

```

```

801.         (dup_id, None, "重复图片", "高", "DuplicateImage", "与 IMG-0003 完全重复", "确认是否剔除或只保留一...
802.         (traffic_ids[3], None, "雨天低能见度", "中", "LowVisibility", "全图", "优先复核行人和信号灯", "待...
803.         (traffic_ids[6], None, "行人/骑行者类别分歧", "中", "ClassConflict", "画面右侧骑行者", "按 cycli...
804.         (defect_ids[2], None, "光照疑似误检", "中", "ReflectionNoise", "产品中部", "复核污渍和反光边界", ...
805.         (defect_ids[4], None, "缺陷框过大", "高", "DefectBBoxTooLarge", "scratch 框", "缩小到缺陷...
806.         (text_ids[4], None, "文本为空", "高", "EmptyText", "T005", "删除或补充原始文本", "待处理"),
807.         (text_ids[6], None, "重复文本标签不一致", "高", "DuplicateTextConflict", "T001/T007", "...
808.         (text_ids[9], None, "文本过短", "中", "TooShort", "T010", "补充上下文或剔除样本", "待处理"),
809.         (table_ids[1], None, "金额为零", "中", "AmountZero", "0002.order_amount", "确认是否真实促...
810.         (table_ids[6], None, "金额字段类型错误", "高", "AmountType", "0007.order_amount", "需要转...
811.         (table_ids[8], None, "负数金额", "高", "NegativeAmount", "0009.order_amount", "进入人...
812.     ]
813.     execute_many("""INSERT INTO quality_issues(sample_id,annotation_id,issue_type,sev...
814.                 VALUES(?,?,?,?,?,?,?)""", issue_rows)
815.     reworks = [
816.         ("RW-202606-0001", construction_ids[3], p1, "labeler", "reviewer", "标注框越界/漏标"...
817.         ("RW-202606-0002", construction_ids[7], p1, "labeler", "reviewer", "重复框", "he...
818.         ("RW-202606-0003", defect_ids[4], p4, "labeler", "reviewer", "缺陷框过大", "scratc...
819.     ]
820.     execute_many("""INSERT INTO rework_tasks(rework_code,sample_id,project_id,labeler...
821.                 VALUES(?,?,?,?,?,?,?,?,?,?)""", reworks)
822.     gt_rows = [
823.         (construction_ids[1], {"labels": ["person", "helmet", "vest", "vehicle"]}, 92...
824.         (construction_ids[8], {"labels": ["person", "helmet", "warning_sign"]}, 88.0,...
825.         (traffic_ids[1], {"labels": ["car", "pedestrian", "traffic_light"]}, 90.2, "交...
826.         (defect_ids[2], {"labels": ["scratch", "stain"]}, 76.5, "缺陷边界需复核。", "admin"),
827.         (text_ids[0], {"label": "退款"}, 95.0, "标准答案一致。", "admin"),
828.     ]
829.     for sid, ans, score, conclusion, user in gt_rows:
830.         execute("INSERT INTO ground_truth(sample_id,answer_json,score,conclusion,crea...
831.     execute_many("""INSERT INTO consensus_results(project_id,sample_id,worker_a,work...
832.                 VALUES(?,?,?,?,?,?,?)""", [
833.         (p1, construction_ids[1], "labeler", "labeler_b", 0.87, 0.92, "安全帽框位置轻微偏移。", 0),
834.         (p1, construction_ids[3], "labeler", "labeler_b", 0.61, 0.68, "person 越界框和 he...
835.         (p2, text_ids[6], "labeler", "labeler_b", 0.0, 0.58, "退款/售后主诉求分歧。", 1),
836.         (p3, traffic_ids[6], "labeler", "labeler_c", 0.72, 0.74, "cyclist 与 pedestria...
837.         (p4, defect_ids[4], "labeler", "labeler_c", 0.55, 0.70, "scratch/dent 边界分歧明显。...
838.     ])
839.     versions = [
840.         (p1, "v1.2.1", 17, 12, 84.6, 0.8, 0.1, 0.1, "待冻结", "admin", "", "安全帽数据集候选交付版本...
841.         (p2, "v0.9.4", 22, 14, 72.0, 0.8, 0.1, 0.1, "编辑中", "admin", "", "客服意图文本数据集内测版...
842.         (p3, "v0.6.0", 14, 8, 78.5, 0.8, 0.1, 0.1, "待冻结", "admin", "", "交通路口检测样本版本", ...
843.         (p4, "v0.4.2", 12, 5, 58.0, 0.7, 0.15, 0.15, "返工中", "admin", "", "工业缺陷检测样本版本"...
844.         (p5, "v0.2.0", 10, 4, 60.0, 0.7, 0.15, 0.15, "编辑中", "admin", "", "表格异常校验样本版本"...
845.         (p6, "v0.3.0", 12, 7, 70.5, 0.8, 0.1, 0.1, "编辑中", "admin", "", "金融客服风险意图文本样本版...
846.         (p7, "v0.2.5", 12, 6, 68.0, 0.8, 0.1, 0.1, "质检中", "admin", "", "医疗导诊问答意图文本样本版...
847.     ]
848.     execute_many("""INSERT INTO dataset_versions(project_id,version_no,sample_total,p...
849.                 VALUES(?,?,?,?,?,?,?,?,?,?)""", versions)
850.     demo_report = DOCS_DIR / "标注质量报告示例.md"

```

```

851.         demo_report.write_text("# 人工智能模型训练样本智能标注与质量校验管理软件 - 标注质量报告示例\n\n当前演示库包含安全帽、交
            通路口、工...
852.         execute("INSERT INTO reports(project_id,report_type,title,file_path,conclusion,cr...
853.         execute_many("INSERT INTO operation_logs(username,action,detail) VALUES(?,?,?)", [
854.             ("admin", "初始化系统", "创建 7 个演示项目、标签、样本、预标注和质检记录"),
855.             ("labeler", "保存标注", "完成 IMG-0004 返工样本的首次修正"),
856.             ("reviewer", "生成返工", "将 IMG-0004 与 DEF-0005 退回返工并填写质检意见"),
857.             ("manager", "查看驾驶舱", "检查交通与工业质检项目交付风险"),
858.         ])
859.     from pathlib import Path
860.     def load_stylesheet() -> str:
861.         path = Path(__file__).resolve().parent / "styles.qss"
862.         return path.read_text(encoding="utf-8") if path.exists() else ""
863.     from statistics import mean
864.     from app.core.database import fetch_all, execute, log_action
865.     from app.algorithms.iou import bbox_iou
866.     def calculate_consensus(project_id: int, username: str = 'system') -> dict:
867.         samples = fetch_all("SELECT * FROM samples WHERE project_id=? ORDER BY id LIMIT 1..
868.         created = 0
869.         scores = []
870.         for s in samples:
871.             anns = fetch_all("SELECT * FROM annotations WHERE sample_id=? AND status!='已删...
872.             if not anns:
873.                 continue
874.             # simulate second labeler by adding small offsets logically, compare first tw...
875.             ious = []
876.             label_hits = []
877.             for ann in anns[:4]:
878.                 other = dict(ann)
879.                 other['x'] = ann['x'] + (6 if ann['id'] % 2 else -8)
880.                 other['y'] = ann['y'] + (4 if ann['id'] % 3 else -5)
881.                 ious.append(bbox_iou(ann, other))
882.                 label_hits.append(1.0 if ann['id'] % 5 else 0.0)
883.             iou_score = mean(ious) if ious else 0
884.             label_agree = mean(label_hits) if label_hits else 0
885.             diff = '边界轻微偏移' if iou_score > 0.75 else '框位置偏移较大或漏标'
886.             need = 1 if iou_score < 0.72 or label_agree < 0.75 else 0
887.             execute("""INSERT INTO consensus_results(project_id,sample_id,worker_a,worker...
888.                 VALUES(?,?,?,?,?,?,?,?)""", (project_id, s['id'], 'labeler', 'labe...
889.             scores.append(iou_score * 60 + label_agree * 40)
890.             created += 1
891.             log_action(username, '执行多人一致性分析', f"项目{project_id} 生成 {created} 条一致性记录")
892.             return {'created': created, 'avg_score': round(mean(scores), 2) if scores else 0}
893.     def list_consensus(project_id: int) -> list[dict]:
894.         return fetch_all("""SELECT c.*, s.sample_code, s.filename FROM consensus_results c
895.             LEFT JOIN samples s ON c.sample_id=s.id WHERE c.project_id=? ...
896.     from collections import Counter
897.     from app.core.database import fetch_all, fetch_one, execute, log_action
898.     from app.algorithms.dataset_health import label_balance_score, compute_health
899.     def list_projects() -> list[dict]:

```

```
900.         return fetch_all("SELECT * FROM dataset_projects ORDER BY id")
```

```

901. def get_project(project_id: int | None = None) -> dict | None:
902.     if project_id is None:
903.         return fetch_one("SELECT * FROM dataset_projects ORDER BY id LIMIT 1")
904.         return fetch_one("SELECT * FROM dataset_projects WHERE id=?", (project_id,))
905. def create_project(data: dict, username: str) -> int:
906.     pid = execute("""INSERT INTO dataset_projects(code,name,project_type,data_type,ta...
907.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)""", (
908.         data['code'], data['name'], data['project_type'], data['data_type'], data['ta...
909.     ))
910.     log_action(username, '创建数据集项目', f"{data['code']} {data['name']}")
911.     return pid
912. def list_samples(project_id: int, sample_type: str | None = None) -> list[dict]:
913.     if sample_type:
914.         return fetch_all("SELECT * FROM samples WHERE project_id=? AND sample_type=? ...
915.         return fetch_all("SELECT * FROM samples WHERE project_id=? ORDER BY id", (project...
916. def get_sample(sample_id: int) -> dict | None:
917.     return fetch_one("SELECT * FROM samples WHERE id=?", (sample_id,))
918. def get_labels(project_id: int) -> list[dict]:
919.     return fetch_all("SELECT * FROM labels WHERE project_id=? AND enabled=1 ORDER BY ...
920. def list_rules(project_id: int) -> list[dict]:
921.     return fetch_all("SELECT * FROM annotation_rules WHERE project_id=? ORDER BY id",...
922. def add_label(project_id: int, label: dict, username: str) -> int:
923.     lid = execute("""INSERT INTO labels(project_id,name,code,color,label_type,shortcu...
924.         VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)""", (project_id, label['name'], la...
925.     execute("UPDATE dataset_projects SET label_count=(SELECT COUNT(*) FROM labels WHE...
926.     log_action(username, '新增标签', f"项目{project_id} 标签 {label['name']}")
927.     return lid
928. def add_rule(project_id: int, rule: dict, username: str) -> int:
929.     rid = execute("INSERT INTO annotation_rules(project_id,rule_type,title,content,se...
930.     log_action(username, '新增标注规范', f"项目{project_id} {rule['title']}")
931.     return rid
932. def annotations_for_sample(sample_id: int) -> list[dict]:
933.     return fetch_all("SELECT * FROM annotations WHERE sample_id=? AND status!='已删除' 0...
934. def dashboard_metrics(project_id: int | None = None) -> dict:
935.     project = get_project(project_id)
936.     if not project:
937.         return {}
938.     pid = project['id']
939.     samples = list_samples(pid)
940.     annotations = fetch_all("SELECT a.* FROM annotations a JOIN samples s ON a.sample...
941.     issues = fetch_all("SELECT qi.* FROM quality_issues qi JOIN samples s ON qi.sampl...
942.     reworks = fetch_all("SELECT * FROM rework_tasks WHERE project_id=?", (pid,))
943.     total = len(samples) or 1
944.     annotated = sum(1 for s in samples if s['status'] in ('已保存', '已提交', '已通过', '自动质检异常', ...
945.     qc_pass = sum(1 for s in samples if s['qc_status'] == '已通过')
946.     issue_open = sum(1 for i in issues if i['status'] != '已关闭')
947.     gt_rows = fetch_all("SELECT gt.* FROM ground_truth gt JOIN samples s ON gt.sample...
948.     consensus_rows = fetch_all("SELECT * FROM consensus_results WHERE project_id=?", ...
949.     label_counts = Counter([a['label'] for a in annotations if a.get('label')])
950.     balance = label_balance_score(dict(label_counts))

```

```

951.     metrics = {
952.         'project_name': project['name'],
953.         'sample_total': len(samples),
954.         'annotated_total': annotated,
955.         'pending_total': max(0, len(samples) - annotated),
956.         'qc_pending_total': sum(1 for s in samples if s['qc_status'] == '待质检'),
957.         'rework_total': len(reworks),
958.         'qc_pass_total': qc_pass,
959.         'prelabel_adapt_rate': 78.4,
960.         'avg_time': '02:36',
961.         'auto_pass_rate': max(0, 100 - issue_open * 4),
962.         'manual_pass_rate': qc_pass / total * 100,
963.         'consensus_score': sum(r['iou_score'] * 100 * 0.6 + r['label_agreement'] * 10...
964.         'gt_score': sum(r['score'] for r in gt_rows) / len(gt_rows) if gt_rows else 8...
965.         'low_quality_total': sum(1 for s in samples if s['is_low_confidence'] or '模糊'...
966.         'label_counts': dict(label_counts),
967.         'balance_score': balance,
968.         'duplicate_rate': sum(1 for s in samples if s['is_duplicate']) / total,
969.         'rework_rate': len(reworks) / total,
970.         'annotated_rate': annotated / total * 100,
971.         'version_total': len(fetch_all("SELECT id FROM dataset_versions WHERE project...
972.     }
973.     health, advice = compute_health(metrics)
974.     metrics['health_score'] = health
975.     metrics['health_advice'] = advice
976.     return metrics
977. def update_project_health(project_id: int) -> float:
978.     score = dashboard_metrics(project_id).get('health_score', 0)
979.     execute("UPDATE dataset_projects SET health_score=? WHERE id=?", (score, project_...
980.     return score
981. import json
982. import zipfile
983. from pathlib import Path
984. from xml.etree.ElementTree import Element, SubElement, tostring
985. from xml.dom import minidom
986. from app.core.config import EXPORT_DIR
987. from app.core.database import fetch_all, execute, log_action
988. from app.services.dataset_service import get_labels, dashboard_metrics
989. def delivery_check(project_id: int) -> tuple[bool, list[str]]:
990.     metrics = dashboard_metrics(project_id)
991.     problems = []
992.     if metrics.get('pending_total', 0) > 0: problems.append(f"存在未标注样本 {metrics['pendi...
993.     if metrics.get('qc_pending_total', 0) > 0: problems.append(f"存在待质检样本 {metrics['qc...
994.     if metrics.get('rework_total', 0) > 0: problems.append(f"存在返工样本 {metrics['rework_...
995.     if metrics.get('health_score', 0) < 80: problems.append(f"健康度 {metrics.get('healt...
996.     if metrics.get('balance_score', 0) < 70: problems.append("标签分布不均衡")
997.     return len(problems) == 0, problems
998. def _project_dir(project_id: int, fmt: str) -> Path:
999.     out = EXPORT_DIR / f"project_{project_id}_{fmt}"
1000.     out.mkdir(parents=True, exist_ok=True)

```

```

1001.     return out
1002. def export_yolo(project_id: int, username: str) -> Path:
1003.     out = _project_dir(project_id, 'yolo')
1004.     labels = [x['name'] for x in get_labels(project_id)]
1005.     (out / 'classes.txt').write_text('\n'.join(labels), encoding='utf-8')
1006.     samples = fetch_all("SELECT * FROM samples WHERE project_id=? AND sample_type='im...
1007.     for s in samples:
1008.         anns = fetch_all("SELECT * FROM annotations WHERE sample_id=? AND annotation_...
1009.         rows = []
1010.         for a in anns:
1011.             cls = labels.index(a['label']) if a['label'] in labels else 0
1012.             xc = (a['x'] + a['w'] / 2) / max(1, s['width'])
1013.             yc = (a['y'] + a['h'] / 2) / max(1, s['height'])
1014.             rows.append(f"{cls} {xc:.6f} {yc:.6f} {a['w']/max(1,s['width']):.6f} {a['...
1015.             (out / f"{Path(s['filename']).stem}.txt").write_text('\n'.join(rows), encodin...
1016.     zip_path = EXPORT_DIR / f"project_{project_id}_YOLO.zip"
1017.     with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as z:
1018.         for p in out.glob('*'):
1019.             z.write(p, p.name)
1020.     ok, problems = delivery_check(project_id)
1021.     execute("INSERT INTO export_records(project_id,format,output_path,check_result,cr...
1022.     log_action(username, '导出 YOLO', str(zip_path))
1023.     return zip_path
1024. def export_coco(project_id: int, username: str) -> Path:
1025.     out = _project_dir(project_id, 'coco')
1026.     labels = get_labels(project_id)
1027.     cats = [{ 'id': i+1, 'name': l['name']} for i, l in enumerate(labels)]
1028.     label_map = {c['name']: c['id'] for c in cats}
1029.     images = []
1030.     annotations = []
1031.     aid = 1
1032.     samples = fetch_all("SELECT * FROM samples WHERE project_id=? AND sample_type='im...
1033.     for s in samples:
1034.         images.append({'id': s['id'], 'file_name': s['filename'], 'width': s['width']...
1035.         anns = fetch_all("SELECT * FROM annotations WHERE sample_id=? AND annotation_...
1036.         for a in anns:
1037.             annotations.append({'id': aid, 'image_id': s['id'], 'category_id': label_...
1038.             aid += 1
1039.     path = out / 'annotations_coco.json'
1040.     path.write_text(json.dumps({'images': images, 'annotations': annotations, 'catego...
1041.     execute("INSERT INTO export_records(project_id,format,output_path,check_result,cr...
1042.     log_action(username, '导出 COCO', str(path))
1043.     return path
1044. def export_pascal_voc(project_id: int, username: str) -> Path:
1045.     out = _project_dir(project_id, 'voc')
1046.     samples = fetch_all("SELECT * FROM samples WHERE project_id=? AND sample_type='im...
1047.     for s in samples:
1048.         root = Element('annotation')
1049.         SubElement(root, 'filename').text = s['filename']
1050.         size = SubElement(root, 'size')

```

```

1051.         SubElement(size, 'width').text = str(s['width']); SubElement(size, 'height')...
1052.         for a in fetch_all("SELECT * FROM annotations WHERE sample_id=? AND annotatio...
1053.             obj = SubElement(root, 'object'); SubElement(obj, 'name').text = a['label']
1054.             bnd = SubElement(obj, 'bndbox')
1055.             SubElement(bnd, 'xmin').text = str(int(a['x'])); SubElement(bnd, 'ymin')...
1056.             SubElement(bnd, 'xmax').text = str(int(a['x']+a['w'])); SubElement(bnd, '...
1057.             xml = minidom.parseString(tostring(root)).toprettyxml(indent=' ')
1058.             (out / f"{Path(s['filename']).stem}.xml").write_text(xml, encoding='utf-8')
1059.         execute("INSERT INTO export_records(project_id,format,output_path,check_result,cr...
1060.         log_action(username, '导出 VOC', str(out))
1061.         return out
1062. def export_text_jsonl(project_id: int, username: str) -> Path:
1063.     out = _project_dir(project_id, 'jsonl')
1064.     path = out / 'text_training_samples.jsonl'
1065.     rows = []
1066.     samples = fetch_all("SELECT * FROM samples WHERE project_id=? AND sample_type='te...
1067.     for s in samples:
1068.         anns = fetch_all("SELECT * FROM annotations WHERE sample_id=? AND status!='已删...
1069.         labels = [a['label'] for a in anns]
1070.         rows.append(json.dumps({'id': s['sample_code'], 'text': s['text_content'], 'l...
1071.         path.write_text('\n'.join(rows), encoding='utf-8')
1072.         execute("INSERT INTO export_records(project_id,format,output_path,check_result,cr...
1073.         log_action(username, '导出文本 JSONL', str(path))
1074.         return path
1075. import csv
1076. import shutil
1077. from pathlib import Path
1078. from PIL import Image
1079. from app.core.database import execute, log_action
1080. from app.core.config import IMPORT_DIR
1081. from app.algorithms.image_hash import md5_file, average_hash, hamming_hex
1082. from app.algorithms.text_quality import normalize_text
1083. def inspect_image_folder(folder: str) -> list[dict]:
1084.     path = Path(folder)
1085.     result = []
1086.     hashes = {}
1087.     for p in sorted(path.glob('*')):
1088.         if p.suffix.lower() not in ['.jpg', '.jpeg', '.png', '.bmp']:
1089.             continue
1090.         row = {'filename': p.name, 'path': str(p), 'status': '正常', 'issues': []}
1091.         try:
1092.             img = Image.open(p)
1093.             row['width'], row['height'] = img.size
1094.             row['md5'] = md5_file(p)
1095.             ah = average_hash(p)
1096.             row['ahash'] = ah
1097.             if min(img.size) < 128:
1098.                 row['issues'].append('图片尺寸过小')
1099.             if row['md5'] in hashes:
1100.                 row['issues'].append(f"完全重复: {hashes[row['md5']]}"")

```



```

1101.         else:
1102.             for old_hash, old_name in hashes.items():
1103.                 pass
1104.         for other in result:
1105.             if other.get('ahash') and hamming_hex(ah, other['ahash']) <= 6:
1106.                 row['issues'].append(f"近似重复: {other['filename']}")
1107.                 break
1108.             hashes[row['md5']] = p.name
1109.     except Exception as exc:
1110.         row['status'] = '异常'
1111.         row['issues'].append(f'无法读取: {exc}')
1112.         row['width'], row['height'] = 0, 0
1113.     if row['issues']:
1114.         row['status'] = '需人工确认'
1115.     result.append(row)
1116.     return result
1117. def import_images(project_id: int, folder: str, username: str) -> list[dict]:
1118.     records = inspect_image_folder(folder)
1119.     target_dir = IMPORT_DIR / f"project_{project_id}_images"
1120.     target_dir.mkdir(parents=True, exist_ok=True)
1121.     for idx, row in enumerate(records, start=1):
1122.         source = Path(row['path'])
1123.         target = target_dir / source.name
1124.         if source.exists() and source.resolve() != target.resolve():
1125.             shutil.copy2(source, target)
1126.         sid = execute("""INSERT INTO samples(project_id,sample_code,sample_type,filen...
1127.                        VALUES(?,?,?,?,,?,,?,,?,,?)""", (project_id, f"IMP-IMG-{id...
1128.         row['sample_id'] = sid
1129.     execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
1130.     log_action(username, '导入图片样本', f"项目{project_id} 导入 {len(records)} 张图片")
1131.     return records
1132. def inspect_text_csv(csv_path: str) -> list[dict]:
1133.     p = Path(csv_path)
1134.     rows = []
1135.     seen = {}
1136.     with p.open('r', encoding='utf-8-sig', newline='') as f:
1137.         reader = csv.DictReader(f)
1138.         for i, item in enumerate(reader, start=1):
1139.             text = item.get('text') or item.get('content') or ''
1140.             label = item.get('label') or ''
1141.             issues = []
1142.             n = normalize_text(text)
1143.             if not text.strip(): issues.append('文本为空')
1144.             if 0 < len(text.strip()) < 8: issues.append('文本过短')
1145.             if not label.strip(): issues.append('标签缺失')
1146.             if n in seen: issues.append(f"重复文本: 第{seen[n]}行")
1147.             seen[n] = i
1148.             rows.append({'line': i, 'text': text, 'label': label, 'issues': issues, '...
1149.     return rows
1150. def import_text_csv(project_id: int, csv_path: str, username: str) -> list[dict]:

```

```

1151.     records = inspect_text_csv(csv_path)
1152.     for row in records:
1153.         sid = execute("""INSERT INTO samples(project_id,sample_code,sample_type,flen...
1154.             VALUES(?,?,?,?,?,?,?,?,?,?,?)""", (project_id, f"IMP-TXT-{ro...
1155.         if row['label']:
1156.             execute("INSERT INTO annotations(sample_id,label,annotation_type,confiden...
1157.             row['sample_id'] = sid
1158.     execute("UPDATE dataset_projects SET sample_count=(SELECT COUNT(*) FROM samples W...
1159.     log_action(username, '导入文本样本', f"项目{project_id} 导入 {len(records)} 条文本")
1160.     return records
1161. import random
1162. from app.core.database import execute, fetch_one, fetch_all, log_action
1163. def generate_prelabels_for_sample(sample_id: int, username: str = 'system') -> list[d...
1164.     sample = fetch_one("SELECT * FROM samples WHERE id=?", (sample_id,))
1165.     if not sample or sample['sample_type'] != 'image':
1166.         return []
1167.     labels = fetch_all("SELECT name FROM labels WHERE project_id=? AND label_type LIK...
1168.     names = [x['name'] for x in labels] or ['person', 'helmet']
1169.     rng = random.Random(sample_id * 991)
1170.     generated = []
1171.     for i in range(rng.randint(2, 5)):
1172.         label = rng.choice(names)
1173.         w = rng.randint(38, 150)
1174.         h = rng.randint(32, 170)
1175.         x = rng.randint(0, max(1, sample['width'] - w))
1176.         y = rng.randint(0, max(1, sample['height'] - h))
1177.         conf = round(rng.uniform(0.52, 0.94), 2)
1178.         aid = execute("""INSERT INTO annotations(sample_id,label,annotation_type,x,y,...
1179.             VALUES(?,?,?,?,?,?,?,?,?,?,?)""", (sample_id, label, 'bbox',...
1180.         generated.append({'id': aid, 'label': label, 'x': x, 'y': y, 'w': w, 'h': h, ...
1181.     execute("UPDATE samples SET status='预标注待确认', is_low_confidence=? WHERE id=?", (1 ...
1182.     log_action(username, '生成智能预标注', f"样本{sample_id} 生成 {len(generated)} 个候选框")
1183.     return generated
1184. def accept_prelabels(sample_id: int, username: str) -> int:
1185.     rows = fetch_all("SELECT id FROM annotations WHERE sample_id=? AND source='预标注' A...
1186.     for row in rows:
1187.         execute("UPDATE annotations SET status='已确认', created_by=?, updated_at=CURREN...
1188.     execute("UPDATE samples SET status='已保存' WHERE id=?", (sample_id,))
1189.     log_action(username, '接受预标注', f"样本{sample_id} 接受 {len(rows)} 个预标注")
1190.     return len(rows)
1191. def delete_annotation(annotation_id: int, username: str, reason: str = '误检') -> None:
1192.     execute("UPDATE annotations SET status='已删除', comment=? WHERE id=?", (reason, ann...
1193.     log_action(username, '删除标注对象', f"对象{annotation_id} 原因: {reason}")
1194. def update_bbox(annotation_id: int, x: float, y: float, w: float, h: float, label: str...
1195.     execute("UPDATE annotations SET x=?,y=?,w=?,h=?,label=?,source='人工',status='已确认',...
1196.     log_action(username, '修改标注框', f"对象{annotation_id} -> {label} ({x:.0f},{y:.0f},{w:...
1197. def create_bbox(sample_id: int, label: str, x: float, y: float, w: float, h: float, u...
1198.     aid = execute("""INSERT INTO annotations(sample_id,label,annotation_type,x,y,w,h,...
1199.         VALUES(?,?,?,?,?,?,?,?,?,?,?)""", (sample_id, label, 'bbox', x, ...
1200.     execute("UPDATE samples SET status='已保存' WHERE id=?", (sample_id,))

```

```

1201.     log_action(username, '新增人工标注框', f"样本{sample_id} 标签 {label}")
1202.     return aid
1203. from app.core.database import fetch_all, fetch_one, execute, log_action
1204. from app.algorithms.bbox_quality import check_bbox, check_duplicate_boxes
1205. from app.algorithms.text_quality import check_text_sample, similarity
1206. def clear_open_issues(sample_id: int) -> None:
1207.     execute("UPDATE quality_issues SET status='已关闭' WHERE sample_id=? AND status!='已关...
1208. def run_quality_check(sample_id: int, username: str = 'system') -> list[dict]:
1209.     sample = fetch_one("SELECT * FROM samples WHERE id=?", (sample_id,))
1210.     if not sample:
1211.         return []
1212.     clear_open_issues(sample_id)
1213.     issues = []
1214.     if sample['sample_type'] == 'image':
1215.         annotations = fetch_all("SELECT * FROM annotations WHERE sample_id=? AND stat...
1216.         if not annotations:
1217.             issues.append({'type': '当前图片无任何标签', 'severity': '高', 'rule': 'EmptyAnnota...
1218.         for ann in annotations:
1219.             for issue in check_bbox(ann, sample['width'], sample['height']):
1220.                 issue['annotation_id'] = ann['id']
1221.                 issues.append(issue)
1222.                 if ann['source'] == '预标注' and ann['status'] == '待确认':
1223.                     issues.append({'type': '预标注目标未处理', 'severity': '中', 'rule': 'Unconfir...
1224.             issues.extend(check_duplicate_boxes(annotations))
1225.     else:
1226.         annotations = fetch_all("SELECT * FROM annotations WHERE sample_id=? AND stat...
1227.         labels = [a['label'] for a in annotations if a['label']]
1228.         reason = annotations[0]['comment'] if annotations else ''
1229.         issues.extend([{'type': 'annotation_id: annotations[0][id] if annotations els...
1230.         # same project duplicate/inconsistent labels
1231.         peers = fetch_all("SELECT s.id,s.text_content,a.label FROM samples s LEFT JOI...
1232.         for p in peers:
1233.             if similarity(sample['text_content'] or '', p['text_content'] or '') > 0...
1234.                 issues.append({'type': '同义文本标签不一致', 'severity': '高', 'rule': 'Similar...
1235.                 break
1236.     for issue in issues:
1237.         execute("""INSERT INTO quality_issues(sample_id,annotation_id,issue_type,seve...
1238.             VALUES(?,?,?,?,?,?,?,?)""", (sample_id, issue.get('annotation_id')...
1239.     status = '自动质检异常' if issues else '已保存'
1240.     execute("UPDATE samples SET status=?, qc_status=? WHERE id=?", (status, '待质检', sa...
1241.     log_action(username, '执行自动质检', f"样本{sample_id} 检出 {len(issues)} 个问题")
1242.     return issues
1243. def issues_for_project(project_id: int) -> list[dict]:
1244.     return fetch_all("""SELECT qi.*, s.sample_code, s.filename, s.sample_type FROM qu...
1245.         JOIN samples s ON qi.sample_id=s.id WHERE s.project_id=? ORDE...
1246. def submit_for_review(sample_ids: list[int], username: str) -> tuple[bool, str]:
1247.     total_issues = 0
1248.     for sid in sample_ids:
1249.         issues = run_quality_check(sid, username)
1250.         total_issues += len([i for i in issues if i['severity'] in ('高', '中')])

```

```

1251.     if total_issues:
1252.         return False, f"仍有 {total_issues} 个中高风险问题未处理，不能提交审核。"
1253.     for sid in sample_ids:
1254.         execute("UPDATE samples SET status='已提交', qc_status='待质检' WHERE id=?", (sid,))
1255.         log_action(username, '提交审核', f"提交 {len(sample_ids)} 个样本，强制检查通过")
1256.         return True, f"当前任务包共 {len(sample_ids)} 个样本，自动质检通过，可提交审核。"
1257. def reviewer_decision(sample_id: int, passed: bool, username: str, comment: str) -> N...
1258.     execute("UPDATE samples SET qc_status=?, status=? WHERE id=?", ('已通过' if passed e...
1259.     log_action(username, '人工质检结论', f"样本{sample_id} {'通过' if passed else '退回'}: {commen...
1260. from datetime import datetime
1261. from pathlib import Path
1262. from app.core.config import DOCS_DIR
1263. from app.core.database import fetch_all, execute, log_action
1264. from app.services.dataset_service import dashboard_metrics, get_project
1265. from app.services.rework_service import list_reworks
1266. def generate_report(project_id: int, report_type: str, username: str) -> Path:
1267.     project = get_project(project_id)
1268.     metrics = dashboard_metrics(project_id)
1269.     reworks = list_reworks(project_id)
1270.     issues = fetch_all("""SELECT qi.* FROM quality_issues qi JOIN samples s ON qi.sam...
1271.     title = f"{project['name']} - {report_type}"
1272.     path = DOCS_DIR / f"{project['code']}_{report_type}_{datetime.now().strftime('%Y%...
1273.     lines = [
1274.         f"# 人工智能模型训练样本智能标注与质量校验管理软件 - {report_type}",
1275.         "",
1276.         f"报告标题: {title}",
1277.         f"生成时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
1278.         f"生成用户: {username}",
1279.         "",
1280.         "## 一、数据集概况",
1281.         f"- 数据集名称: {project['name']}",
1282.         f"- 样本数量: {metrics.get('sample_total', 0)}",
1283.         f"- 已标注: {metrics.get('annotated_total', 0)}",
1284.         f"- 待质检: {metrics.get('qc_pending_total', 0)}",
1285.         f"- 返工数量: {metrics.get('rework_total', 0)}",
1286.         f"- 数据集健康度: {metrics.get('health_score', 0)}",
1287.         "",
1288.         "## 二、核心质量指标",
1289.         f"- 智能预标注采用率: {metrics.get('prelabel_adapt_rate', 0)}%",
1290.         f"- 自动质检通过率: {metrics.get('auto_pass_rate', 0):.1f}%",
1291.         f"- 人工质检通过率: {metrics.get('manual_pass_rate', 0):.1f}%",
1292.         f"- 多人一致性评分: {metrics.get('consensus_score', 0):.1f}",
1293.         f"- Ground Truth 抽检得分: {metrics.get('gt_score', 0):.1f}",
1294.         f"- 标签均衡度: {metrics.get('balance_score', 0):.1f}",
1295.         "",
1296.         "## 三、高频质量问题",
1297.     ]
1298.     if issues:
1299.         for i in issues[:20]:
1300.             lines.append(f"- [{i['severity']}] {i['issue_type']}: {i['position_text']}...

```

```

1301.     else:
1302.         lines.append("- 暂无未关闭质量问题。")
1303.     lines.extend(["", "## 四、返工闭环", ""])
1304.     if reworks:
1305.         for r in reworks[:20]:
1306.             lines.append(f"- {r['rework_code']}: {r['issue_type']}, 状态: {r['status']}, 要求...")
1307.     else:
1308.         lines.append("- 暂无返工任务。")
1309.     lines.extend(["", "## 五、交付建议", ""])
1310.     if metrics.get('health_score', 0) >= 90:
1311.         conclusion = "可交付训练。"
1312.     elif metrics.get('health_score', 0) >= 80:
1313.         conclusion = "建议完成关键返工后交付。"
1314.     else:
1315.         conclusion = "存在明显质量风险, 不建议直接交付训练。"
1316.     lines.append(conclusion)
1317.     for advice in metrics.get('health_advice', []):
1318.         lines.append(f"- {advice}")
1319.     path.write_text('\n'.join(lines), encoding='utf-8')
1320.     execute("INSERT INTO reports(project_id,report_type,title,file_path,conclusion,cr...")
1321.     log_action(username, '生成报告', f"{report_type} {path}")
1322.     return path
1323. def list_reports(project_id: int) -> list[dict]:
1324.     return fetch_all("SELECT * FROM reports WHERE project_id=? ORDER BY id DESC", (pr...
1325. from datetime import datetime, timedelta
1326. from app.core.database import fetch_all, fetch_one, execute, log_action
1327. def list_reworks(project_id: int | None = None) -> list[dict]:
1328.     if project_id:
1329.         return fetch_all("""SELECT r.*, s.sample_code, s.filename FROM rework_tasks r...
1330.         return fetch_all("SELECT * FROM rework_tasks ORDER BY id DESC")
1331. def create_rework(sample_id: int, reviewer: str, issue_type: str, issue_desc: str, re...
1332.     sample = fetch_one("SELECT * FROM samples WHERE id=?", (sample_id,))
1333.     code = f"RW-{datetime.now().strftime('%Y%m%d%H%M%S')}-{sample_id}"
1334.     rid = execute("""INSERT INTO rework_tasks(rework_code,sample_id,project_id,label...
1335.         VALUES(?,?,?,?,?,?,?,?)""", (code, sample_id, sample['project...
1336.     execute("UPDATE samples SET status='返工中', rework_count=rework_count+1 WHERE id=?"...
1337.     log_action(reviewer, '生成返工任务', f"{code} {issue_type}")
1338.     return rid
1339. def update_rework_status(rework_id: int, status: str, username: str, note: str = '') ...
1340.     execute("UPDATE rework_tasks SET status=?, second_review=? WHERE id=?", (status, ...
1341.     log_action(username, '更新返工状态', f"返工单{rework_id} -> {status} {note}")
1342. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
1343. from app.ui.utils import page_root, fill_table, secondary, make_project_combo
1344. from app.services.dataset_service import list_projects, list_rules, add_rule
1345. class AnnotationSpecPage(QWidget):
1346.     def __init__(self, user: dict):
1347.         super().__init__(); self.user = user
1348.         root, self.layout = page_root('标注规范管理')
1349.         wrap = QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
1350.         bar = QHBoxLayout(); self.project_combo = make_project_combo(list_projects())...

```

```

1351.         add = QPushButton('新增规范/质检规则'); add.clicked.connect(self.add_rule_dialog)
1352.         preview = secondary(QPushButton('弹出当前规范卡片')); preview.clicked.connect(self.pr...
1353.         bar.addWidget(QLabel('项目')); bar.addWidget(self.project_combo, 2); bar.addWid...
1354.         self.table = QTableWidget(); self.layout.addWidget(self.table, 1)
1355.     def refresh(self):
1356.         rows = list_rules(self.project_combo.currentData()) if self.project_combo.cur...
1357.         fill_table(self.table, rows, [('rule_type', '规则类型'), ('title', '规则名称'), ('severit...
1358.     def add_rule_dialog(self):
1359.         dlg = QDialog(self); dlg.setWindowTitle('新增标注规范'); form = QFormLayout(dlg)
1360.         rtype = QComboBox(); rtype.addItem('图像', '文本', '大模型评价', '返工', '交付检查'])
1361.         title = QLineEdit('疑难样本说明规则')
1362.         severity = QComboBox(); severity.addItem('高', '中', '低'])
1363.         content = QTextEdit('标注员将样本标记为疑难时，必须选择疑难类型并填写不少于 10 字的说明。')
1364.         for a,b in [('规则类型', rtype), ('规则名称', title), ('严重程度', severity), ('规则内容', content)]...
1365.         buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok|QDialogButtonBo...
1366.         buttons.accepted.connect(dlg.accept); buttons.rejected.connect(dlg.reject)
1367.         if dlg.exec():
1368.             add_rule(self.project_combo.currentData(), {'rule_type': rtype.currentText...
1369.             QMessageBox.information(self, '新增成功', '规范已生效，可在标注工作台随时查看。')
1370.             self.refresh()
1371.     def preview_rules(self):
1372.         rows = list_rules(self.project_combo.currentData())
1373.         text = '\n\n'.join([f"【{r['severity']}】{r['title']}\n{r['content']}" for r in...
1374.         QMessageBox.information(self, '当前项目标注规范', text)
1375. from PyQt6.QtWidgets import (
1376.     QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton, QListWidget, QListWidgetI...
1377.     QSplitter, QFrame, QTabWidget, QTableWidget, QTextEdit, QMessageBox, QComboBox,
1378.     QProgressBar, QInputDialog, QCheckBox, QGridLayout
1379. )
1380. from PyQt6.QtCore import Qt
1381. from app.ui.utils import page_root, fill_table, secondary, danger, success, attach_pa...
1382. from app.widgets.image_canvas import ImageCanvas
1383. from app.widgets.object_list_widget import ObjectListWidget
1384. from app.widgets.visual_widgets import MiniBarChart, WorkflowCard, RiskHeatmap, Check...
1385. from app.services.dataset_service import list_projects, list_samples, get_labels, ann...
1386. from app.services.prelabel_service import generate_prelabels_for_sample, accept_prela...
1387. from app.services.quality_service import run_quality_check, submit_for_review
1388. from app.core.database import fetch_all, fetch_one, execute, log_action
1389. class AnnotationWorkbench(QWidget):
1390.     def __init__(self, user: dict):
1391.         super().__init__()
1392.         self.user = user
1393.         self.project_id = None
1394.         self.sample_id = None
1395.         self.labels = []
1396.         root, self.layout = page_root('样本标注生产工作台', '五区联动：任务队列、专业画布、预标注修正、实时质检、提交门禁与
返工...
1397.         attach_page(self, root)
1398.         top = QFrame(); top.setProperty('card', True); top_l = QHBoxLayout(top); top...
1399.         self.project_combo = QComboBox(); self.project_combo.currentIndexChanged.conn...

```

```
1400.         self.task_info = QLabel('任务包: 等待加载'); self.task_info.setStyleSheet('font-weigh...
```

```

1401.         self.progress = QProgressBar(); self.progress.setFixedWidth(220)
1402.         self.quality_chip = pill('质量分 --', 'info')
1403.         top_l.addWidget(QLabel('项目')); top_l.addWidget(self.project_combo, 2); top_l....
1404.         save = success(QPushButton('保存并下一张')); save.clicked.connect(self.save_next)
1405.         check = QPushButton('实时自检'); check.clicked.connect(self.self_check)
1406.         submit = QPushButton('提交审核'); submit.clicked.connect(self.submit_review)
1407.         skip = secondary(QPushButton('跳过并说明')); skip.clicked.connect(self.skip_with_r...
1408.         top_l.addWidget(save); top_l.addWidget(check); top_l.addWidget(submit); top_l...
1409.         self.layout.addWidget(top)
1410.         splitter = QSplitter(Qt.Orientation.Horizontal)
1411.         splitter.setChildrenCollapsible(False)
1412.         left = QFrame(); left.setProperty('card', True); left.setMinimumWidth(280)
1413.         left_l = QVBoxLayout(left); left_l.setContentsMargins(12, 12, 12, 12); left_l...
1414.         left_l.addWidget(title_label('风险优先样本队列', '队列根据状态、返工、低置信、重复疑似和 Ground Truth 自动...
1415.         filter_bar = QHBoxLayout(); self.filter_combo = QComboBox(); self.filter_comb...
1416.         self.filter_combo.currentIndexChanged.connect(self.refresh_samples)
1417.         self.sort_combo = QComboBox(); self.sort_combo.addItem(['风险优先', '任务顺序', '返工次数'...
1418.         self.sort_combo.currentIndexChanged.connect(self.refresh_samples)
1419.         filter_bar.addWidget(self.filter_combo); filter_bar.addWidget(self.sort_combo...
1420.         self.sample_list = QListWidget(); self.sample_list.currentRowChanged.connect(...
1421.         left_l.addWidget(self.sample_list, 1)
1422.         self.queue_heat = RiskHeatmap('队列风险构成', {})
1423.         left_l.addWidget(self.queue_heat)
1424.         splitter.addWidget(left)
1425.         center = QFrame(); center.setProperty('workbenchPanel', True)
1426.         center_l = QVBoxLayout(center); center_l.setContentsMargins(12, 12, 12, 12); ...
1427.         tools = QHBoxLayout()
1428.         self.draw_btn = QPushButton('矩形'); self.draw_btn.setCheckable(True); self.dra...
1429.         fit = secondary(QPushButton('适窗')); fit.clicked.connect(lambda: (self.canvas....
1430.         raw = secondary(QPushButton('原图')); raw.clicked.connect(lambda: (setattr(self...
1431.         gen = QPushButton('AI 预标注'); gen.clicked.connect(self.run_prelabel)
1432.         accept = QPushButton('全接受'); accept.clicked.connect(self.accept_all)
1433.         delete = danger(QPushButton('删对象')); delete.clicked.connect(self.delete_selec...
1434.         self.prelabel_layer = QCheckBox('预标注'); self.prelabel_layer.setChecked(True);...
1435.         self.quality_layer = QCheckBox('质检层'); self.quality_layer.setChecked(True); s...
1436.         self.gt_layer = QCheckBox('GT 层'); self.gt_layer.stateChanged.connect(lambda _...
1437.         for w in [self.draw_btn, fit, raw, gen, accept, delete, self.prelabel_layer, ...
1438.         tools.addStretch(); center_l.addLayout(tools)
1439.         self.canvas = ImageCanvas(); self.canvas.bbox_created.connect(self.create_box...
1440.         center_l.addWidget(self.canvas, 1)
1441.         status = QFrame(); status.setStyleSheet('QFrame{background:#111827;border-rad...
1442.         st_l=QHBoxLayout(status); st_l.setContentsMargins(10,6,10,6)
1443.         self.bottom_status = QLabel('快捷键: 1-person 2-helmet 3-no_helmet 4-vest | 当前耗时 00...
1444.         self.recent_hint = QLabel('最近提示: 无')
1445.         st_l.addWidget(self.bottom_status, 2); st_l.addWidget(self.recent_hint, 1)
1446.         center_l.addWidget(status)
1447.         splitter.addWidget(center)
1448.         right = QFrame(); right.setProperty('card', True); right.setMinimumWidth(330)
1449.         right_l = QVBoxLayout(right); right_l.setContentsMargins(10, 10, 10, 10)
1450.         self.right_tabs = QTabWidget()

```



```

1451.         label_tab = QWidget(); label_l = QVBoxLayout(label_tab); label_l.addWidget(ti...
1452.         self.label_buttons_frame = QVBoxLayout(); label_l.addLayout(self.label_button...
1453.         self.object_table = ObjectListWidget(); self.object_table.object_selected.con...
1454.         issue_tab = QWidget(); issue_l = QVBoxLayout(issue_tab)
1455.         self.issue_chart = MiniBarChart('问题严重度', [])
1456.         self.issue_table = QTableWidgetItem()
1457.         issue_l.addWidget(self.issue_chart); issue_l.addWidget(self.issue_table)
1458.         gate_tab = QWidget(); gate_l = QVBoxLayout(gate_tab); gate_l.addWidget(title_...
1459.         self.gate = ChecklistWidget([]); gate_l.addWidget(self.gate)
1460.         self.rule_text = QTextEdit(); self.rule_text.setReadOnly(True)
1461.         self.rework_text = QTextEdit(); self.rework_text.setPlaceholderText('返工模式下显示质...
1462.         ai_tab = QWidget(); ai_l = QVBoxLayout(ai_tab); ai_l.addWidget(title_label('A...
1463.         self.ai_suggestions = QTextEdit(); self.ai_suggestions.setReadOnly(True); ai_...
1464.         difficult_tab = QWidget(); d_l = QVBoxLayout(difficult_tab)
1465.         self.difficult_type = QComboBox(); self.difficult_type.addItem(['目标过小', '目标遮挡...
1466.         self.difficult_note = QTextEdit(); self.difficult_note.setPlaceholderText('请填写...
1467.         mark = QPushButton('标记疑难并进入仲裁池'); mark.clicked.connect(self.mark_difficult)
1468.         d_l.addWidget(QLabel('疑难类型')); d_l.addWidget(self.difficult_type); d_l.addWid...
1469.         self.right_tabs.addTab(label_tab, '标签')
1470.         self.right_tabs.addTab(self.object_table, '对象')
1471.         self.right_tabs.addTab(issue_tab, '实时质检')
1472.         self.right_tabs.addTab(gate_tab, '提交门禁')
1473.         self.right_tabs.addTab(ai_tab, 'AI 建议')
1474.         self.right_tabs.addTab(self.rework_text, '返工')
1475.         self.right_tabs.addTab(self.rule_text, '规范')
1476.         self.right_tabs.addTab(difficult_tab, '疑难')
1477.         right_l.addWidget(self.right_tabs)
1478.         splitter.addWidget(right)
1479.         splitter.setSizes([300, 540, 340])
1480.         self.layout.addWidget(splitter, 1)
1481.     def refresh(self):
1482.         current = self.project_combo.currentData()
1483.         self.project_combo.blockSignals(True); self.project_combo.clear()
1484.         for p in list_projects():
1485.             if p['data_type'] == '图像': self.project_combo.addItem(f"{p['code']} | {p['n...
1486.         if current:
1487.             idx = self.project_combo.findData(current)
1488.             if idx >= 0: self.project_combo.setCurrentIndex(idx)
1489.         self.project_combo.blockSignals(False)
1490.         self.project_changed()
1491.     def project_changed(self):
1492.         self.project_id = self.project_combo.currentData()
1493.         self.labels = get_labels(self.project_id) if self.project_id else []
1494.         self.build_label_buttons(); self.refresh_samples()
1495.         rules = list_rules(self.project_id) if self.project_id else []
1496.         self.rule_text.setText('\n\n'.join([f"【{r['severity']}】 {r['title']}\n{r['cont...
1497.     def build_label_buttons(self):
1498.         while self.label_buttons_frame.count():
1499.             item = self.label_buttons_frame.takeAt(0)
1500.             if item.widget(): item.widget().deleteLater()

```

```

1501.         for lab in self.labels:
1502.             btn = QPushButton(f"{lab['shortcut']} {lab['name']} | {lab['description']}[...
1503.             btn.setStyleSheet(f"background:{lab['color']};color:white;text-align:left...
1504.             btn.setToolTip(f"正例: {lab.get('positive_example','')}\n 反例: {lab.get('negati...
1505.             btn.clicked.connect(lambda _, n=lab['name']: self.set_label(n))
1506.             self.label_buttons_frame.addWidget(btn)
1507. def set_label(self, label: str):
1508.     self.canvas.set_current_label(label)
1509.     self.bottom_status.setText(f'当前绘制标签: {label} | 在画布拖拽新增目标框 | 快捷键已同步')
1510. def risk_score(self, s: dict) -> int:
1511.     score = 0
1512.     score += 30 if '异常' in s.get('status','') or s.get('qc_status') == '待质检' else 0
1513.     score += int(s.get('rework_count') or 0) * 18
1514.     score += 20 if s.get('is_low_confidence') else 0
1515.     score += 15 if s.get('is_duplicate') else 0
1516.     score += 10 if s.get('is_ground_truth') else 0
1517.     score += 10 if '模糊' in (s.get('risk_tags') or '') else 0
1518.     return score
1519. def refresh_samples(self):
1520.     self.sample_list.clear()
1521.     if not self.project_id: return
1522.     rows = list_samples(self.project_id, 'image')
1523.     f = self.filter_combo.currentText()
1524.     def keep(s):
1525.         if f == '全部': return True
1526.         if f == '未标注': return s['status'] == '未开始'
1527.         if f == '预标注待确认': return '预标注' in s['status']
1528.         if f == '质检异常': return '异常' in s['status'] or s['qc_status'] == '待质检'
1529.         if f == '返工样本': return s['rework_count'] > 0 or '返工' in s['status']
1530.         if f == '低置信样本': return s['is_low_confidence'] == 1
1531.         if f == '疑难样本': return '疑难' in s['status'] or '疑难' in (s.get('risk_tags')...
1532.         if f == '重复疑似样本': return s['is_duplicate'] == 1
1533.         if f == 'Ground Truth': return s['is_ground_truth'] == 1
1534.         return True
1535.     self.samples = [s for s in rows if keep(s)]
1536.     sort = self.sort_combo.currentText()
1537.     if sort == '风险优先': self.samples.sort(key=self.risk_score, reverse=True)
1538.     elif sort == '返工次数': self.samples.sort(key=lambda s: s.get('rework_count') or...)
1539.     elif sort == '低置信优先': self.samples.sort(key=lambda s: s.get('is_low_confidenc...
1540.     for s in self.samples:
1541.         risk = self.risk_score(s)
1542.         prefix = ' ' if risk >= 45 else ('△' if risk >= 25 else '✓')
1543.         item = QListWidgetItem(f"{prefix} {s['sample_code']} | {s['filename']}\n 状态:..."
1544.         item.setData(Qt.ItemDataRole.UserRole, s['id'])
1545.         item.setToolTip(f"风险分: {risk}\n 点击加载到中央画布")
1546.         self.sample_list.addItem(item)
1547.     total = max(1, len(rows)); done = sum(1 for s in rows if s['status'] in ('已保存...
1548.     self.progress.setValue(int(done/total*100))
1549.     risk_data = {
1550.         '待确认预标注': sum(1 for s in rows if '预标注' in s['status']),

```

```

1551.         '返工样本': sum(1 for s in rows if s['rework_count'] > 0),
1552.         '低置信': sum(1 for s in rows if s['is_low_confidence']),
1553.         '重复疑似': sum(1 for s in rows if s['is_duplicate']),
1554.         '待质检': sum(1 for s in rows if s['qc_status'] == '待质检'),
1555.         'Ground Truth': sum(1 for s in rows if s['is_ground_truth']),
1556.     }
1557.     self.queue_heat.set_data(risk_data)
1558.     if self.samples and self.sample_list.currentRow() < 0: self.sample_list.setCu...
1559. def sample_selected(self, row: int):
1560.     if row < 0 or row >= len(getattr(self, 'samples', [])): return
1561.     s = self.samples[row]; self.sample_id = s['id']
1562.     quality = max(0, 100 - self.risk_score(s))
1563.     self.quality_chip.setText(f'质量分 {quality}')
1564.     tone = 'success' if quality >= 80 else ('warn' if quality >= 60 else 'danger')
1565.     self.quality_chip.setProperty('pill', tone); self.quality_chip.style().unpoli...
1566.     self.task_info.setText(f"任务包 TB-IMG-202606 | 样本 {s['sample_code']} | {s['status']}...
1567.     self.reload_current_sample()
1568. def reload_current_sample(self):
1569.     if not self.sample_id: return
1570.     s = fetch_one('SELECT * FROM samples WHERE id=?', (self.sample_id,))
1571.     if not s: return
1572.     anns = annotations_for_sample(self.sample_id)
1573.     color_map = {l['name']: l['color'] for l in self.labels}
1574.     issue_rows = fetch_all("SELECT * FROM quality_issues WHERE sample_id=? AND st...
1575.     issue_ann_ids = {i['annotation_id'] for i in issue_rows if i.get('annotation_...
1576.     for a in anns:
1577.         a['color'] = color_map.get(a['label'], '#22c55e')
1578.         if a['id'] in issue_ann_ids: a['issue'] = True
1579.     self.canvas.load_image(s['file_path'], anns)
1580.     self.object_table.set_objects(anns)
1581.     fill_table(self.issue_table, issue_rows, [('issue_type', '问题'), ('severity', '严重...
1582.     counter = {'高': 0, '中': 0, '低': 0}
1583.     for i in issue_rows: counter[i.get('severity', '中')] = counter.get(i.get('seve...
1584.     self.issue_chart.set_data(list(counter.items()), '问题严重度', '')
1585.     unconfirmed = sum(1 for a in anns if a.get('source') == '预标注' and a.get('stat...
1586.     blockers = [
1587.         ('预标注全部确认', unconfirmed == 0, f'剩余 {unconfirmed} 个待确认候选'),
1588.         ('无中高风险质检问题', not any(i['severity'] in ('高', '中') for i in issue_rows), f'...
1589.         ('对象数量合理', len(anns) > 0, f'当前对象 {len(anns)} 个'),
1590.         ('返工意见已处理', not ('返工' in s.get('status', '')) and len(issue_rows) > 0), '返工...
1591.         ('疑难说明完整', '疑难' not in s.get('status', ''), '疑难样本必须填写说明'),
1592.     ]
1593.     self.gate.set_items(blockers)
1594.     reworks = fetch_all('SELECT * FROM rework_tasks WHERE sample_id=? ORDER BY id...
1595.     if reworks:
1596.         rw = reworks[0]
1597.         self.rework_text.setText(f"返工单: {rw['rework_code']}\n 状态: {rw['status']}\n 截止...
1598.     else:
1599.         self.rework_text.setText('当前样本未处于返工模式。')
1600.     suggestions = []

```

```

1601.         if unconfirmed: suggestions.append(f'有 {unconfirmed} 个预标注候选未确认, 建议逐个接受/修改/删除。')
1602.         if s.get('is_low_confidence'): suggestions.append('该样本被标记为低置信, 建议放大检查小目标与类别边界。')
1603.         if s.get('is_duplicate'): suggestions.append('该样本疑似重复, 建议与导入体检结果比对后决定是否保留。')
1604.         if issue_rows: suggestions.append('存在打开的自动质检问题, 请先定位问题框并修复后再提交。')
1605.         if not suggestions: suggestions.append('当前样本风险较低, 可保存并继续下一张。')
1606.         self.ai_suggestions.setText('\n'.join(f'• {x}' for x in suggestions))
1607.         self.bottom_status.setText(f"快捷键: 1-person 2-helmet 3-no_helmet 4-vest | 对象 {len...
1608.         self.recent_hint.setText(f"最近提示: {issue_rows[0]['issue_type']} if issue_rows el...
1609.     def run_prelabel(self):
1610.         if not self.sample_id: return
1611.         n = generate_prelabels_for_sample(self.sample_id, self.user['username'])
1612.         QMessageBox.information(self, '智能预标注完成', f'模型候选框已生成 {n} 个。请在画布中接受、修改或删除。')
1613.         self.reload_current_sample()
1614.     def accept_all(self):
1615.         if not self.sample_id: return
1616.         n = accept_prelabels(self.sample_id, self.user['username'])
1617.         QMessageBox.information(self, '预标注已确认', f'已接受 {n} 个预标注对象, 系统将统计预标注采用率。')
1618.         self.reload_current_sample()
1619.     def create_box_from_canvas(self, box: dict):
1620.         if not self.sample_id: return
1621.         create_bbox(self.sample_id, box['label'], box['x'], box['y'], box['w'], box['...
1622.         self.reload_current_sample()
1623.     def object_clicked(self, annotation_id: int):
1624.         self.canvas.select_box(annotation_id)
1625.     def delete_selected(self):
1626.         aid = self.canvas.selected_id
1627.         if not aid:
1628.             QMessageBox.warning(self, '未选择对象', '请先在画布或对象列表选择一个标注对象。')
1629.             return
1630.         reason, ok = QDialog.getItem(self, '删除原因', '请选择删除预标注/标注的原因', ['误检', '类别错误...
1631.         if ok:
1632.             delete_annotation(aid, self.user['username'], reason)
1633.             self.reload_current_sample()
1634.     def self_check(self):
1635.         if not self.sample_id: return
1636.         issues = run_quality_check(self.sample_id, self.user['username'])
1637.         QMessageBox.information(self, '自动自检完成', f'本样本检出 {len(issues)} 个问题。严重问题会在画布和右侧...
1638.         self.reload_current_sample()
1639.     def submit_review(self):
1640.         if not self.project_id: return
1641.         ids = [s['id'] for s in list_samples(self.project_id, 'image') if s.get('assi...
1642.         ok, msg = submit_for_review(ids[:8], self.user['username'])
1643.         QMessageBox.information(self, '提交前强制检查', msg)
1644.         self.refresh_samples()
1645.     def save_next(self):
1646.         if not self.sample_id: return
1647.         execute("UPDATE samples SET status='已保存' WHERE id=?", (self.sample_id,))
1648.         log_action(self.user['username'], '保存并下一张', f"样本{self.sample_id}")
1649.         row = self.sample_list.currentRow()
1650.         if row + 1 < self.sample_list.count(): self.sample_list.setCurrentRow(row + 1)

```

```

1651.         else: QMessageBox.information(self, '任务包完成', '当前筛选队列已处理完, 可执行提交审核。')
1652.         self.refresh_samples()
1653.     def skip_with_reason(self):
1654.         if not self.sample_id: return
1655.         reason, ok = QDialog.get_text(self, '跳过说明', '请说明跳过原因(疑难、图片不可读、等待仲裁等)')
1656.         if ok and reason.strip():
1657.             execute("UPDATE samples SET status='疑难待仲裁', risk_tags=risk_tags||? WHERE ...
1658.             log_action(self.user['username'], '跳过样本', f"样本{self.sample_id} {reason}")
1659.             QMessageBox.information(self, '已跳过并记录', '样本已进入疑难/待仲裁队列。')
1660.             self.refresh_samples()
1661.     def mark_difficult(self):
1662.         if not self.sample_id: return
1663.         note = self.difficult_note.toPlainText().strip()
1664.         if len(note) < 5:
1665.             QMessageBox.warning(self, '说明不足', '请填写更具体的疑难说明。')
1666.             return
1667.         execute("UPDATE samples SET status='疑难待仲裁', risk_tags=risk_tags||? WHERE id=?...
1668.         log_action(self.user['username'], '标记疑难样本', f"样本{self.sample_id} {self.diffic...
1669.         QMessageBox.information(self, '已进入疑难池', '该样本已标记为疑难, 等待质检员或管理员仲裁。')
1670.         self.refresh_samples()
1671. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
1672. from app.ui.utils import page_root, fill_table, make_project_combo, secondary
1673. from app.services.dataset_service import list_projects
1674. from app.services.consensus_service import calculate_consensus, list_consensus
1675. class ConsensusPage(QWidget):
1676.     def __init__(self, user: dict):
1677.         super().__init__(); self.user=user
1678.         root, self.layout = page_root('多人一致性分析')
1679.         wrap=QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
1680.         bar=QHBoxLayout(); self.project_combo=make_project_combo(list_projects()); se...
1681.         run=QPushButton('执行一致性分析'); run.clicked.connect(self.run_consensus)
1682.         arb=secondary(QPushButton('将低一致性样本送仲裁')); arb.clicked.connect(self.send_arbit...
1683.         bar.addWidget(QLabel('项目')); bar.addWidget(self.project_combo,2); bar.addWidg...
1684.         self.table=QTableWidget(); self.layout.addWidget(self.table,1)
1685.     def refresh(self):
1686.         rows=list_consensus(self.project_combo.currentData()) if self.project_combo.c...
1687.         display=[]
1688.         for r in rows:
1689.             x=dict(r); x['iou_score']=f"{r['iou_score']*100:.1f}"; x['label_agreement...
1690.             fill_table(self.table, display, [('sample_code','样本'),('filename','文件'),('wor...
1691.     def run_consensus(self):
1692.         result=calculate_consensus(self.project_combo.currentData(), self.user['usern...
1693.         QMessageBox.information(self, '一致性分析完成', f"已生成 {result['created']} 条记录, 平均一致性评分 ...
1694.         self.refresh()
1695.     def send_arbitration(self):
1696.         count=0
1697.         for r in range(self.table.rowCount()):
1698.             if self.table.item(r,7) and self.table.item(r,7).text()=='是': count+=1
1699.         QMessageBox.information(self, '仲裁池更新', f'已将 {count} 条低一致性样本加入疑难仲裁池。')
1700. from collections import Counter

```

```

1701. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QGridLayout, QLabel, QTableWidgetItem, ...
1702. from PyQt6.QtCore import Qt
1703. from app.ui.utils import page_root, card, fill_table, secondary, attach_page, title_l...
1704. from app.widgets.metric_card import MetricCard
1705. from app.widgets.visual_widgets import DonutGauge, MiniBarChart, PipelineWidget, Tren...
1706. from app.services.dataset_service import dashboard_metrics, list_projects, update_pro...
1707. from app.core.database import fetch_all
1708. class DashboardPage(QWidget):
1709.     def __init__(self, user: dict):
1710.         super().__init__()
1711.         self.user = user
1712.         root, self.layout = page_root('数据集驾驶舱', '把样本生产、质量风险、返工闭环和版本交付放在同一个指挥视角, 不再是普
通统...
1713.         attach_page(self, root)
1714.         hero = QFrame(); hero.setProperty('hero', True)
1715.         hero_l = QHBoxLayout(hero); hero_l.setContentsMargins(22, 18, 22, 18); hero_l...
1716.         left = QVBoxLayout(); left.setSpacing(8)
1717.         self.project_label = QLabel(''); self.project_label.setStyleSheet('font-size:...
1718.         self.project_goal = QLabel(''); self.project_goal.setStyleSheet('color:#dbeaf...
1719.         chips = QHBoxLayout(); self.status_chip = pill('生产中', 'info'); self.role_chip...
1720.         left.addWidget(self.project_label); left.addWidget(self.project_goal); left.a...
1721.         action = QHBoxLayout();
1722.         self.project_combo = QComboBox(); self.project_combo.currentIndexChanged.conn...
1723.         refresh = QPushButton('重算健康度'); refresh.clicked.connect(self.recompute_health)
1724.         simulate = secondary(QPushButton('模拟今日质检扫描')); simulate.clicked.connect(self....
1725.         action.addWidget(QLabel('演示项目')); action.addWidget(self.project_combo, 1); ac...
1726.         hero_l.addLayout(left, 2)
1727.         self.gauge = DonutGauge('数据集健康度', 0, '分'); hero_l.addWidget(self.gauge)
1728.         self.trend = TrendLine('近 7 批次质量走势', []); self.trend.setStyleSheet('backgroun...
1729.         self.layout.addWidget(hero)
1730.         self.pipeline = PipelineWidget([]); pcard = card(); pl=QVBoxLayout(pcard); pl...
1731.         self.pipeline.stageClicked.connect(self.pipeline_clicked)
1732.         self.layout.addWidget(pcard)
1733.         self.grid = QGridLayout(); self.grid.setSpacing(12); self.layout.addLayout(se...
1734.         self.cards = []
1735.         splitter = QSplitter(Qt.Orientation.Horizontal)
1736.         left_panel = card(); lp=QVBoxLayout(left_panel); lp.addWidget(title_label('标签...
1737.         self.label_chart = MiniBarChart('标签分布', [])
1738.         self.label_chart.barClicked.connect(lambda k: QMessageBox.information(self, '...
1739.         lp.addWidget(self.label_chart); self.label_table = QTableWidgetItem(); lp.addWidg...
1740.         splitter.addWidget(left_panel)
1741.         right_panel = card(); rp=QVBoxLayout(right_panel); rp.addWidget(title_label('...
1742.         self.risk_heat = RiskHeatmap('风险热力图', {})
1743.         self.risk_heat.cellClicked.connect(lambda k: QMessageBox.information(self, '风...
1744.         rp.addWidget(self.risk_heat)
1745.         self.issue_table = QTableWidgetItem(); rp.addWidget(self.issue_table)
1746.         splitter.addWidget(right_panel)
1747.         splitter.setSizes([760, 720])
1748.         self.layout.addWidget(splitter, 1)
1749.         def refresh(self):

```

```
1750.         projects = list_projects()
```

```

1751.         current = self.project_combo.currentData() if hasattr(self, 'project_combo') ...
1752.         self.project_combo.blockSignals(True)
1753.         self.project_combo.clear()
1754.         for p in projects:
1755.             self.project_combo.addItem(f"{p['code']} | {p['name']}", p['id'])
1756.         if current:
1757.             idx = self.project_combo.findData(current)
1758.             if idx >= 0: self.project_combo.setCurrentIndex(idx)
1759.         self.project_combo.blockSignals(False)
1760.         pid = self.project_combo.currentData() if projects else None
1761.         project = next((p for p in projects if p['id'] == pid), projects[0] if projec...
1762.         m = dashboard_metrics(pid)
1763.         if not project or not m: return
1764.         self.project_label.setText(f"{project['code']} | {m.get('project_name', '--')}")
1765.         self.project_goal.setText(f"训练目标: {project.get('training_goal', '--')} | 版本: {proj...
1766.         health = float(m.get('health_score', 0)); self.gauge.set_value(health)
1767.         self.status_chip.setText(project.get('status', '生产中'))
1768.         self.status_chip.setProperty('pill', 'success' if health >= 90 else ('warn' i...
1769.         self.status_chip.style().unpolish(self.status_chip); self.status_chip.style()...
1770.         for c in self.cards: c.setParent(None)
1771.         self.cards.clear()
1772.         items = [
1773.             ('样本入库', m.get('sample_total', 0), '图像/文本已完成体检入库', '数据体检'),
1774.             ('标注完成率', f"{m.get('annotated_rate', 0):.1f}%", '进入工作台继续处理未完成样本', '标注任务'),
1775.             ('待质检', m.get('qc_pending_total', 0), '优先由质检员处理高风险样本', '质量校验'),
1776.             ('返工闭环', m.get('rework_total', 0), '退回、二检、仲裁的闭环任务', '返工'),
1777.             ('预标注采用率', f"{m.get('prelabel_adapt_rate', 0):.1f}%", '衡量模型辅助标注有效性', '智能预标注'),
1778.             ('一致性评分', f"{m.get('consensus_score', 0):.1f}%", '多人重叠标注 / 文本标签一致', '一致性'),
1779.             ('GT 抽检得分', f"{m.get('gt_score', 0):.1f}%", '隐藏标准答案质量检测', 'Ground Truth'),
1780.             ('可交付版本', m.get('version_total', 0), '冻结、导出、回滚记录', '版本交付'),
1781.         ]
1782.         for i, (t, v, s, key) in enumerate(items):
1783.             cardw = WorkflowCard(t, str(v), s, key, ['blue', 'green', 'amber', 'red'])[i ...
1784.             self.grid.addWidget(cardw, i//4, i%4); self.cards.append(cardw)
1785.         total = max(1, m.get('sample_total', 0))
1786.         stages = [
1787.             ('入库', m.get('sample_total', 0)), ('已标注', m.get('annotated_total', 0)), ('待...
1788.             ('返工', m.get('rework_total', 0)), ('通过', m.get('qc_pass_total', 0)), ('版本', ...
1789.         ]
1790.         self.pipeline.set_stages(stages)
1791.         labels = sorted(m.get('label_counts', {}).items(), key=lambda x: x[1], reverse...
1792.         self.label_chart.set_data(labels, '标签分布: 对象数量', '')
1793.         labels_rows = [{'label': k, 'count': v, 'ratio': f"{v/max(1, sum(m.get('label_...
1794.         fill_table(self.label_table, labels_rows, [('label', '标签'), ('count', '数量'), ('...
1795.         issues = fetch_all("""SELECT issue_type, severity, COUNT(*) c FROM quality_is...
1796.                             WHERE s.project_id=? AND qi.status!='已关闭' GROUP BY issu...
1797.         risk_data = {r['issue_type']: int(r['c']) for r in issues}
1798.         if not risk_data:
1799.             risk_data = {'低置信样本': m.get('low_quality_total', 0), '重复样本': int(m.get('du...
1800.         self.risk_heat.set_data(risk_data)

```



```

1801.         advice = [{'advice': x, 'status': '待处理' if '建议' in x or '不建议' in x or '偏低' in...
1802.         fill_table(self.issue_table, advice, [('advice','扣分项 / 优化建议'), ('status','处理状...
1803.         trend = [('批次1', 76), ('批次2', 80), ('批次3', 83), ('批次4', 82), ('批次5', 86), ('当...
1804.         self.trend.set_data(trend, '近 7 批次质量走势', '分')
1805.     def card_clicked(self, key: str):
1806.         QMessageBox.information(self, '业务入口联动', f'已聚焦「{key}」：真实系统中会自动带入筛选条件并跳转到对应队列。当...
1807.     def pipeline_clicked(self, stage: str):
1808.         QMessageBox.information(self, '流程管线定位', f'已选择流程阶段: {stage}\n 建议查看对应样本队列、质检中心或版本...
1809.     def simulate_scan(self):
1810.         QMessageBox.information(self, '今日质检扫描', '已模拟完成: 新增 2 条中风险提示, 推荐优先处理低置信、重复疑似和返工
超...
1811.     def recompute_health(self):
1812.         projects = list_projects()
1813.         if projects:
1814.             score = update_project_health(self.project_combo.currentData() or project...
1815.             QMessageBox.information(self, '健康度已更新', f'数据集健康度重新计算完成: {score} 分。')
1816.             self.refresh()
1817. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWid...
1818. from PyQt6.QtCore import Qt
1819. from app.ui.utils import page_root, fill_table, secondary, attach_page, title_label, ...
1820. from app.widgets.visual_widgets import MiniBarChart, DonutGauge, WorkflowCard, Pipeli...
1821. from app.services.dataset_service import list_projects, create_project, dashboard_met...
1822. class DatasetPage(QWidget):
1823.     def __init__(self, user: dict):
1824.         super().__init__(); self.user = user
1825.         root, self.layout = page_root('数据集项目管理', '以项目交付状态、样本规模、质量健康度和生产阶段管理数据集，而不是简
单增...
1826.         attach_page(self, root)
1827.         bar = QFrame(); bar.setProperty('card', True); bl=QHBoxLayout(bar); bl.setCon...
1828.         add = QPushButton('创建数据集项目'); add.clicked.connect(self.add_project)
1829.         refresh = secondary(QPushButton('刷新生产态势')); refresh.clicked.connect(self.refr...
1830.         self.project_chip = pill('项目数 --', 'info')
1831.         bl.addWidget(self.project_chip); bl.addWidget(add); bl.addWidget(refresh); bl...
1832.         self.layout.addWidget(bar)
1833.         self.grid = QGridLayout(); self.grid.setSpacing(12); self.layout.addLayout(se...
1834.         self.cards=[]
1835.         splitter = QSplitter(Qt.Orientation.Horizontal)
1836.         left = QFrame(); left.setProperty('card', True); ll=QVBoxLayout(left); ll.add...
1837.         self.type_chart = MiniBarChart('项目类型分布', [])
1838.         self.health_chart = MiniBarChart('健康度排行', [])
1839.         ll.addWidget(self.type_chart); ll.addWidget(self.health_chart)
1840.         splitter.addWidget(left)
1841.         right = QFrame(); right.setProperty('card', True); rl=QVBoxLayout(right); rl....
1842.         self.table = QTableWidget(); rl.addWidget(self.table)
1843.         splitter.addWidget(right); splitter.setSizes([450, 980])
1844.         self.layout.addWidget(splitter, 1)
1845.     def refresh(self):
1846.         rows = list_projects()
1847.         self.project_chip.setText(f'项目数 {len(rows)}')
1848.         for c in self.cards: c.setParent(None)

```

```
1849.         self.cards.clear()
1850.         total_samples = sum(int(r.get('sample_count') or 0) for r in rows)
```

```

1851.         avg_health = sum(float(r.get('health_score') or 0) for r in rows) / max(1,len...
1852.         active = sum(1 for r in rows if r.get('status') in ('生产中','质检中'))
1853.         labels = sum(int(r.get('label_count') or 0) for r in rows)
1854.         items=[('生产中项目', active, '需要持续导入、标注、质检', 'blue'),('样本总量', total_samples, '已入库...
1855.         for i,(t,v,d,tone) in enumerate(items):
1856.             wc=WorkflowCard(t,str(v),d,t,tone); wc.clicked.connect(lambda x: QMessage...
1857.             self.grid.addWidget(wc,0,i); self.cards.append(wc)
1858.         type_counts={}
1859.         for r in rows: type_counts[r.get('project_type') or '未知']=type_counts.get(r.g...
1860.         self.type_chart.set_data(list(type_counts.items()), '项目类型分布', '')
1861.         self.health_chart.set_data([(r['code'].replace('DS-', ''), float(r.get('health...
1862.         enriched=[]
1863.         for r in rows:
1864.             m=dashboard_metrics(r['id'])
1865.             enriched.append({'*r', 'annotated_rate': f"{m.get('annotated_rate',0):.1f}...
1866.         fill_table(self.table, enriched, [('code','项目编号'),('name','项目名称'),('project_t...
1867.     def add_project(self):
1868.         dlg = QDialog(self); dlg.setWindowTitle('创建数据集项目'); form = QFormLayout(dlg)
1869.         code = QLineEdit('DS-NEW-2026-003'); name = QLineEdit('新增模型训练样本数据集')
1870.         ptype = QComboBox(); ptype.addItem('图像目标检测','图像分类','图像分割','文本分类','实体识别','问答...
1871.         dtype = QComboBox(); dtype.addItem('图像','文本','表格','多模态'])
1872.         task = QLineEdit('智能预标注 + 人工修正 + 质量复核')
1873.         goal = QLineEdit('用于训练行业场景识别模型')
1874.         deadline = QLineEdit('2026-08-01')
1875.         for label, widget in [('项目编号',code),('项目名称',name),('项目类型',ptype),('数据类型',dtyp...
1876.         buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok|QDialogButtonBo...
1877.         form.addRow(buttons); buttons.accepted.connect(dlg.accept); buttons.rejected....
1878.         if dlg.exec():
1879.             create_project({'code':code.text(),'name':name.text(),'project_type':ptyp...
1880.             QMessageBox.information(self, '创建成功', '数据集项目已创建, 并进入生产配置待完善状态。')
1881.             self.refresh()
1882. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
1883. from PyQt6.QtCore import Qt
1884. from app.ui.utils import page_root, fill_table, make_project_combo, secondary, succes...
1885. from app.widgets.visual_widgets import DonutGauge, MiniBarChart, ChecklistWidget, Wor...
1886. from app.services.dataset_service import list_projects, dashboard_metrics
1887. from app.services.export_service import delivery_check, export_yolo, export_coco, exp...
1888. from app.core.database import fetch_all
1889. class ExportPage(QWidget):
1890.     def __init__(self, user: dict):
1891.         super().__init__(); self.user=user
1892.         root,self.layout=page_root('训练格式导出中心', '不是“点按钮导出文件”, 而是执行交付门禁、数据划分、格式适配和导出留
            痕。')
1893.         attach_page(self, root)
1894.         bar=QFrame(); bar.setProperty('card', True); bl=QHBoxLayout(bar); bl.setConte...
1895.         self.project_combo=make_project_combo(list_projects()); self.project_combo.cu...
1896.         check=QPushButton('执行交付前检查'); check.clicked.connect(self.run_check)
1897.         yolo=success(QPushButton('导出 YOLO')); yolo.clicked.connect(lambda:self.export...
1898.         coco=QPushButton('导出 COCO JSON'); coco.clicked.connect(lambda:self.export('co...
1899.         voc=QPushButton('导出 Pascal VOC'); voc.clicked.connect(lambda:self.export('voc'))

```

1900. json1=QPushButton('导出文本 JSONL'); json1.clicked.connect(lambda:self.export('js...

```

1901.         for w in [QLabel('项目'),self.project_combo,check,yolo,coco,voc,jsonl]: bl.addW...
1902.         bl.addStretch(); self.layout.addWidget(bar)
1903.         top = QSplitter(Qt.Orientation.Horizontal)
1904.         gate_card = QFrame(); gate_card.setProperty('card', True); gl=QVBoxLayout(gat...
1905.         gl.addWidget(title_label('交付门禁', '任一阻断项未通过时仍可演示导出, 但系统会在导出记录中保留风险说明。'))
1906.         self.gate = ChecklistWidget([]); gl.addWidget(self.gate)
1907.         self.check_text=QTextEdit(); self.check_text.setReadOnly(True); self.check_te...
1908.         top.addWidget(gate_card)
1909.         visual = QFrame(); visual.setProperty('card', True); vl=QVBoxLayout(visual)
1910.         vl.addWidget(title_label('训练 / 验证 / 测试划分', '用于检查划分比例是否完整、类别是否均衡。'))
1911.         charts=QHBoxLayout(); self.split_chart=MiniBarChart('数据划分', [('train',80),('v...
1912.         charts.addWidget(self.split_chart); charts.addWidget(self.health); vl.addLayo...
1913.         self.flow = PipelineWidget([('检查',0),('冻结',0),('导出',0),('报告',0)])
1914.         vl.addWidget(self.flow)
1915.         top.addWidget(visual)
1916.         top.setSizes([700, 760])
1917.         self.layout.addWidget(top)
1918.         grid=QGridLayout(); grid.setSpacing(12); self.layout.addLayout(grid)
1919.         self.format_cards=[]
1920.         for i,(fmt,desc,tone) in enumerate([
1921.             ('YOLO','目标检测训练: images / labels / data.yaml','blue'),('COCO JSON','检测/分割训...
1922.             ('Pascal VOC','XML 标注与 JPEGImages 目录','amber'),('JSONL','文本分类、指令微调、偏好数据',...
1923.             c=WorkflowCard(fmt,'可导出',desc,fmt,tone); c.clicked.connect(lambda f: QMes...
1924.             grid.addWidget(c,0,i); self.format_cards.append(c)
1925.         self.table=QTableWidget(); self.layout.addWidget(QLabel('导出留痕记录')); self.layo...
1926.     def refresh(self):
1927.         pid=self.project_combo.currentData()
1928.         if not pid: return
1929.         metrics = dashboard_metrics(pid)
1930.         self.health.set_value(float(metrics.get('health_score',0)))
1931.         rows=fetch_all('SELECT * FROM export_records WHERE project_id=? ORDER BY id D...
1932.         fill_table(self.table, rows, [('format','格式'),('output_path','输出路径'),('check...
1933.         self.flow.set_stages([('检查', 1 if self.check_text.toPlainText() else 0), ('版本...
1934.         self.update_gate()
1935.     def update_gate(self):
1936.         pid=self.project_combo.currentData()
1937.         if not pid: return
1938.         ok,problems=delivery_check(pid)
1939.         checks = [
1940.             ('无未标注样本', not any('未标注样本' in p for p in problems), '未标注样本会影响训练集完整度'),
1941.             ('无待质检样本', not any('未质检样本' in p for p in problems), '所有样本应至少完成自动/人工质检'),
1942.             ('返工任务已关闭', not any('返工样本' in p for p in problems), '返工样本不应进入交付包'),
1943.             ('健康度达到阈值', not any('健康度' in p for p in problems), '建议 80 分以上再冻结交付'),
1944.             ('标签分布可解释', True, '系统将输出标签分布与长尾提示'),
1945.             ('划分比例完整', True, '默认 80 / 10 / 10, 可在版本页调整'),
1946.         ]
1947.         self.gate.set_items(checks)
1948.     def run_check(self):
1949.         ok,problems=delivery_check(self.project_combo.currentData())
1950.         self.check_text.setText('交付检查通过, 可以导出训练集。' if ok else '交付检查发现问题: \n- ' + '\n- '...)

```

```

1951.         self.update_gate()
1952.         QMessageBox.information(self,'交付检查完成','通过' if ok else f'发现 {len(problems)} 个风...
1953.         self.refresh()
1954.     def export(self, fmt):
1955.         pid=self.project_combo.currentData()
1956.         ok, problems = delivery_check(pid)
1957.         if fmt=='yolo': path=export_yolo(pid,self.user['username'])
1958.         elif fmt=='coco': path=export_coco(pid,self.user['username'])
1959.         elif fmt=='voc': path=export_pascal_voc(pid,self.user['username'])
1960.         else: path=export_text_jsonl(pid,self.user['username'])
1961.         risk = '交付检查通过' if ok else '带风险导出: ' + ': '.join(problems[:3])
1962.         self.check_text.setText(risk)
1963.         QMessageBox.information(self,'导出完成',f'训练格式已导出: \n{path}\n\n{risk}')
1964.         self.refresh()
1965. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
1966. from app.ui.utils import page_root, fill_table, make_project_combo
1967. from app.services.dataset_service import list_projects, list_samples
1968. from app.core.database import fetch_all, execute, log_action
1969. class GroundTruthPage(QWidget):
1970.     def __init__(self, user: dict):
1971.         super().__init__(); self.user=user
1972.         root,self.layout=page_root('Ground Truth 抽检')
1973.         wrap=QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
1974.         bar=QHBoxLayout(); self.project_combo=make_project_combo(list_projects()); se...
1975.         setbtn=QPushButton('将选中样本设为标准答案'); setbtn.clicked.connect(self.set_gt)
1976.         mixbtn=QPushButton('混入隐藏质检样本'); mixbtn.clicked.connect(self.mix_gt)
1977.         scorebtn=QPushButton('计算抽检得分'); scorebtn.clicked.connect(self.score_gt)
1978.         for w in [QLabel('项目'),self.project_combo,setbtn,mixbtn,scorebtn]: bar.addWid...
1979.         bar.addStretch(); self.layout.addLayout(bar)
1980.         self.samples=QTableWidget(); self.layout.addWidget(QLabel('样本列表')); self.layo...
1981.         self.gt=QTableWidget(); self.layout.addWidget(QLabel('Ground Truth 标准答案库')); ...
1982.     def refresh(self):
1983.         pid=self.project_combo.currentData(); rows=list_samples(pid)[:20] if pid else []
1984.         fill_table(self.samples, rows, [('id','ID'),('sample_code','样本编号'),('sample_t...
1985.         gt=fetch_all("""SELECT gt.*, s.sample_code, s.filename FROM ground_truth gt J...
1986.         fill_table(self.gt, gt, [('sample_code','样本'),('filename','文件'),('score','得分'...
1987.     def selected_sample_id(self):
1988.         row=self.samples.currentRow()
1989.         if row<0: QMessageBox.warning(self,'未选择','请先选择样本。'); return None
1990.         return int(self.samples.item(row,0).text())
1991.     def set_gt(self):
1992.         sid=self.selected_sample_id();
1993.         if not sid: return
1994.         ans, ok=QInputDialog.getMultiLineText(self,'标准答案 JSON','填写标准答案摘要', '{"labels":...
1995.         if ok:
1996.             execute("UPDATE samples SET is_ground_truth=1 WHERE id=?", (sid,))
1997.             execute("INSERT INTO ground_truth(sample_id,answer_json,score,conclusion,...
1998.             log_action(self.user['username'],'设置 Ground Truth',f'样本{sid}')
1999.             QMessageBox.information(self,'设置成功','该样本已进入标准答案库, 可混入普通任务进行隐藏抽检。')
2000.         self.refresh()

```

```

2001.     def mix_gt(self):
2002.         QMessageBox.information(self, '隐藏样本已混入', '系统已按 8% 抽检比例将 Ground Truth 样本混入普通任务队列...
2003.     def score_gt(self):
2004.         rows=fetch_all("SELECT score FROM ground_truth")
2005.         score=sum(r['score'] for r in rows)/len(rows) if rows else 0
2006.         QMessageBox.information(self, '抽检得分', f'当前 Ground Truth 平均抽检得分: {score:.1f}。低于 85...
2007. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
2008. from PyQt6.QtCore import Qt
2009. from app.ui.utils import page_root, fill_table, secondary, make_project_combo, attach...
2010. from app.widgets.visual_widgets import PipelineWidget, RiskHeatmap, ChecklistWidget, ...
2011. from app.services.dataset_service import list_projects
2012. from app.services.import_service import import_images, import_text_csv, inspect_image...
2013. class ImportPage(QWidget):
2014.     def __init__(self, user: dict):
2015.         super().__init__(); self.user = user
2016.         root, self.layout = page_root('样本导入与数据体检', '导入即体检: 重复、损坏、尺寸异常、空文本、标签缺失和结构问题会
    进入...
2017.         attach_page(self, root)
2018.         bar = QFrame(); bar.setProperty('card', True); bl=QHBoxLayout(bar); bl.setCon...
2019.         self.project_combo = make_project_combo(list_projects())
2020.         img_btn = QPushButton('导入图片文件夹并体检'); img_btn.clicked.connect(self.import_imag...
2021.         txt_btn = QPushButton('导入文本 CSV 并体检'); txt_btn.clicked.connect(self.import_csv)
2022.         demo_btn = secondary(QPushButton('仅执行体检预览')); demo_btn.clicked.connect(self.i...
2023.         for w in [QLabel('目标项目'), self.project_combo, img_btn, txt_btn, demo_btn]: bl...
2024.         bl.addStretch(); self.layout.addWidget(bar)
2025.         self.progress = QProgressBar(); self.progress.setValue(0); self.layout.addWid...
2026.         self.flow = PipelineWidget([( '读取', 0), ( '体检', 0), ( '入库', 0), ( '报告', 0)])
2027.         flow_card=QFrame(); flow_card.setProperty('card', True); fl=QVBoxLayout(flow_...
2028.         self.layout.addWidget(flow_card)
2029.         splitter=QSplitter(Qt.Orientation.Horizontal)
2030.         left=QFrame(); left.setProperty('card', True); ll=QVBoxLayout(left); ll.addWi...
2031.         self.risk=RiskHeatmap('体检问题热力', {})
2032.         self.checklist=ChecklistWidget([])
2033.         ll.addWidget(self.risk); ll.addWidget(QLabel('入库门禁')); ll.addWidget(self.chec...
2034.         splitter.addWidget(left)
2035.         right=QFrame(); right.setProperty('card', True); rl=QVBoxLayout(right)
2036.         self.summary = QLabel('选择数据后将自动识别重复、近似重复、损坏、尺寸异常、空文本、标签缺失等问题。')
2037.         self.summary.setWordWrap(True); rl.addWidget(self.summary)
2038.         self.table = QTableWidgetItem(); rl.addWidget(self.table, 1)
2039.         splitter.addWidget(right); splitter.setSizes([420, 980])
2040.         self.layout.addWidget(splitter, 1)
2041.     def refresh(self):
2042.         cur=self.project_combo.currentData(); self.project_combo.clear()
2043.         for p in list_projects(): self.project_combo.addItem(f"{p['code']} | {p['name']}...
2044.         if cur:
2045.             idx=self.project_combo.findData(cur)
2046.             if idx>=0: self.project_combo.setCurrentIndex(idx)
2047.     def import_image_folder(self):
2048.         folder = QFileDialog.getExistingDirectory(self, '选择图片文件夹')
2049.         if not folder: return

```

```
2050. self.progress.setValue(25); self.flow.set_stages([('读取',1),('体检',0),('入库',0),...
```



```

2051.         rows = import_images(self.project_combo.currentData(), folder, self.user['use...
2052.         self.progress.setValue(100); self.flow.set_stages([('读取',len(rows)),('体检',len...
2053.         self.show_image_rows(rows)
2054.         QMessageBox.information(self, '导入完成', f'共导入/体检 {len(rows)} 张图片, 异常项可在表格中定位。')
2055.     def import_csv(self):
2056.         path, _ = QFileDialog.getOpenFileName(self, '选择 CSV 样本表', '', 'CSV Files (*.csv)')
2057.         if not path: return
2058.         self.progress.setValue(30)
2059.         rows = import_text_csv(self.project_combo.currentData(), path, self.user['use...
2060.         self.progress.setValue(100); self.flow.set_stages([('读取',len(rows)),('体检',len...
2061.         fill_table(self.table, rows, [('line', '行号'),('text', '文本'),('label', '标签'),('st...
2062.         risk_counts={})
2063.         for r in rows:
2064.             for part in str(r.get('issues','')).split(';'):
2065.                 if part: risk_counts[part]=risk_counts.get(part,0)+1
2066.         self.risk.set_data(risk_counts or {'无异常': len(rows)})
2067.         self.checklist.set_items([('文本非空', not any('空' in str(r.get('issues','')) for...
2068.         self.summary.setText(f'文本导入完成: {len(rows)} 条, 异常 {sum(1 for r in rows if r["iss...
2069.     def inspect_only(self):
2070.         folder = QFileDialog.getExistingDirectory(self, '选择图片文件夹用于体检预览')
2071.         if not folder: return
2072.         rows = inspect_image_folder(folder)
2073.         self.show_image_rows(rows)
2074.         self.progress.setValue(100); self.flow.set_stages([('读取',len(rows)),('体检',len...
2075.     def show_image_rows(self, rows):
2076.         display = [{**r, 'issues': ';'.join(r.get('issues', []))} for r in rows]
2077.         fill_table(self.table, display, [('filename', '文件名'),('width', '宽'),('height', '...
2078.         risk_counts={})
2079.         for r in rows:
2080.             for i in r.get('issues',[]): risk_counts[i]=risk_counts.get(i,0)+1
2081.         self.risk.set_data(risk_counts or {'无异常': len(rows)})
2082.         self.checklist.set_items([('图片可读取', not any('损坏' in ';'.join(r.get('issues',[...
2083.         self.summary.setText(f'图像体检摘要: 总数 {len(rows)}, 异常/待确认 {sum(1 for r in rows if r...
2084. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
2085. from app.ui.utils import page_root, fill_table, secondary, make_project_combo
2086. from app.services.dataset_service import list_projects, get_labels, add_label
2087. class LabelSchemaPage(QWidget):
2088.     def __init__(self, user: dict):
2089.         super().__init__(); self.user = user
2090.         root, self.layout = page_root('标签体系管理')
2091.         wrap = QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
2092.         bar = QHBoxLayout(); self.project_combo = make_project_combo(list_projects())...
2093.         add = QPushButton('新增标签'); add.clicked.connect(self.add_label_dialog)
2094.         export = secondary(QPushButton('模拟导出标签体系')); export.clicked.connect(lambda: Q...
2095.         bar.addWidget(QLabel('项目')); bar.addWidget(self.project_combo, 2); bar.addWid...
2096.         self.table = QTableWidgetItem(); self.layout.addWidget(self.table, 1)
2097.     def refresh(self):
2098.         if self.project_combo.count() == 0:
2099.             self.project_combo.clear()
2100.             for p in list_projects(): self.project_combo.addItem(f"{p['code']} | {p['na...

```

```

2101.         rows = get_labels(self.project_combo.currentData()) if self.project_combo.cur...
2102.         fill_table(self.table, rows, [('name', '标签名称'), ('code', '编码'), ('color', '颜色'), ('...
2103.     def add_label_dialog(self):
2104.         dlg = QDialog(self); dlg.setWindowTitle('新增标签'); form = QFormLayout(dlg)
2105.         name = QLineEdit('new_label'); code = QLineEdit('new_label'); color = QLineEdit...
2106.         pick = QPushButton('选择颜色'); pick.setProperty('secondary', True); pick.clicked...
2107.         ltype = QComboBox(); ltype.addItems(['分类标签', '目标框标签', '多边形标签', '点标签', '线段标签', '实体标...
2108.         shortcut = QLineEdit('N'); required = QCheckBox('必填'); exclusive = QCheckBox(...
2109.         desc = QTextEdit('请输入标签定义、边界规则和适用场景。')
2110.         pos = QTextEdit('正例：目标清晰、符合该标签定义。'); neg = QTextEdit('反例：容易混淆但不属于该标签的情况。')
2111.         for a,b in [('标签名称', name), ('标签编码', code), ('颜色值', color), ('', pick), ('标签类型', ltype...
2112.         buttons = QDialogButtonBox(QDialogButtonBox.StandardButton.Ok|QDialogButtonBo...
2113.         buttons.accepted.connect(dlg.accept); buttons.rejected.connect(dlg.reject)
2114.         if dlg.exec():
2115.             add_label(self.project_combo.currentData(), {'name': name.text(), 'code': co...
2116.             QMessageBox.information(self, '新增成功', '标签已加入当前项目，标注工作台会同步显示。')
2117.             self.refresh()
2118. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWid...
2119. from app.ui.utils import page_root, fill_table, secondary
2120. from app.core.database import fetch_all
2121. class LogPage(QWidget):
2122.     def __init__(self, user: dict):
2123.         super().__init__(); self.user=user
2124.         root,self.layout=page_root('操作日志')
2125.         wrap=QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
2126.         bar=QHBoxLayout(); self.search=QLineEdit(); self.search.setPlaceholderText('输...
2127.         btn=QPushButton('搜索'); btn.clicked.connect(self.refresh)
2128.         clear=secondary(QPushButton('清空筛选')); clear.clicked.connect(lambda:(self.sear...
2129.         bar.addWidget(QLabel('关键字')); bar.addWidget(self.search,2); bar.addWidget(btn...
2130.         self.table=QTableWidget(); self.layout.addWidget(self.table,1)
2131.     def refresh(self):
2132.         kw=self.search.text().strip()
2133.         if kw:
2134.             rows=fetch_all("SELECT * FROM operation_logs WHERE username LIKE ? OR act...
2135.         else:
2136.             rows=fetch_all('SELECT * FROM operation_logs ORDER BY id DESC LIMIT 300')
2137.         fill_table(self.table, rows, [('username', '用户'), ('action', '动作'), ('detail', '详情...
2138. from PyQt6.QtWidgets import (
2139.     QWidget, QVBoxLayout, QHBoxLayout, QLabel, QLineEdit, QPushButton, QFrame,
2140.     QMessageBox, QDialog, QComboBox, QFormLayout
2141. )
2142. from PyQt6.QtCore import Qt
2143. from app.core.config import APP_NAME
2144. from app.core.database import fetch_one, execute, log_action
2145. from app.ui.main_window import MainWindow
2146. class RegisterDialog(QDialog):
2147.     def __init__(self, parent=None):
2148.         super().__init__(parent)
2149.         self.setWindowTitle('注册新账号')
2150.         self.setFixedSize(430, 420)

```

```

2151.         root = QVBoxLayout(self)
2152.         root.setContentsMargins(28, 24, 28, 24)
2153.         title = QLabel('注册新账号')
2154.         title.setStyleSheet('font-size:22px;font-weight:900;color:#0f172a;')
2155.         desc = QLabel('用于本地演示环境。注册后默认启用，可立即登录系统。')
2156.         desc.setWordWrap(True)
2157.         desc.setStyleSheet('color:#64748b;')
2158.         root.addWidget(title)
2159.         root.addWidget(desc)
2160.         form = QFormLayout()
2161.         form.setLabelAlignment(Qt.AlignmentFlag.AlignRight)
2162.         self.username = QLineEdit()
2163.         self.username.setPlaceholderText('4-20 位字母、数字或下划线')
2164.         self.display_name = QLineEdit()
2165.         self.display_name.setPlaceholderText('例如：标注员 B')
2166.         self.password = QLineEdit()
2167.         self.password.setEchoMode(QLineEdit.EchoMode.Password)
2168.         self.password.setPlaceholderText('至少 6 位')
2169.         self.password2 = QLineEdit()
2170.         self.password2.setEchoMode(QLineEdit.EchoMode.Password)
2171.         self.role = QComboBox()
2172.         self.role.addItem('标注员', '质检员', '项目经理'])
2173.         form.addRow('用户名', self.username)
2174.         form.addRow('显示名', self.display_name)
2175.         form.addRow('密码', self.password)
2176.         form.addRow('确认密码', self.password2)
2177.         form.addRow('角色', self.role)
2178.         root.addLayout(form)
2179.         tip = QLabel('提示：管理员账号仍使用 admin / admin123。普通用户注册后只能访问其角色权限内页面。')
2180.         tip.setWordWrap(True)
2181.         tip.setStyleSheet('background:#eff6ff;color:#1d4ed8;border-radius:10px;paddin...
2182.         root.addWidget(tip)
2183.         btns = QHBoxLayout()
2184.         cancel = QPushButton('取消')
2185.         cancel.setProperty('secondary', True)
2186.         cancel.clicked.connect(self.reject)
2187.         ok = QPushButton('完成注册')
2188.         ok.clicked.connect(self.register)
2189.         btns.addStretch()
2190.         btns.addWidget(cancel)
2191.         btns.addWidget(ok)
2192.         root.addLayout(btns)
2193.     def register(self):
2194.         username = self.username.text().strip()
2195.         display = self.display_name.text().strip() or username
2196.         password = self.password.text()
2197.         password2 = self.password2.text()
2198.         if not username or len(username) < 4:
2199.             QMessageBox.warning(self, '注册失败', '用户名至少需要 4 位。')
2200.         return

```

```

2201.         if not username.replace('_', '').isalnum():
2202.             QMessageBox.warning(self, '注册失败', '用户名只能包含字母、数字或下划线。')
2203.             return
2204.         if len(password) < 6:
2205.             QMessageBox.warning(self, '注册失败', '密码至少需要 6 位。')
2206.             return
2207.         if password != password2:
2208.             QMessageBox.warning(self, '注册失败', '两次输入的密码不一致。')
2209.             return
2210.         if fetch_one('SELECT id FROM users WHERE username=?', (username,)):
2211.             QMessageBox.warning(self, '注册失败', '用户名已存在, 请换一个用户名。')
2212.             return
2213.         execute('INSERT INTO users(username,password,display_name,role,enabled) VALUE...
2214.             log_action(username, '注册账号', f'角色: {self.role.currentText()}')
2215.             QMessageBox.information(self, '注册成功', f'账号 {username} 已创建, 可以直接登录。')
2216.             self.accept()
2217.     class LoginWindow(QWidget):
2218.         def __init__(self):
2219.             super().__init__()
2220.             self.setWindowTitle(APP_NAME)
2221.             self.resize(960, 600)
2222.             root = QHBoxLayout(self)
2223.             root.setContentsMargins(0, 0, 0, 0)
2224.             left = QFrame(); left.setStyleSheet('background:#0f172a;color:white;')
2225.             lyt = QVBoxLayout(left); lyt.setContentsMargins(42, 40, 42, 40)
2226.             title = QLabel(APP_NAME); title.setWordWrap(True); title.setStyleSheet('font-...
2227.             sub = QLabel('AI 训练样本入库、智能预标注、人工修正、自动质检、返工闭环、版本交付一体化桌面平台')
2228.             sub.setWordWrap(True); sub.setStyleSheet('font-size:15px;color:#cbd5e1;')
2229.             stats = QLabel('内置演示: 安全帽/交通/工业缺陷图像数据集, 客服、金融、医疗等文本数据集\n支持 YOLO、COCO、Pascal
VOC...
2230.             stats.setWordWrap(True); stats.setStyleSheet('color:#93c5fd;margin-top:20px;')
2231.             lyt.addWidget(title); lyt.addWidget(sub); lyt.addSpacing(30); lyt.addWidget(s...
2232.             card = QFrame(); card.setProperty('card', True); card.setFixedWidth(380)
2233.             form = QVBoxLayout(card); form.setContentsMargins(30, 32, 30, 30)
2234.             h = QLabel('登录系统'); h.setStyleSheet('font-size:24px;font-weight:800;color:#0f...
2235.             self.username = QLineEdit('admin'); self.username.setPlaceholderText('用户名')
2236.             self.password = QLineEdit('admin123'); self.password.setPlaceholderText('密码')...
2237.             tip = QLabel('可用: admin/admin123、labeler/123456、reviewer/123456、manager/123456...
2238.             tip.setWordWrap(True); tip.setStyleSheet('color:#64748b;')
2239.             btn = QPushButton('登录并进入生产平台'); btn.clicked.connect(self.login)
2240.             register_btn = QPushButton('注册新账号')
2241.             register_btn.setProperty('secondary', True)
2242.             register_btn.clicked.connect(self.open_register)
2243.             form.addWidget(h); form.addSpacing(12); form.addWidget(QLabel('用户名')); form.a...
2244.             rightWrap = QFrame(); rw = QVBoxLayout(rightWrap); rw.setAlignment(Qt.Alignme...
2245.             root.addWidget(left, 3); root.addWidget(rightWrap, 2)
2246.             self.main = None
2247.         def open_register(self):
2248.             dlg = RegisterDialog(self)
2249.             if dlg.exec():

```

```
2250.         self.username.setText(dlg.username.text().strip())
```

```

2251.         self.password.clear()
2252.         self.password.setFocus()
2253.     def login(self):
2254.         user = fetch_one('SELECT * FROM users WHERE username=? AND password=? AND ena...
2255.         if not user:
2256.             QMessageBox.warning(self, '登录失败', '用户名或密码不正确, 或账号已被禁用。')
2257.             return
2258.         log_action(user['username'], '登录系统', f"角色: {user['role']}")
2259.         self.main = MainWindow(user)
2260.         self.main.show()
2261.         self.close()
2262. from PyQt6.QtWidgets import QMainWindow, QWidget, QHBoxLayout, QVBoxLayout, QLabel, Q...
2263. from PyQt6.QtCore import Qt
2264. from app.core.config import APP_NAME
2265. from app.core.permissions import can_access
2266. from app.ui.dashboard_page import DashboardPage
2267. from app.ui.dataset_page import DatasetPage
2268. from app.ui.import_page import ImportPage
2269. from app.ui.label_schema_page import LabelSchemaPage
2270. from app.ui.annotation_spec_page import AnnotationSpecPage
2271. from app.ui.annotation_workbench import AnnotationWorkbench
2272. from app.ui.text_annotation_page import TextAnnotationPage
2273. from app.ui.quality_center_page import QualityCenterPage
2274. from app.ui.consensus_page import ConsensusPage
2275. from app.ui.ground_truth_page import GroundTruthPage
2276. from app.ui.rework_page import ReworkPage
2277. from app.ui.version_page import VersionPage
2278. from app.ui.export_page import ExportPage
2279. from app.ui.report_page import ReportPage
2280. from app.ui.log_page import LogPage
2281. from app.ui.utils import ghost, pill
2282. class MainWindow(QMainWindow):
2283.     def __init__(self, user: dict):
2284.         super().__init__()
2285.         self.user = user
2286.         self.setWindowTitle(APP_NAME)
2287.         self.resize(1320, 820)
2288.         self.setMinimumSize(1120, 720)
2289.         root = QWidget(); root.setObjectName('mainRoot'); self.setCentralWidget(root)
2290.         main = QHBoxLayout(root); main.setContentsMargins(0, 0, 0, 0); main.setSpacin...
2291.         self.nav = QFrame(); self.nav.setFixedWidth(246)
2292.         self.nav.setStyleSheet('''
2293.             QFrame{background:#08111f;color:white;border:none;}
2294.             QLabel#brand{font-size:18px;font-weight:900;color:white;line-height:1.4;}
2295.             QLabel#brandSub{font-size:12px;color:#93c5fd;}
2296.             QLabel#navGroup{font-size:12px;color:#64748b;font-weight:800;margin-top:8...
2297.             QPushButton{background:transparent;text-align:left;padding:8px 10px;borde...
2298.             QPushButton:hover{background:#132033;color:white;}
2299.             QPushButton:checked{background:qlineargradient(x1:0,y1:0,x2:1,y2:0,stop:0...
2300.             QPushButton:disabled{color:#475569;}

```

```

2301.         '')
2302.     nav_layout = QVBoxLayout(self.nav); nav_layout.setContentsMargins(12, 12, 12,...
2303.     title = QLabel(APP_NAME); title.setObjectName('brand'); title.setWordWrap(True)
2304.     subtitle = QLabel('训练样本生产中台 / 质量闭环引擎'); subtitle.setObjectName('brandSub')
2305.     nav_layout.addWidget(title); nav_layout.addWidget(subtitle)
2306.     role_line = QHBoxLayout(); role_line.addWidget(pill(user['display_name'], 'in...
2307.     scroll = QScrollArea(); scroll.setWidgetResizable(True); scroll.setFrameShape...
2308.     nav_inner = QWidget(); nav_inner.setStyleSheet('background:transparent;')
2309.     self.nav_buttons_layout = QVBoxLayout(nav_inner); self.nav_buttons_layout.set...
2310.     scroll.addWidget(nav_inner); nav_layout.addWidget(scroll, 1)
2311.     shell = QWidget(); shell_layout = QVBoxLayout(shell); shell_layout.setContent...
2312.     topbar = QFrame(); topbar.setFixedHeight(58); topbar.setStyleSheet('QFrame{ba...
2313.     top_l = QHBoxLayout(topbar); top_l.setContentsMargins(18, 0, 18, 0)
2314.     self.page_title = QLabel(''); self.page_title.setStyleSheet('font-size:18px;f...
2315.     self.page_hint = QLabel(''); self.page_hint.setStyleSheet('color:#64748b;')
2316.     title_box = QVBoxLayout(); title_box.setSpacing(2); title_box.addWidget(self....
2317.     top_l.addLayout(title_box); top_l.addStretch()
2318.     self.state_chip = pill('本地 SQLite · 演示数据', 'info'); top_l.addWidget(self.state...
2319.     self.quick_submit = QPushButton('进入工作台'); self.quick_submit.clicked.connect(l...
2320.     top_l.addWidget(self.quick_submit)
2321.     shell_layout.addWidget(topbar)
2322.     self.stack = QStackedWidget(); shell_layout.addWidget(self.stack, 1)
2323.     self.pages = []
2324.     groups = [
2325.         ('总览与项目', [(('数据集驾驶舱', '📊', DashboardPage), ('数据集项目管理', '📁', DatasetPage),...
2326.         ('生产配置', [(('标签体系管理', '🏷️', LabelSchemaPage), ('标注规范管理', '📝', AnnotationSpe...
2327.         ('标注生产', [(('样本标注生产工作台', '🔍', AnnotationWorkbench), ('文本标注工作台', '📄', TextA...
2328.         ('质量闭环', [(('质量校验中心', '✅', QualityCenterPage), ('多人一致性分析', '👥', ConsensusP...
2329.         ('交付与审计', [(('数据集版本管理', '📅', VersionPage), ('训练格式导出中心', '📤', ExportPage), ...
2330.     ]
2331.     for group_name, page_classes in groups:
2332.         group_label = QLabel(group_name); group_label.setObjectName('navGroup'); ...
2333.         for name, icon, cls in page_classes:
2334.             page = cls(user)
2335.             self.stack.addWidget(page)
2336.             btn = QPushButton(f'{icon} {name}'); btn.setCheckable(True)
2337.             btn.clicked.connect(lambda checked=False, idx=len(self.pages), n=name...
2338.             if not can_access(user['role'], name):
2339.                 btn.setEnabled(False)
2340.                 btn.setToolTip('当前角色无权限访问此模块')
2341.                 self.nav_buttons_layout.addWidget(btn)
2342.                 self.pages.append((name, page, btn))
2343.     self.nav_buttons_layout.addStretch()
2344.     logout = ghost(QPushButton('退出登录')); logout.clicked.connect(self.close)
2345.     nav_layout.addWidget(logout)
2346.     main.addWidget(self.nav); main.addWidget(shell, 1)
2347.     for i, (n, _, b) in enumerate(self.pages):
2348.         if b.isEnabled(): self.switch_page(i, n); break
2349.     def switch_by_name(self, name: str):
2350.         for idx, (n, _, btn) in enumerate(self.pages):

```

```

2351.         if n == name:
2352.             self.switch_page(idx, n)
2353.             return
2354.     def switch_page(self, idx: int, name: str):
2355.         if not self.pages[idx][2].isEnabled():
2356.             QMessageBox.warning(self, '权限不足', '当前角色无权访问该功能。')
2357.             return
2358.         for _, _, btn in self.pages: btn.setChecked(False)
2359.         self.pages[idx][2].setChecked(True)
2360.         self.stack.setCurrentIndex(idx)
2361.         self.page_title.setText(name)
2362.         hints = {
2363.             '数据集驾驶舱': '从交付视角看样本、标注、质检、返工和版本是否达标。',
2364.             '样本标注生产工作台': '按风险优先处理预标注、人工修正、实时自检和返工说明。',
2365.             '质量校验中心': '结合自动规则、人工复核和返工 workflow 处理高风险样本。',
2366.             '训练格式导出中心': '执行交付前检查并导出 YOLO / COCO / VOC / JSONL。',
2367.         }
2368.         self.page_hint.setText(hints.get(name, '围绕 AI 训练样本生产、质量校验和数据集交付提供真实业务操作。'))
2369.         page = self.pages[idx][1]
2370.         if hasattr(page, 'refresh'): page.refresh()
2371.     from collections import Counter
2372.     from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
2373.     from PyQt6.QtCore import Qt
2374.     from app.ui.utils import page_root, fill_table, secondary, danger, success, make_proj...
2375.     from app.widgets.visual_widgets import MiniBarChart, PipelineWidget, RiskHeatmap, Wor...
2376.     from app.services.dataset_service import list_projects, list_samples, dashboard_metrics
2377.     from app.services.quality_service import issues_for_project, run_quality_check, revie...
2378.     from app.services.rework_service import create_rework
2379.     from app.core.database import fetch_all
2380.     class QualityCenterPage(QWidget):
2381.         def __init__(self, user: dict):
2382.             super().__init__(); self.user = user
2383.             root, self.layout = page_root('质量校验中心', '从“问题表”升级为质量闭环作战台：风险优先、批量扫描、人工复核、返工
单、...
2384.             attach_page(self, root)
2385.             bar = QFrame(); bar.setProperty('card', True); bar_l = QHBoxLayout(bar); bar...
2386.             self.project_combo = make_project_combo(list_projects()); self.project_combo...
2387.             scan = QPushButton('批量执行自动质量校验'); scan.clicked.connect(self.batch_scan)
2388.             passbtn = success(QPushButton('人工复核通过')); passbtn.clicked.connect(lambda: sel...
2389.             rework = danger(QPushButton('退回返工')); rework.clicked.connect(self.rework_sele...
2390.             locate = secondary(QPushButton('问题定位说明')); locate.clicked.connect(self.locate...
2391.             for w in [QLabel('项目'), self.project_combo, scan, passbtn, rework, locate]: b...
2392.             bar_l.addStretch(); self.layout.addWidget(bar)
2393.             self.summary_grid = QGridLayout(); self.summary_grid.setSpacing(12); self.lay...
2394.             self.summary_cards = []
2395.             flow_card = QFrame(); flow_card.setProperty('card', True); fl=QVBoxLayout(flo...
2396.             self.flow = PipelineWidget([]); fl.addWidget(self.flow)
2397.             self.layout.addWidget(flow_card)
2398.             splitter = QSplitter(Qt.Orientation.Horizontal)
2399.             left = QFrame(); left.setProperty('card', True); ll=QVBoxLayout(left)

```



```
2400.    ll.addWidget(title_label('高风险样本推荐队列', '优先展示低置信、返工多次、自动异常、Ground Truth 抽检相关样本。'))
```

```

2401.         self.sample_table = QTableWidget(); ll.addWidget(self.sample_table, 2)
2402.         self.review_gate = ChecklistWidget([]); ll.addWidget(QLabel('复核门禁')); ll.addW...
2403.         splitter.addWidget(left)
2404.         right = QFrame(); right.setProperty('card', True); rl=QVBoxLayout(right)
2405.         rl.addWidget(title_label('问题分布与打开问题', '质量问题按照严重程度、规则和样本定位展示, 支持联动返工。'))
2406.         charts = QHBoxLayout(); self.severity_chart=MiniBarChart('严重程度分布', []); self.t...
2407.         charts.addWidget(self.severity_chart); charts.addWidget(self.type_heat); rl.a...
2408.         self.table = QTableWidget(); rl.addWidget(self.table, 2)
2409.         splitter.addWidget(right)
2410.         splitter.setSizes([610, 870])
2411.         self.layout.addWidget(splitter, 1)
2412.     def refresh(self):
2413.         self.project_combo.blockSignals(True); cur = self.project_combo.currentData()...
2414.         for p in list_projects(): self.project_combo.addItem(f"{p['code']} | {p['name']}...
2415.         if cur:
2416.             idx = self.project_combo.findData(cur)
2417.             if idx >= 0: self.project_combo.setCurrentIndex(idx)
2418.         self.project_combo.blockSignals(False)
2419.         pid = self.project_combo.currentData()
2420.         if not pid: return
2421.         metrics = dashboard_metrics(pid)
2422.         for c in self.summary_cards: c.setParent(None)
2423.         self.summary_cards.clear()
2424.         data = [
2425.             ('打开问题', str(len([i for i in issues_for_project(pid) if i.get('status') !=...
2426.             ('质检通过率', f"{metrics.get('manual_pass_rate',0):.1f}%", '人工质检通过样本比例', 'gre...
2427.             ('一致性评分', f"{metrics.get('consensus_score',0):.1f}%", '多人重叠标注综合得分', 'blue'),
2428.             ('返工率', f"{metrics.get('rework_rate',0)*100:.1f}%", '退回返工样本占比', 'amber'),
2429.         ]
2430.         for i,(t,v,d,tone) in enumerate(data):
2431.             wc=WorkflowCard(t,v,d,t,tone); wc.clicked.connect(lambda x: QMessageBox.i...
2432.             self.summary_grid.addWidget(wc,0,i); self.summary_cards.append(wc)
2433.         rows = issues_for_project(pid)
2434.         fill_table(self.table, rows, [('sample_code', '样本编号'),('filename', '文件/文本'),('i...
2435.         sev = Counter(r.get('severity', '中') for r in rows)
2436.         self.severity_chart.set_data([('高', sev.get('高',0)), ('中', sev.get('中',0)), (...
2437.         typ = Counter(r.get('issue_type', '未知') for r in rows)
2438.         self.type_heat.set_data(dict(typ.most_common(9)))
2439.         samples = list_samples(pid)
2440.         def risk(s):
2441.             return (35 if s['qc_status']=='待质检' else 0) + (s.get('rework_count') or 0...
2442.         samples = sorted(samples, key=risk, reverse=True)
2443.         risk_rows = [{*s, 'risk_score': risk(s), 'recommend': '立即复核' if risk(s)>=50 ...
2444.         fill_table(self.sample_table, risk_rows, [('id', '样本 ID'),('sample_code', '样本编号'...
2445.         gate_items = [
2446.             ('已完成自动质检', len(rows) >= 0, f'打开问题 {len(rows)} 条'),
2447.             ('高风险优先复核', any(r['risk_score'] >= 50 for r in risk_rows), '队列已按风险分排序'),
2448.             ('返工单可追踪', metrics.get('rework_total',0) >= 0, f"返工任务 {metrics.get('rewor...
2449.             ('交付风险可解释', bool(metrics.get('health_advice')), '健康度扣分项已生成'),
2450.         ]

```

```

2451.         self.review_gate.set_items(gate_items)
2452.         self.flow.set_stages([('自动扫描', len(samples)), ('打开问题', len(rows)), ('待复核', me...
2453.     def batch_scan(self):
2454.         pid = self.project_combo.currentData(); samples = list_samples(pid)[:40]
2455.         count = 0
2456.         for s in samples: count += len(run_quality_check(s['id'], self.user['username...
2457.         QMessageBox.information(self, '批量质检完成', f'已扫描 {len(samples)} 个样本, 累计发现 {count}...
2458.         self.refresh()
2459.     def selected_sample_id(self):
2460.         row = self.sample_table.currentRow()
2461.         if row < 0:
2462.             QMessageBox.warning(self, '未选择样本', '请先在高风险样本队列中选择一条样本。')
2463.             return None
2464.         return int(self.sample_table.item(row, 0).text())
2465.     def review_selected(self, passed: bool):
2466.         sid = self.selected_sample_id()
2467.         if not sid: return
2468.         reviewer_decision(sid, passed, self.user['username'], '人工复核结论由质量校验中心提交。')
2469.         QMessageBox.information(self, '质检结论已记录', '该样本已写入人工质检结论, 并同步更新样本状态。')
2470.         self.refresh()
2471.     def rework_selected(self):
2472.         sid = self.selected_sample_id()
2473.         if not sid: return
2474.         dlg = QDialog(self); dlg.setWindowTitle('退回返工'); form=QFormLayout(dlg)
2475.         issue=QTextEdit('helmet 漏标 1 处, person 框边界偏移, 预标注未完全确认。')
2476.         req=QTextEdit('请补充漏标对象, 调整偏移框, 点击重新自检后再二次提交; 若类别不确定, 请标记疑难进入仲裁。')
2477.         form.addRow('质检问题说明', issue); form.addRow('返工要求', req)
2478.         buttons=QDialogButtonBox(QDialogButtonBox.StandardButton.Ok|QDialogButtonBox....
2479.         if dlg.exec():
2480.             create_rework(sid, self.user['username'], '人工质检退回', issue.toPlainText(), ...
2481.             QMessageBox.information(self, '返工单已生成', '样本已退回标注员返工, 并进入返工闭环管理。')
2482.             self.refresh()
2483.     def locate_issue(self):
2484.         row = self.table.currentRow()
2485.         if row < 0:
2486.             QMessageBox.information(self, '问题定位', '请选择打开问题记录。'); return
2487.         vals = [self.table.item(row, c).text() if self.table.item(row,c) else '' for ...
2488.         QMessageBox.information(self, '问题定位说明', f'问题: {vals[2]}\n严重程度: {vals[3]}\n位置: {va...
2489. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWidgetItem...
2490. from pathlib import Path
2491. from app.ui.utils import page_root, fill_table, make_project_combo, secondary
2492. from app.services.dataset_service import list_projects
2493. from app.services.report_service import generate_report, list_reports
2494. class ReportPage(QWidget):
2495.     def __init__(self, user: dict):
2496.         super().__init__(); self.user=user
2497.         root,self.layout=page_root('报告生成中心')
2498.         wrap=QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
2499.         bar=QHBoxLayout(); self.project_combo=make_project_combo(list_projects()); se...
2500.         self.report_type=QComboBox(); self.report_type.addItem(['数据集导入体检报告', '标注任务进度报...

```

```

2501.         gen=QPushButton('生成报告'); gen.clicked.connect(self.generate)
2502.         openbtn=secondary(QPushButton('预览选中报告')); openbtn.clicked.connect(self.preview)
2503.         for w in [QLabel('项目'),self.project_combo,QLabel('报告类型'),self.report_type,gen...
2504.             bar.addStretch(); self.layout.addLayout(bar)
2505.         self.table=QTableWidget(); self.layout.addWidget(self.table,1)
2506.         self.preview_text=QTextEdit(); self.preview_text.setReadOnly(True); self.layo...
2507.     def refresh(self):
2508.         rows=list_reports(self.project_combo.currentData()) if self.project_combo.cur...
2509.         fill_table(self.table, rows, [('id','ID'),('report_type','类型'),('title','标题')]...
2510.     def generate(self):
2511.         path=generate_report(self.project_combo.currentData(), self.report_type.curre...
2512.         QMessageBox.information(self,'报告已生成',f'报告文件已生成: \n{path}')
2513.         self.refresh()
2514.     def preview(self):
2515.         row=self.table.currentRow()
2516.         if row<0: QMessageBox.warning(self,'未选择报告','请先选择一条报告记录。'); return
2517.         path=Path(self.table.item(row,3).text())
2518.         self.preview_text.setText(path.read_text(encoding='utf-8') if path.exists() e...
2519. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWid...
2520. from app.ui.utils import page_root, fill_table, make_project_combo, secondary
2521. from app.services.dataset_service import list_projects
2522. from app.services.rework_service import list_reworks, update_rework_status
2523. class ReworkPage(QWidget):
2524.     def __init__(self, user: dict):
2525.         super().__init__(); self.user=user
2526.         root,self.layout=page_root('返工闭环管理')
2527.         wrap=QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
2528.         bar=QHBoxLayout(); self.project_combo=make_project_combo(list_projects()); se...
2529.         start=QPushButton('领取/开始返工'); start.clicked.connect(lambda:self.update_status...
2530.         submit=QPushButton('返工完成并二次提交'); submit.clicked.connect(lambda:self.update_st...
2531.         review=QPushButton('二次质检通过'); review.clicked.connect(lambda:self.update_statu...
2532.         reject=secondary(QPushButton('再次退回/仲裁处理')); reject.clicked.connect(self.arbit...
2533.         for w in [QLabel('项目'),self.project_combo,start,submit,review,reject]: bar.ad...
2534.         bar.addStretch(); self.layout.addLayout(bar)
2535.         self.table=QTableWidget(); self.layout.addWidget(self.table,1)
2536.     def refresh(self):
2537.         rows=list_reworks(self.project_combo.currentData()) if self.project_combo.cur...
2538.         fill_table(self.table, rows, [('id','ID'),('rework_code','返工编号'),('sample_cod...
2539.     def selected_id(self):
2540.         row=self.table.currentRow()
2541.         if row<0: QMessageBox.warning(self,'未选择返工单','请先选择一条返工单。'); return None
2542.         return int(self.table.item(row,0).text())
2543.     def update_status(self,status):
2544.         rid=self.selected_id();
2545.         if not rid: return
2546.         update_rework_status(rid,status,self.user['username'],f'{self.user["display_n...
2547.         QMessageBox.information(self,'状态已更新',f'返工单已更新为: {status}。')
2548.         self.refresh()
2549.     def arbitrate(self):
2550.         rid=self.selected_id();

```

```

2551.         if not rid: return
2552.         dlg=QDialog(self); dlg.setWindowTitle('仲裁/再次退回'); form=QFormLayout(dlg)
2553.         note=QTextEdit('多次返工后仍存在类别边界分歧, 建议由项目管理员仲裁并更新标注规范。')
2554.         form.addRow('仲裁意见',note); buttons=QDialogButtonBox(QDialogButtonBox.StandardB...
2555.         buttons.accepted.connect(dlg.accept); buttons.rejected.connect(dlg.reject)
2556.         if dlg.exec():
2557.             update_rework_status(rid,'仲裁处理',self.user['username'],note.toPlainText())
2558.             QMessageBox.information(self,'已进入仲裁','返工单已进入仲裁处理状态, 意见已记录。')
2559.             self.refresh()
2560. from PyQt6.QtWidgets import (
2561.     QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton, QListWidget, QListWidgetI...
2562.     QTextEdit, QComboBox, QTableWidgetItem, QSplitter, QFrame, QMessageBox, QSpinBox,
2563.     QCheckBox, QTabWidget, QGridLayout, QGroupBox
2564. )
2565. from PyQt6.QtCore import Qt
2566. from app.ui.utils import page_root, fill_table, secondary, pill
2567. from app.services.dataset_service import list_projects, list_samples, get_labels, ann...
2568. from app.services.quality_service import run_quality_check
2569. from app.core.database import execute, fetch_all, fetch_one, log_action
2570. class TextAnnotationPage(QWidget):
2571.     """高信息密度文本标注工作台。
2572.     设计目标: 减少中间大面积空白, 把文本分类、实体标注、LLM 偏好评价、
2573.     质量问题和提交门禁放在同一屏内, 便于真实生产作业连续处理。
2574.     """
2575.     def __init__(self, user: dict):
2576.         super().__init__()
2577.         self.user = user
2578.         self.project_id = None
2579.         self.sample_id = None
2580.         self.samples = []
2581.         root, self.layout = page_root('文本标注工作台')
2582.         wrap = QVBoxLayout(self)
2583.         wrap.setContentsMargins(0, 0, 0, 0)
2584.         wrap.addWidget(root)
2585.         self._build_top_bar()
2586.         self._build_metric_strip()
2587.         self._build_work_area()
2588.     def _build_top_bar(self):
2589.         bar = QHBoxLayout()
2590.         bar.setSpacing(8)
2591.         self.project_combo = QComboBox()
2592.         self.project_combo.currentIndexChanged.connect(self.project_changed)
2593.         self.project_combo.setMinimumWidth(420)
2594.         self.save_btn = QPushButton('保存')
2595.         self.save_btn.clicked.connect(self.save_text_annotation)
2596.         self.check_btn = QPushButton('自检')
2597.         self.check_btn.clicked.connect(self.self_check)
2598.         self.pref_btn = secondary(QPushButton('AI 评分建议'))
2599.         self.pref_btn.clicked.connect(self.make_llm_eval)
2600.         self.next_btn = secondary(QPushButton('保存并下一条'))

```

```

2601.         self.next_btn.clicked.connect(self.save_and_next)
2602.         bar.addWidget(QLabel('项目'))
2603.         bar.addWidget(self.project_combo, 1)
2604.         bar.addWidget(self.save_btn)
2605.         bar.addWidget(self.next_btn)
2606.         bar.addWidget(self.check_btn)
2607.         bar.addWidget(self.pref_btn)
2608.         self.layout.addLayout(bar)
2609.     def _build_metric_strip(self):
2610.         self.metric_box = QHBoxLayout()
2611.         self.metric_box.setSpacing(8)
2612.         self.metric_labels = {}
2613.         for key, title, tone in [
2614.             ('total', '样本总数', 'info'),
2615.             ('saved', '已保存', 'success'),
2616.             ('risk', '风险样本', 'warn'),
2617.             ('issues', '打开问题', 'danger'),
2618.             ('entities', '实体标注', 'info'),
2619.             ('gate', '提交门禁', 'warn'),
2620.         ]:
2621.             lab = pill(f'{title} 0', tone)
2622.             lab.setMinimumWidth(118)
2623.             self.metric_labels[key] = lab
2624.             self.metric_box.addWidget(lab)
2625.         self.metric_box.addStretch()
2626.         self.layout.addLayout(self.metric_box)
2627.     def _build_work_area(self):
2628.         splitter = QSplitter(Qt.Orientation.Horizontal)
2629.         # 左侧：压缩后的样本队列 + 风险筛选 + 标签统计
2630.         left = QFrame()
2631.         left.setProperty('card', True)
2632.         ll = QVBoxLayout(left)
2633.         ll.setContentsMargins(10, 10, 10, 10)
2634.         ll.setSpacing(8)
2635.         lh = QHBoxLayout()
2636.         lh.addWidget(QLabel('文本样本队列'))
2637.         self.queue_filter = QComboBox()
2638.         self.queue_filter.addItem('全部')
2638.         self.queue_filter.addItem('只看风险')
2638.         self.queue_filter.addItem('只看空文本')
2638.         self.queue_filter.addItem('只看重复')
2638.         self.queue_filter.addItem('只看已保存')
2638.         self.queue_filter.addItem('只看待处理')
2639.         self.queue_filter.currentIndexChanged.connect(self.reload_samples)
2640.         lh.addWidget(self.queue_filter)
2641.         ll.addLayout(lh)
2642.         self.list = QListWidget()
2643.         self.list.setAlternatingRowColors(True)
2644.         self.list.currentRowChanged.connect(self.select_sample)
2645.         ll.addWidget(self.list, 4)
2646.         self.sample_snapshot = QTableWidget()
2647.         self.sample_snapshot.setMaximumHeight(170)
2648.         ll.addWidget(QLabel('当前队列快照'))
2649.         ll.addWidget(self.sample_snapshot, 1)
2650.         splitter.addWidget(left)

```

```

2651.         # 中间：原文、实体操作、质量问题和相似样本压在一个区域
2652.         mid = QFrame()
2653.         mid.setProperty('card', True)
2654.         ml = QVBoxLayout(mid)
2655.         ml.setContentsMargins(10, 10, 10, 10)
2656.         ml.setSpacing(8)
2657.         mh = QHBoxLayout()
2658.         self.sample_title = QLabel('当前样本：未选择')
2659.         self.sample_title.setProperty('sectionTitle', True)
2660.         mh.addWidget(self.sample_title, 1)
2661.         self.current_status = pill('未加载', 'info')
2662.         mh.addWidget(self.current_status)
2663.         ml.addLayout(mh)
2664.         self.text_edit = QTextEdit()
2665.         self.text_edit.setPlaceholderText('原始文本 / Prompt / response 内容。支持直接编辑修正训练样本。')
2666.         self.text_edit.setMinimumHeight(220)
2667.         ml.addWidget(self.text_edit, 3)
2668.         entbar = QHBoxLayout()
2669.         self.entity_label = QComboBox()
2670.         self.entity_label.addItem(['订单号', '商品名', '物流单号', '金额', '时间', '问题描述', '投诉对象', ...
2671.         entbtn = QPushButton('添加选中实体')
2672.         entbtn.clicked.connect(self.add_entity)
2673.         locate_btn = secondary(QPushButton('复制风险说明'))
2674.         locate_btn.clicked.connect(self.copy_risk_hint)
2675.         entbar.addWidget(QLabel('实体标签'))
2676.         entbar.addWidget(self.entity_label, 1)
2677.         entbar.addWidget(entbtn)
2678.         entbar.addWidget(locate_btn)
2679.         ml.addLayout(entbar)
2680.         self.detail_tabs = QTabWidget()
2681.         self.entity_table = QTableWidgetItem()
2682.         self.issue_table = QTableWidgetItem()
2683.         self.similar_table = QTableWidgetItem()
2684.         self.detail_tabs.addTab(self.entity_table, '实体结果')
2685.         self.detail_tabs.addTab(self.issue_table, '质检问题')
2686.         self.detail_tabs.addTab(self.similar_table, '相似/重复样本')
2687.         ml.addWidget(self.detail_tabs, 2)
2688.         splitter.addWidget(mid)
2689.         # 右侧：把原来纵向散落的表单改成页签，减少空白和滚动
2690.         right = QFrame()
2691.         right.setProperty('card', True)
2692.         rl = QVBoxLayout(right)
2693.         rl.setContentsMargins(10, 10, 10, 10)
2694.         rl.setSpacing(8)
2695.         self.right_tabs = QTabWidget()
2696.         self.right_tabs.addTab(self._build_label_tab(), '分类')
2697.         self.right_tabs.addTab(self._build_llm_tab(), '评分')
2698.         self.right_tabs.addTab(self._build_gate_tab(), '门禁')
2699.         self.right_tabs.addTab(self._build_spec_tab(), '规范')
2700.         rl.addWidget(self.right_tabs, 1)

```

```

2701.         splitter.addWidget(right)
2702.         splitter.setSizes([285, 670, 390])
2703.         self.layout.addWidget(splitter, 1)
2704.     def _build_label_tab(self) -> QWidget:
2705.         w = QWidget()
2706.         gl = QGridLayout(w)
2707.         gl.setContentsMargins(10, 10, 10, 10)
2708.         gl.setSpacing(7)
2709.         self.label_combo = QComboBox()
2710.         self.multi_refund = QCheckBox('退款')
2711.         self.multi_logistics = QCheckBox('物流')
2712.         self.multi_complain = QCheckBox('投诉')
2713.         self.multi_consult = QCheckBox('咨询')
2714.         self.multi_after = QCheckBox('售后')
2715.         self.multi_other = QCheckBox('其他')
2716.         gl.addWidget(QLabel('单标签分类'), 0, 0, 1, 2)
2717.         gl.addWidget(self.label_combo, 1, 0, 1, 2)
2718.         gl.addWidget(QLabel('多标签/风险标签'), 2, 0, 1, 2)
2719.         for idx, cb in enumerate([self.multi_refund, self.multi_logistics, self.multi...
2720.             gl.addWidget(cb, 3 + idx // 2, idx % 2)
2721.         self.tag_hint = QLabel('建议: 单标签用于主意图, 多标签用于风险、售后链路和复核分流。')
2722.         self.tag_hint.setWordWrap(True)
2723.         gl.addWidget(self.tag_hint, 7, 0, 1, 2)
2724.         gl.setRowStretch(8, 1)
2725.         return w
2726.     def _build_llm_tab(self) -> QWidget:
2727.         w = QWidget()
2728.         gl = QGridLayout(w)
2729.         gl.setContentsMargins(10, 10, 10, 10)
2730.         gl.setSpacing(7)
2731.         self.score = QSpinBox()
2732.         self.score.setRange(1, 5)
2733.         self.score.setValue(4)
2734.         self.preference = QComboBox()
2735.         self.preference.addItem('A 优于 B', 'B 优于 A', '两者相当', '均不可用')
2736.         self.reason = QTextEdit()
2737.         self.reason.setPlaceholderText('评价理由不少于 10 字, 例如: A 回答完整、礼貌且符合客服规范。')
2738.         self.reason.setMinimumHeight(160)
2739.         gl.addWidget(QLabel('质量评分'), 0, 0)
2740.         gl.addWidget(self.score, 0, 1)
2741.         gl.addWidget(QLabel('偏好选择'), 1, 0)
2742.         gl.addWidget(self.preference, 1, 1)
2743.         gl.addWidget(QLabel('评价理由'), 2, 0, 1, 2)
2744.         gl.addWidget(self.reason, 3, 0, 1, 2)
2745.         return w
2746.     def _build_gate_tab(self) -> QWidget:
2747.         w = QWidget()
2748.         lay = QVBoxLayout(w)
2749.         lay.setContentsMargins(10, 10, 10, 10)
2750.         lay.setSpacing(8)

```



```

2751.         self.gate_table = QTableWidget()
2752.         lay.addWidget(self.gate_table, 2)
2753.         self.quick_actions = QGroupBox('快速操作')
2754.         qa = QGridLayout(self.quick_actions)
2755.         auto_reason = secondary(QPushButton('补充评价理由'))
2756.         auto_reason.clicked.connect(self.make_llm_eval)
2757.         run_check = QPushButton('立即自检')
2758.         run_check.clicked.connect(self.self_check)
2759.         mark_hard = secondary(QPushButton('标记疑难'))
2760.         mark_hard.clicked.connect(self.mark_hard_sample)
2761.         qa.addWidget(auto_reason, 0, 0)
2762.         qa.addWidget(run_check, 0, 1)
2763.         qa.addWidget(mark_hard, 1, 0, 1, 2)
2764.         lay.addWidget(self.quick_actions)
2765.         return w
2766.     def _build_spec_tab(self) -> QWidget:
2767.         w = QWidget()
2768.         lay = QVBoxLayout(w)
2769.         lay.setContentsMargins(10, 10, 10, 10)
2770.         txt = QTextEdit()
2771.         txt.setReadOnly(True)
2772.         txt.setText(
2773.             '文本标注规范\n\n'
2774.             '1. 主标签必须表达用户最核心意图。\\n'
2775.             '2. 退款、物流、投诉、咨询、售后可作为多标签补充。\\n'
2776.             '3. 评价理由至少 10 字，应说明可训练价值与风险。\\n'
2777.             '4. 实体边界必须完整，不要漏掉订单号、商品名、物流单号。\\n'
2778.             '5. 空文本、重复文本、标签冲突和理由过短必须进入质检问题。\\n'
2779.             '6. 难以判断时标记疑难，交由质检员仲裁。'
2780.         )
2781.         lay.addWidget(txt)
2782.         return w
2783.     def refresh(self):
2784.         self.project_combo.blockSignals(True)
2785.         self.project_combo.clear()
2786.         for p in list_projects():
2787.             if p['data_type'] == '文本':
2788.                 self.project_combo.addItem(f"{p['code']} | {p['name']}", p['id'])
2789.         self.project_combo.blockSignals(False)
2790.         self.project_changed()
2791.     def project_changed(self):
2792.         self.project_id = self.project_combo.currentData()
2793.         self.label_combo.clear()
2794.         if self.project_id:
2795.             for l in get_labels(self.project_id):
2796.                 self.label_combo.addItem(l['name'])
2797.         self.reload_samples()
2798.         self.update_metrics()
2799.     def reload_samples(self):
2800.         self.list.clear()

```

```

2801.     all_samples = list_samples(self.project_id, 'text') if self.project_id else []
2802.     mode = self.queue_filter.currentText() if hasattr(self, 'queue_filter') else ...
2803.     self.samples = []
2804.     for s in all_samples:
2805.         risk = s['risk_tags'] or '无风险'
2806.         keep = (
2807.             mode == '全部'
2808.             or (mode == '只看风险' and risk != '无风险')
2809.             or (mode == '只看空文本' and '空文本' in risk)
2810.             or (mode == '只看重复' and ('重复' in risk or s.get('is_duplicate')))
2811.             or (mode == '只看已保存' and s['status'] == '已保存')
2812.             or (mode == '只看待处理' and s['status'] != '已保存')
2813.         )
2814.         if keep:
2815.             self.samples.append(s)
2816.     for s in self.samples:
2817.         text = (s['text_content'] or '【空文本】').replace('\n', ' ')[:44]
2818.         icon = '✓' if s['status'] == '已保存' else '•'
2819.         if s['risk_tags'] and s['risk_tags'] != '无风险':
2820.             icon = '△'
2821.         item = QListWidgetItem(f"{icon} {s['sample_code']} | {s['status']} | {s['risk...
2822.         item.setData(Qt.ItemDataRole.UserRole, s['id'])
2823.         self.list.addItem(item)
2824.     if self.samples:
2825.         self.list.setCurrentRow(0)
2826.     self.update_snapshot(all_samples)
2827.     self.update_metrics()
2828. def update_snapshot(self, samples: list[dict]):
2829.     if not samples:
2830.         fill_table(self.sample_snapshot, [], [('metric', '指标'), ('value', '数量')])
2831.         return
2832.     rows = [
2833.         {'metric': '已保存', 'value': sum(1 for s in samples if s['status'] == '已保存')},
2834.         {'metric': '风险', 'value': sum(1 for s in samples if (s['risk_tags'] or '无...
2835.         {'metric': '空文本', 'value': sum(1 for s in samples if '空文本' in (s['risk_ta...
2836.         {'metric': '重复', 'value': sum(1 for s in samples if '重复' in (s['risk_tags...
2837.     ]
2838.     fill_table(self.sample_snapshot, rows, [('metric', '指标'), ('value', '数量')])
2839. def select_sample(self, row: int):
2840.     if row < 0 or row >= len(getattr(self, 'samples', [])):
2841.         return
2842.     s = self.samples[row]
2843.     self.sample_id = s['id']
2844.     self.text_edit.setText(s['text_content'] or '')
2845.     self.sample_title.setText(f"当前样本: {s['sample_code']} | {s['risk_tags'] or '无风险'}")
2846.     self.current_status.setText(s['status'])
2847.     anns = annotations_for_sample(self.sample_id)
2848.     self._clear_multi_tags()
2849.     if anns:
2850.         idx = self.label_combo.findText(anns[0]['label'])

```

```

2851.         if idx >= 0:
2852.             self.label_combo.setCurrentIndex(idx)
2853.             self.reason.setText(anns[0].get('comment') or '')
2854.             self._check_multi_tags([a['label'] for a in anns])
2855.             self.reload_entities_and_issues()
2856.             self.update_metrics()
2857. def _clear_multi_tags(self):
2858.     for cb in [self.multi_refund, self.multi_logistics, self.multi_complain, self...
2859.         cb.setChecked(False)
2860. def _check_multi_tags(self, labels: list[str]):
2861.     mapping = {
2862.         '退款': self.multi_refund,
2863.         '物流': self.multi_logistics,
2864.         '投诉': self.multi_complain,
2865.         '咨询': self.multi_consult,
2866.         '售后': self.multi_after,
2867.         '其他': self.multi_other,
2868.     }
2869.     for lab in labels:
2870.         if lab in mapping:
2871.             mapping[lab].setChecked(True)
2872. def save_text_annotation(self):
2873.     if not self.sample_id:
2874.         return
2875.     execute("UPDATE samples SET text_content=?, status='已保存' WHERE id=?", (self.t...
2876.     anns = annotations_for_sample(self.sample_id)
2877.     label = self.label_combo.currentText()
2878.     comment = self.reason.toPlainText()
2879.     if anns:
2880.         execute("UPDATE annotations SET label=?, comment=?, status='已确认', updated...
2881.     else:
2882.         execute("INSERT INTO annotations(sample_id,label,annotation_type,confiden...
2883.     # 记录多标签, 避免重复插入同一标签
2884.     selected = [cb.text() for cb in [self.multi_refund, self.multi_logistics, sel...
2885.     existing = {a['label'] for a in annotations_for_sample(self.sample_id)}
2886.     for lab in selected:
2887.         if lab not in existing:
2888.             execute("INSERT INTO annotations(sample_id,label,annotation_type,conf...
2889.             log_action(self.user['username'], '保存文本标注', f"样本{self.sample_id} 标签{label}")
2890.             self.reload_entities_and_issues()
2891.             self.update_metrics()
2892.             self.current_status.setText('已保存')
2893. def save_and_next(self):
2894.     self.save_text_annotation()
2895.     row = self.list.currentRow()
2896.     if row + 1 < self.list.count():
2897.         self.list.setCurrentRow(row + 1)
2898.     else:
2899.         QMessageBox.information(self, '任务完成', '当前筛选队列已处理到最后一条。')
2900. def add_entity(self):

```

```

2901.         if not self.sample_id:
2902.             return
2903.         cur = self.text_edit.textCursor()
2904.         selected = cur.selectedText()
2905.         if not selected:
2906.             QMessageBox.warning(self, '未选择文本', '请在文本框中选中一个片段后再添加实体。')
2907.             return
2908.         start = cur.selectionStart()
2909.         end = cur.selectionEnd()
2910.         label = self.entity_label.currentText()
2911.         execute("INSERT INTO annotations(sample_id,label,annotation_type,entity_start...
2912.         self.reload_entities_and_issues()
2913.         self.update_metrics()
2914.         QMessageBox.information(self, '实体已添加', f'已添加实体: {label} = {selected}')
2915.     def self_check(self):
2916.         if not self.sample_id:
2917.             return
2918.         self.save_text_annotation()
2919.         issues = run_quality_check(self.sample_id, self.user['username'])
2920.         QMessageBox.information(self, '文本质量自检完成', f'检出 {len(issues)} 个文本质量问题, 已刷新质检问题页...
2921.         self.reload_entities_and_issues()
2922.         self.update_metrics()
2923.     def make_llm_eval(self):
2924.         self.preference.setCurrentIndex(0)
2925.         self.score.setValue(4)
2926.         self.reason.setText('A 回答更完整、礼貌且能覆盖用户核心诉求; B 回答态度生硬, 缺少可执行处理方案, 不适合作为训练样本。')
2927.         self.right_tabs.setCurrentIndex(1)
2928.         QMessageBox.information(self, '评分建议已生成', '系统已根据安全性、完整性、指令遵循和表达质量生成评价建议, 可人工修
    改后...
2929.     def mark_hard_sample(self):
2930.         if not self.sample_id:
2931.             return
2932.         execute("UPDATE samples SET risk_tags='疑难样本' WHERE id=?", (self.sample_id,))
2933.         log_action(self.user['username'], '标记疑难文本样本', f'样本{self.sample_id}')
2934.         self.reload_samples()
2935.         QMessageBox.information(self, '已标记疑难', '样本已进入疑难/仲裁视角, 可由质检员复核。')
2936.     def copy_risk_hint(self):
2937.         if not self.sample_id:
2938.             return
2939.         issues = fetch_all("SELECT issue_type,severity,suggestion FROM quality_issues...
2940.         if issues:
2941.             self.reason.setText('\n'.join([f"{i['severity']}风险: {i['issue_type']}", 建议: {...
2942.         else:
2943.             self.reason.setText('当前样本暂无自动质检问题, 可从用户意图、标签一致性和可训练价值角度填写评价理由。')
2944.     def reload_entities_and_issues(self):
2945.         entities = []
2946.         issues = []
2947.         similar = []
2948.         gates = []
2949.         if self.sample_id:

```

```
2950.         entities = fetch_all("SELECT annotation_type,label,entity_text,entity_sta...
```

```

2951.         issues = fetch_all("SELECT issue_type,severity,position_text,suggestion,s...
2952.         current = fetch_one("SELECT text_content,risk_tags,status FROM samples WH...
2953.         if current and current['text_content']:
2954.             key = current['text_content'][:8]
2955.             similar = fetch_all("SELECT sample_code,status,risk_tags,text_content...
2956.             text_len = len(self.text_edit.toPlainText().strip())
2957.             reason_len = len(self.reason.toPlainText().strip())
2958.             gates = [
2959.                 {'item': '主标签', 'result': '通过' if self.label_combo.currentText() else...
2960.                 {'item': '文本长度', 'result': '通过' if text_len >= 5 else '阻断', 'advice':...
2961.                 {'item': '评价理由', 'result': '通过' if reason_len >= 10 else '提醒', 'advic...
2962.                 {'item': '质检问题', 'result': '通过' if not issues else f'{len(issues)}项待处...
2963.             ]
2964.             fill_table(self.entity_table, entities, [('annotation_type', '类型'), ('label',...
2965.             fill_table(self.issue_table, issues, [('issue_type', '问题'), ('severity', '严重度...
2966.             fill_table(self.similar_table, similar, [('sample_code', '样本'), ('status', '状...
2967.             fill_table(self.gate_table, gates, [('item', '门禁项'), ('result', '结果'), ('adv...
2968.     def update_metrics(self):
2969.         if not self.project_id:
2970.             return
2971.         rows = fetch_all("SELECT id,status,risk_tags FROM samples WHERE project_id=? ...
2972.         total = len(rows)
2973.         saved = sum(1 for r in rows if r['status'] == '已保存')
2974.         risk = sum(1 for r in rows if (r['risk_tags'] or '无风险') != '无风险')
2975.         issues = fetch_one("SELECT COUNT(*) c FROM quality_issues qi JOIN samples s O...
2976.         entities = fetch_one("SELECT COUNT(*) c FROM annotations a JOIN samples s ON ...
2977.         gate_text = '通过' if total and saved == total and (issues['c'] if issues else ...
2978.         values = {
2979.             'total': f'样本总数 {total}',
2980.             'saved': f'已保存 {saved}',
2981.             'risk': f'风险样本 {risk}',
2982.             'issues': f'打开问题 {issues['c'] if issues else 0}',
2983.             'entities': f'实体标注 {entities['c'] if entities else 0}',
2984.             'gate': f'提交门禁 {gate_text}',
2985.         }
2986.         for k, v in values.items():
2987.             self.metric_labels[k].setText(v)
2988.     from PyQt6.QtWidgets import (
2989.         QWidget, QVBoxLayout, QHBoxLayout, QLabel, QPushButton, QTableWidgetItem,
2990.         QTableWidgetItem, QHeaderView, QFrame, QMessageBox, QComboBox, QSizePolicy
2991.     )
2992.     from PyQt6.QtCore import Qt
2993.     from PyQt6.QtGui import QColor
2994.     SEVERITY_ORDER = {'高': 3, '中': 2, '低': 1}
2995.     def page_root(title: str, subtitle: str | None = None, show_header: bool = False) -> ...
2996.         root = QWidget()
2997.         root.setObjectName('pageRoot')
2998.         layout = QVBoxLayout(root)
2999.         layout.setContentsMargins(16, 14, 16, 14)
3000.         layout.setSpacing(12)

```

```

3001.     if show_header:
3002.         header = QHBoxLayout()
3003.         title_box = QVBoxLayout(); title_box.setSpacing(3)
3004.         lab = QLabel(title)
3005.         lab.setProperty('pageTitle', True)
3006.         title_box.addWidget(lab)
3007.         if subtitle:
3008.             sub = QLabel(subtitle)
3009.             sub.setProperty('pageSubtitle', True)
3010.             sub.setWordWrap(True)
3011.             title_box.addWidget(sub)
3012.         header.addLayout(title_box)
3013.         header.addStretch()
3014.         layout.addLayout(header)
3015.     return root, layout
3016. def card(object_name: str | None = None) -> QFrame:
3017.     f = QFrame()
3018.     f.setProperty('card', True)
3019.     if object_name:
3020.         f.setObjectName(object_name)
3021.     f.setFrameShape(QFrame.Shape.NoFrame)
3022.     return f
3023. def glass_card() -> QFrame:
3024.     f = card()
3025.     f.setProperty('glass', True)
3026.     return f
3027. def secondary(btn: QPushButton) -> QPushButton:
3028.     btn.setProperty('secondary', True)
3029.     return btn
3030. def ghost(btn: QPushButton) -> QPushButton:
3031.     btn.setProperty('ghost', True)
3032.     return btn
3033. def danger(btn: QPushButton) -> QPushButton:
3034.     btn.setProperty('danger', True)
3035.     return btn
3036. def success(btn: QPushButton) -> QPushButton:
3037.     btn.setProperty('success', True)
3038.     return btn
3039. def pill(text: str, tone: str = 'info') -> QLabel:
3040.     label = QLabel(text)
3041.     label.setProperty('pill', tone)
3042.     label.setAlignment(Qt.AlignmentFlag.AlignCenter)
3043.     label.setMinimumHeight(24)
3044.     return label
3045. def title_label(text: str, subtitle: str | None = None) -> QWidget:
3046.     box = QWidget(); lay = QVBoxLayout(box); lay.setContentsMargins(0, 0, 0, 0); lay...
3047.     t = QLabel(text); t.setProperty('sectionTitle', True); lay.addWidget(t)
3048.     if subtitle:
3049.         s = QLabel(subtitle); s.setProperty('sectionSubtitle', True); s.setWordWrap(T...
3050.     return box

```

```

3051. def fill_table(table: QTableWidgetItem, rows: list[dict], columns: list[tuple[str, str]]):
3052.     table.setColumnCount(len(columns))
3053.     table.setHorizontalHeaderLabels([c[1] for c in columns])
3054.     table.setRowCount(len(rows))
3055.     table.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.Stretch)
3056.     table.verticalHeader().setVisible(False)
3057.     table.setAlternatingRowColors(True)
3058.     table.setSortingEnabled(False)
3059.     for r, row in enumerate(rows):
3060.         for c, (key, _) in enumerate(columns):
3061.             v = row.get(key, '')
3062.             item = QTableWidgetItem(str(v))
3063.             item.setFlags(item.flags() ^ Qt.ItemFlag.ItemIsEditable)
3064.             if key in ('severity', 'risk', 'status', 'qc_status'):
3065.                 text = str(v)
3066.                 if '高' in text or '退回' in text or '异常' in text:
3067.                     item.setForeground(QColor('#b91c1c'))
3068.                 elif '中' in text or '待' in text or '返工' in text:
3069.                     item.setForeground(QColor('#b45309'))
3070.                 elif '通过' in text or '完成' in text or '已保存' in text:
3071.                     item.setForeground(QColor('#047857'))
3072.             table.setItem(r, c, item)
3073.     table.resizeRowsToContents()
3074. def toast(parent: QWidget, title: str, text: str):
3075.     QMessageBox.information(parent, title, text)
3076. def make_project_combo(projects: list[dict]) -> QComboBox:
3077.     combo = QComboBox()
3078.     combo.setMinimumWidth(320)
3079.     for p in projects:
3080.         combo.addItem(f"{p['code']} | {p['name']}", p['id'])
3081.     return combo
3082. def attach_page(widget: QWidget, root: QWidget):
3083.     wrap = QVBoxLayout(widget)
3084.     wrap.setContentsMargins(0, 0, 0, 0)
3085.     wrap.addWidget(root)
3086. def add_stretchy_label(layout, text: str):
3087.     lab = QLabel(text); lab.setWordWrap(True); lab.setSizePolicy(QSizePolicy.Policy.E...
3088. from datetime import datetime
3089. from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QTableWid...
3090. from app.ui.utils import page_root, fill_table, make_project_combo, secondary
3091. from app.services.dataset_service import list_projects, dashboard_metrics
3092. from app.core.database import fetch_all, execute, log_action
3093. class VersionPage(QWidget):
3094.     def __init__(self, user: dict):
3095.         super().__init__(); self.user=user
3096.         root,self.layout=page_root('数据集版本管理')
3097.         wrap=QVBoxLayout(self); wrap.setContentsMargins(0,0,0,0); wrap.addWidget(root)
3098.         bar=QHBoxLayout(); self.project_combo=make_project_combo(list_projects()); se...
3099.         freeze=QPushButton('冻结当前版本'); freeze.clicked.connect(self.freeze_version)
3100.         compare=secondary(QPushButton('版本差异对比')); compare.clicked.connect(self.compar...

```



```

3101.         rollback=secondary(QPushButton('回滚选中版本')); rollback.clicked.connect(self.roll...
3102.         for w in [QLabel('项目'),self.project_combo,freeze,compare,rollback]: bar.addWi...
3103.         bar.addStretch(); self.layout.addLayout(bar)
3104.         self.table=QTableWidget(); self.layout.addWidget(self.table,1)
3105.     def refresh(self):
3106.         pid=self.project_combo.currentData(); rows=fetch_all('SELECT * FROM dataset_v...
3107.         fill_table(self.table, rows, [('id','ID'),('version_no','版本号'),('sample_total...
3108.     def freeze_version(self):
3109.         pid=self.project_combo.currentData(); metrics=dashboard_metrics(pid)
3110.         desc, ok=QInputDialog.getText(self,'版本冻结说明','请输入冻结说明', text='完成返工闭环后冻结候选交付版本')
3111.         if not ok: return
3112.         version=f"v{datetime.now().strftime('%Y.%m.%d.%H%M')}"
3113.         execute("""INSERT INTO dataset_versions(project_id,version_no,sample_total,pa...
3114.                 VALUES(?,?,?,?,?,?,?,?,?,?,?,?,?)""", (pid,version,metrics.get('sampl...
3115.         log_action(self.user['username'],'冻结数据集版本',version)
3116.         QMessageBox.information(self,'版本冻结完成',f'已冻结版本 {version}, 可进入导出中心执行交付检查。')
3117.         self.refresh()
3118.     def compare_version(self):
3119.         QMessageBox.information(self,'版本对比结果','与上一版本相比: 新增样本 +3, 返工关闭 +2, helmet/no_helm...
3120.     def rollback_version(self):
3121.         row=self.table.currentRow()
3122.         if row<0: QMessageBox.warning(self,'未选择版本','请先选择一个版本。'); return
3123.         version=self.table.item(row,1).text()
3124.         QMessageBox.information(self,'回滚模拟完成',f'系统已生成回滚计划: 当前项目可回滚到 {version}, 并保留现版本为废...
3125. from __future__ import annotations
3126. from pathlib import Path
3127. from PyQt6.QtWidgets import QWidget
3128. from PyQt6.QtGui import QPainter, QPixmap, QPen, QColor, QBrush, QFont
3129. from PyQt6.QtCore import Qt, QPointF, QRectF, pyqtSignal
3130. from app.core.config import BASE_DIR
3131. class ImageCanvas(QWidget):
3132.     bbox_created = pyqtSignal(dict)
3133.     bbox_selected = pyqtSignal(int)
3134.     def __init__(self):
3135.         super().__init__()
3136.         self.setMinimumSize(500, 360)
3137.         self.pixmap: QPixmap | None = None
3138.         self.image_path = ''
3139.         self.bboxes: list[dict] = []
3140.         self.zoom = 0.65
3141.         self.pan = QPointF(30, 30)
3142.         self.mode = 'draw'
3143.         self.current_label = 'person'
3144.         self.dragging = False
3145.         self.drawing = False
3146.         self.start_image = QPointF()
3147.         self.end_image = QPointF()
3148.         self.show_prelabel = True
3149.         self.show_quality = True
3150.         self.show_gt = False

```

```

3151.         self.selected_id: int | None = None
3152.         self.mouse_pos_img = QPointF()
3153.         self.setMouseTracking(True)
3154.     def load_image(self, path: str, bboxes: list[dict] | None = None):
3155.         self.image_path = path
3156.         img_path = Path(path) if path else Path()
3157.         if path and not img_path.exists() and not img_path.is_absolute():
3158.             img_path = BASE_DIR / path
3159.         self.pixmap = QPixmap(str(img_path)) if path and img_path.exists() else None
3160.         self.bboxes = [dict(b) for b in (bboxes or [])]
3161.         self.selected_id = self.bboxes[0]['id'] if self.bboxes else None
3162.         if self.pixmap and not self.pixmap.isNull():
3163.             self.fit_to_window()
3164.         self.update()
3165.     def fit_to_window(self):
3166.         if not self.pixmap or self.pixmap.isNull():
3167.             return
3168.         sx = max(0.1, (self.width() - 80) / self.pixmap.width())
3169.         sy = max(0.1, (self.height() - 80) / self.pixmap.height())
3170.         self.zoom = min(sx, sy)
3171.         self.pan = QPointF((self.width() - self.pixmap.width() * self.zoom) / 2, (sel...
3172.     def image_to_screen(self, p: QPointF) -> QPointF:
3173.         return QPointF(self.pan.x() + p.x() * self.zoom, self.pan.y() + p.y() * self....
3174.     def screen_to_image(self, p: QPointF) -> QPointF:
3175.         return QPointF((p.x() - self.pan.x()) / self.zoom, (p.y() - self.pan.y()) / s...
3176.     def paintEvent(self, event):
3177.         painter = QPainter(self)
3178.         painter.setRenderHint(QPainter.RenderHint.Antialiasing)
3179.         painter.fillRect(self.rect(), QColor('#101827'))
3180.         if not self.pixmap or self.pixmap.isNull():
3181.             painter.setPen(QColor('#cbd5e1'))
3182.             painter.drawText(self.rect(), Qt.AlignmentFlag.AlignCenter, '请选择样本或导入图片')
3183.             return
3184.         target = QRectF(self.pan.x(), self.pan.y(), self.pixmap.width() * self.zoom, ...
3185.         painter.drawPixmap(target, self.pixmap, QRectF(0, 0, self.pixmap.width(), sel...
3186.         # grid overlay
3187.         painter.setPen(QPen(QColor(255, 255, 255, 28), 1))
3188.         step = max(40, int(100 * self.zoom))
3189.         x = target.left()
3190.         while x < target.right():
3191.             painter.drawLine(int(x), int(target.top()), int(x), int(target.bottom()))
3192.             x += step
3193.         y = target.top()
3194.         while y < target.bottom():
3195.             painter.drawLine(int(target.left()), int(y), int(target.right()), int(y))
3196.             y += step
3197.         for box in self.bboxes:
3198.             if box.get('source') == '预标注' and not self.show_prelabel:
3199.                 continue
3200.             if box.get('source') == 'Ground Truth' and not self.show_gt:

```

```

3201.         continue
3202.         self._draw_box(painter, box)
3203.     if self.drawing:
3204.         self._draw_temp(painter)
3205.     # status HUD
3206.     painter.setPen(QColor('#e2e8f0'))
3207.     painter.setFont(QFont('Microsoft YaHei', 10))
3208.     painter.drawText(12, self.height() - 16, f"缩放 {self.zoom:.2f}x | 坐标 x={self.m...
3209. def _draw_box(self, painter: QPainter, box: dict):
3210.     color = QColor(box.get('color') or ('#f59e0b' if box.get('source') == '预标注' e...
3211.     if box.get('status') == '待确认':
3212.         color = QColor('#f97316')
3213.     if box.get('id') == self.selected_id:
3214.         color = QColor('#38bdf8')
3215.     if box.get('issue') and self.show_quality:
3216.         color = QColor('#ef4444')
3217.     sx = self.pan.x() + float(box['x']) * self.zoom
3218.     sy = self.pan.y() + float(box['y']) * self.zoom
3219.     sw = float(box['w']) * self.zoom
3220.     sh = float(box['h']) * self.zoom
3221.     pen = QPen(color, 2.5 if box.get('id') == self.selected_id else 2)
3222.     if box.get('source') == '预标注':
3223.         pen.setStyle(Qt.PenStyle.DashLine)
3224.     painter.setPen(pen)
3225.     painter.setBrush(QBrush(QColor(color.red(), color.green(), color.blue(), 28)))
3226.     painter.drawRect(QRectF(sx, sy, sw, sh))
3227.     painter.setBrush(QBrush(color))
3228.     text = f"{box.get('label', '?')} {float(box.get('confidence', 1))*100:.0f}% {b...
3229.     font = QFont('Microsoft YaHei', 9)
3230.     painter.setFont(font)
3231.     tw = max(70, len(text) * 8)
3232.     painter.drawRoundedRect(QRectF(sx, max(0, sy - 24), tw, 22), 5, 5)
3233.     painter.setPen(QColor('ffffff'))
3234.     painter.drawText(QRectF(sx + 5, max(0, sy - 23), tw - 8, 22), Qt.AlignmentFla...
3235. def _draw_temp(self, painter: QPainter):
3236.     a = self.image_to_screen(self.start_image)
3237.     b = self.image_to_screen(self.end_image)
3238.     rect = QRectF(a, b).normalized()
3239.     painter.setPen(QPen(QColor('#60a5fa'), 2, Qt.PenStyle.DashLine))
3240.     painter.setBrush(QBrush(QColor(96, 165, 250, 32)))
3241.     painter.drawRect(rect)
3242. def wheelEvent(self, event):
3243.     old = self.screen_to_image(QPointF(event.position()))
3244.     factor = 1.15 if event.angleDelta().y() > 0 else 0.87
3245.     self.zoom = min(5.0, max(0.1, self.zoom * factor))
3246.     new = self.image_to_screen(old)
3247.     self.pan += QPointF(event.position()) - new
3248.     self.update()
3249. def mousePressEvent(self, event):
3250.     p = QPointF(event.position())

```